

we have this dns resolver we are tyrint to develop, Give me code to implemnet the following feature which we ahve to addd, tell me which code you want to see so that you can add this feature.

Give me code file, test file, and explain how you did it so that we can add this feature.

"CNAME chain resolution — if `www.foo.com` is a CNAME to `foo.com`, follow the chain and return both records. Good bang-for-buck since it's a real DNS gap most naive resolvers miss."

Planning new test cases and preparing to build and verify.

Good catch scope — this one actually touches more than it looks, because your `resolve_from()` currently has a latent bug that only becomes visible once a name can legitimately have one record type but not another (which AAAA introduces immediately: plenty of hosts have an A but no AAAA). Let me build this in a sandbox against your actual files so I can verify it compiles and passes before handing it to you, rather than hand you unverified code.

Created 12 files, ran 10 commands, shared files

What changed, and why

7 files touched, 3 of them are real logic changes:

File	Change
<code>dns_msg.h</code>	Added <code>uint8_t aaaa[16]</code> to the <code>dns_answer rdata</code> union
<code>dns_msg.c</code>	<code>dns_build_response()</code> gets a third branch: 16-byte AAAA rdata
<code>resolver.c</code>	3 changes — see below
<code>resolver.h</code>	comment only
<code>test_dns.c</code>	+31 checks (146 total)
<code>fake_hierarchy.py</code>	restructured to be qtype-aware; added AAAA record

`test_resolver.p` +6 integration checks, AAAA decoding
`y`

The 3 changes in `resolver.c`

1. `dissect()` — parses AAAA RRs out of an answer section the same way it already parsed A (rdlength 16 instead of 4).
2. `dns_resolve()` — the `qtype` gate was `if (qtype != DNS_TYPE_A)`. Loosened to also accept `DNS_TYPE_AAAA`, everything else still gets a clean `NOTIMP`.
3. The NODATA fix in `resolve_from()` — this is the one worth your attention. When a server answers with `NOERROR`, no matching record, no CNAME, and no delegation (`ref.count == 0`), the old code unconditionally `return -1`, which your `dns_resolve()` turns into `SERVFAIL`. That path was never reachable before, because every name in your test zone had exactly the one record type anyone ever asked for. AAAA makes it trivially reachable: any A-only host queried for AAAA now hits it. I reproduced this live — reverted just that one guard, rebuilt, and queried `host.example.net` for AAAA:

AAAA query for an A-only host, WITHOUT the fix -> rcode 2 (SERVFAIL)

Real resolvers correctly return `NOERROR` with zero answers here (NODATA — RFC 2308). `SERVFAIL` tells the client "ask someone else," which is wrong and would make every dual-stack client (i.e. basically everything since ~2015) retry and stall on any of your IPv4-only hosts.

Verified, not asserted

- `make test` equivalent: 146/146 unit checks under ASan+UBSan.
- `make offline` equivalent: 34/34 integration checks against a real fake root→TLD→auth chain, including AAAA through the referral chain, AAAA through a CNAME, and the NODATA case.
- Reverted the fix, rebuilt, reproduced the `SERVFAIL` bug live, then restored it and reran everything clean.

What I deliberately left alone

- `resolver_stub.c` — still A-only. It exists purely to isolate the AF_XDP-vs-UDP transport benchmark from resolution latency; giving it AAAA support wouldn't serve that purpose and would be scope creep.
- `resolve_ns_addr()` — still resolves NS hostnames via `DNS_TYPE_A` only. That's correct: your upstream transport is IPv4 sockets (`AF_INET` in `query_server`) regardless of what the *client* asked for, so NS glue resolution has nothing to do with the client's `qtype`.

- NODATA responses still aren't cached (same gap as your existing negative-caching design, which only caches explicit NXDOMAIN). Repeated AAAA-miss queries against an A-only host will re-walk the hierarchy every time. Worth fixing if you want it, but it's a separate, smaller feature than "add AAAA."

Dns msg
H
Dns msg
C
Resolver
C
Resolver
H
Test dns
C
Fake hierarchy
PY
Test resolver
PY

is there zero copy and Retry/timeout logic for upstream queries (if you're doing the forwarding design) — real resolvers retry a query to a different upstream server (or resend) if no answer comes within some timeout. Good for demonstrating resilience under packet loss.

*

Timing the recovery and coding retry-loop logic in C.

Two separate questions — let me answer them separately since one's already done and one isn't.

Zero-copy: already there, nothing to add. `xsk_setup.c` negotiates `XDP_ZEROCOPY` and falls back to `XDP_COPY` if the driver doesn't support it:

```
c
cfg.bind_flags = want_zerocopy ? XDP_ZEROCOPY : XDP_COPY;
...
if (ret == -EOPNOTSUPP && want_zerocopy) {
    fprintf(stderr, "WARN: zero-copy unsupported on %s, falling back to
XDP_COPY\n", ...);
    cfg.bind_flags &= ~XDP_ZEROCOPY;
```

```
...  
}
```

`xsk_env_is_zerocopy()` reports which mode actually got negotiated, and `af_xdp_user.c -c/--copy` lets you force `XDP_COPY` specifically so you can A/B it. Nothing missing here.

Retry/timeout — you're right, this one's a real gap. Currently `query_server()` sends once to a candidate, blocks up to 2s, and `resolve_from()` moves to the *next candidate in the referral's server list* on failure — but if a level only has one candidate (the common case: one NS, one glue address), a single lost UDP packet (query *or* response) fails that whole resolution with no second attempt at all. Let me build and verify a real fix rather than just describing one.

pasted

now we have to implement the follwoing feature:

- EDNS(0) support — lets you negotiate larger UDP payload sizes (avoiding unnecessary truncation) and is a genuinely modern DNS feature that's usually skipped in a first pass. Also improves realism of your benchmark since real-world resolvers almost all send EDNS0.

Give me all code files and testing files to complete adding this feature, and also Also give me all commands to test this. and explain what fucntionality you added and how

Drafting README updates and manual dig-based verification steps.

EDNS(o) support — implementation

What EDNS(o) actually changes here

Without EDNS, DNS-over-UDP is hard-capped at 512 bytes (RFC 1035). A client that supports EDNS0 (RFC 6891) says so by attaching a pseudo-RR of type `OPT` to the additional section of its query. That `OPT` record's "class" field carries the UDP payload size the client is willing to receive, and its "TTL" field is repurposed to carry a version number, an extended RCODE, and a `DO` (DNSSEC OK) flag bit.

A server that understands this should:

1. Detect the OPT record in the query.
2. Build its UDP response up to the negotiated size instead of always truncating at 512.
3. Echo its own OPT record back, advertising what *it's* willing to receive.

That's what's added below — nothing DNSSEC-related (DO is parsed but ignored, as before), nothing about EDNS options (cookies, NSID, etc. — rdlength is always 0 on our side).

Design choice: the server advertises and holds itself to 1232 bytes (the 2020 DNS-flag-day recommendation — the value modern `dig`, `bind`, `unbound`, `knot` all default to), even if a client claims it can take more. This avoids IP fragmentation while still being a large, real improvement over the old fixed 512-byte ceiling. TCP is untouched — its own 2-byte length prefix is the only size limit, EDNS payload size is UDP-only per spec.

Files changed

`dns_iface.h`

`c`

```
/* 512 = classic UDP DNS ceiling without EDNS0. With EDNS0 negotiated,
 * UDP responses can run up to DNS_EDNS_OUR_UDP_PAYLOAD (1232, see
 * dns_msg.h) instead -- this is sized for that plus header/OPT
 * headroom, and stays comfortably inside one UMEM frame (4096) once
 * L2/L3/L4 headers are added. */
#define DNS_MAX_MSG_LEN 1500
```

`c`

```
#include <stdbool.h>
```

(add near the top, alongside `<stdint.h>`, `<stddef.h>`)

`c`

```
/* Called from N worker threads at once, so it must be thread safe.
 * ...
 * is_udp tells the builder whether to apply any UDP size ceiling at
 * all (classic 512, or the EDNS-negotiated figure) -- TCP has no such
 * ceiling, its own length prefix is the only limit. */
int dns_resolve_query(const uint8_t *query, size_t query_len,
                     uint8_t *resp, size_t resp_cap, size_t *resp_len,
```

```
bool is_udp);
```

dns_msg.h

Add near the type/class constants:

```
c
#define DNS_TYPE_OPT          41      /* EDNS0 pseudo-RR, RFC 6891 */

#define DNS_EDNS_VERSION      0       /* the only version we understand
*/
#define DNS_EDNS_MIN_PAYLOAD  512     /* RFC 6891 floor -- a requestor
    * asking for less is clamped up */
#define DNS_EDNS_OUR_UDP_PAYLOAD 1232 /* what we advertise, and the
    * ceiling we hold OURSELVES to
    * even if a client claims to
    * accept more -- the 2020 DNS
    * flag day value, chosen to stay
    * clear of IP fragmentation */
#define DNS_CLASSIC_UDP_LIMIT 512     /* RFC 1035 ceiling, used
    * verbatim when the query carries
    * no OPT record at all */
#define DNS_OPT_RR_LEN        11     /* root name(1)+type(2)+class(2)
    * +ttl(4)+rdlength(2), no options */
```

Extend dns_query_info:

```
c
struct dns_query_info {
    struct dns_header hdr;
    struct dns_question q;
    bool recursion_desired;

    /* EDNS0 (RFC 6891), parsed from an OPT RR in the additional
    * section if the query carried one. */
    bool has_edns;
    uint16_t edns_udp_payload; /* the OPT RR's "class" field */
    uint8_t edns_version;      /* top byte of the "ttl" field */
    bool edns_do;              /* DNSSEC-OK bit; accepted, ignored --
    * we don't do DNSSEC validation */
};
```

Update the two prototypes:

```
c
int dns_build_response(const uint8_t *query, size_t query_len,
    const struct dns_query_info *q,
```

```

        const struct dns_answer *ans, size_t nans, int rcode,
        bool is_udp,
        uint8_t *out, size_t cap, size_t *out_len);

/* edns_udp_payload == 0 means "don't attach an OPT record at all". */
int dns_build_query(uint16_t id, const char *name, uint16_t qtype, bool
rd,
        uint16_t edns_udp_payload,
        uint8_t *out, size_t cap, size_t *out_len);

```

dns_msg.c

1. Parse EDNS out of a query — in dns_parse_query, right before return 0;:

c

```

/* EDNS0 (RFC 6891): an OPT pseudo-RR in the additional section.
 * This resolver only ever supports one question per message, so
 * we only look at the first additional record. A malformed or
 * absent one just means "no EDNS" -- it does not fail the whole
 * query, since EDNS is optional by definition. */
if (out->hdr.arcount >= 1) {
    struct dns_rr opt;
    size_t opt_next;
    size_t arr_off = after + 4;

    if (dns_parse_rr(msg, len, arr_off, &opt, &opt_next) == 0 &&
        opt.type == DNS_TYPE_OPT && opt.name[0] == '\\0') {
        out->has_edns = true;
        out->edns_udp_payload = opt.rclass;
        out->edns_version = (uint8_t)((opt.ttl >> 16) & 0xFF);
        out->edns_do = (opt.ttl & 0x8000) != 0;
    }
}

out->recursion_desired = (out->hdr.flags & DNS_FLAG_RD) != 0;
return 0;
}

```

2. Build a response with the new size negotiation and OPT echo. Replace

dns_build_response entirely:

c

```

int dns_build_response(const uint8_t *query, size_t query_len,
        const struct dns_query_info *q,
        const struct dns_answer *ans, size_t nans, int rcode,
        bool is_udp,
        uint8_t *out, size_t cap, size_t *out_len)

```

```

{
    size_t o = DNS_HDR_LEN, emitted = 0;
    bool truncated = false;
    uint16_t flags;
    uint16_t arcount = 0;

    if (cap < DNS_HDR_LEN)
        return -1;
    if (q->q.raw_off + q->q.raw_len > query_len)
        return -1;
    if (o + q->q.raw_len > cap)
        return -1;

    memcpy(out + o, query + q->q.raw_off, q->q.raw_len);
    o += q->q.raw_len;

    /* EDNS0 (RFC 6891). No OPT from the client -> stay at the
     * classic 512 byte ceiling; that limit predates EDNS and a
     * client that never asked for more can't be assumed to handle
     * more. A client that DOES send OPT gets up to
     * DNS_EDNS_OUR_UDP_PAYLOAD, clamped down further if it asked for
     * less -- but never further UP than our own advertised figure,
     * regardless of how large the client claims to accept, so we
     * never risk IP fragmentation on our own reply. TCP has no such
     * ceiling at all: its own 2 byte length prefix is the only
     * limit, so is_udp short-circuits every bit of this. */
    size_t effective_cap = cap;

    if (is_udp) {
        effective_cap = DNS_CLASSIC_UDP_LIMIT;

        if (q->has_edns) {
            uint16_t theirs = q->edns_udp_payload;

            if (theirs < DNS_EDNS_MIN_PAYLOAD)
                theirs = DNS_EDNS_MIN_PAYLOAD;
            effective_cap = theirs < DNS_EDNS_OUR_UDP_PAYLOAD
                ? theirs : DNS_EDNS_OUR_UDP_PAYLOAD;
        }
        if (effective_cap > cap)
            effective_cap = cap;
    }

    /* Reserve room for our own OPT record up front so the answer
     * loop below never eats the space it needs. */
    size_t ans_cap = effective_cap;

```



```

if (q->has_edns) {
    ans_cap = effective_cap > DNS_OPT_RR_LEN
        ? effective_cap - DNS_OPT_RR_LEN : o;
}

for (size_t i = 0; i < nans && rcode == DNS_RCODE_NOERROR; i++) {
    size_t start = o, rdlen_at, n;

    if (dns_name_equal(ans[i].owner, q->q.name)) {
        if (o + 2 > ans_cap)
            goto full;
        out[o++] = 0xC0;

        out[o++] = 0x0C;
    } else {
        if (dns_name_encode(ans[i].owner, out + o, ans_cap - o,
&n))
            goto full;
        o += n;
    }

    if (o + 10 > ans_cap)
        goto full;

    wr16(out + o, ans[i].type); o += 2;
    wr16(out + o, DNS_CLASS_IN); o += 2;
    wr32(out + o, ans[i].ttl); o += 4;
    rdlen_at = o; o += 2;

    if (ans[i].type == DNS_TYPE_A) {
        if (o + 4 > ans_cap)
            goto full;
        memcpy(out + o, &ans[i].rdata.a, 4);
        o += 4;
        wr16(out + rdlen_at, 4);
    } else if (ans[i].type == DNS_TYPE_AAAA) {
        if (o + 16 > ans_cap)
            goto full;
        memcpy(out + o, ans[i].rdata.aaaa, 16);
        o += 16;
        wr16(out + rdlen_at, 16);
    } else if (ans[i].type == DNS_TYPE_CNAME) {
        if (dns_name_encode(ans[i].rdata.cname, out + o,
            ans_cap - o, &n))
            goto full;
        o += n;
    }
}

```

```

        wr16(out + rdlen_at, (uint16_t)n);
    } else {
        o = start;
        continue;
    }

    emitted++;
    continue;
full:
    o = start;
    truncated = true;
    break;
}

/* Echo an OPT record back whenever the query carried one --
 * over UDP AND over TCP (a TCP client can send EDNS purely for
 * version negotiation; the size field just isn't meaningful
 * there). Space for it was already reserved above for the UDP
 * case, and for TCP effective_cap == cap so there's essentially
 * always room. */
if (q->has_edns && o + DNS_OPT_RR_LEN <= effective_cap) {
    out[o++] = 0x00;           //root name
    wr16(out + o, DNS_TYPE_OPT); o += 2;
    wr16(out + o, DNS_EDNS_OUR_UDP_PAYLOAD); o += 2;
    wr32(out + o, 0); //ext-rcode=0, version=0, flags=0 (no DO)
    o += 4;
    wr16(out + o, 0); //rdlength: we carry no options
    o += 2;
    arcount = 1;
}

flags = DNS_FLAG_QR | DNS_FLAG_RA;

if (q->recursion_desired)
    flags |= DNS_FLAG_RD;
if (truncated)
    flags |= DNS_FLAG_TC;
flags |= (uint16_t)(rcode & DNS_RCODE_MASK);

wr16(out + 0, q->hdr.id);
wr16(out + 2, flags);
wr16(out + 4, 1);
wr16(out + 6, (uint16_t)emitted);
wr16(out + 8, 0);
wr16(out + 10, arcount);

```

```

    *out_len = 0;
    return 0;
}

```

3. Let outgoing (upstream) queries carry EDNSo too. Replace dns_build_query:

```

c
int dns_build_query(uint16_t id, const char *name, uint16_t qtype, bool
rd,
                    uint16_t edns_udp_payload, uint8_t *out, size_t cap,
                    size_t *out_len)
{
    size_t n, o;

    if (cap < DNS_HDR_LEN)
        return -1;

    wr16(out + 0, id);
    wr16(out + 2, rd ? DNS_FLAG_RD : 0);
    wr16(out + 4, 1);
    wr16(out + 6, 0);
    wr16(out + 8, 0);
    wr16(out + 10, edns_udp_payload ? 1 : 0);

    if (dns_name_encode(name, out + DNS_HDR_LEN, cap - DNS_HDR_LEN, &n))
        return -1;

    o = DNS_HDR_LEN + n;
    if (o + 4 > cap)
        return -1;

    wr16(out + o, qtype); o += 2;
    wr16(out + o, DNS_CLASS_IN); o += 2;

    if (edns_udp_payload) {
        if (o + DNS_OPT_RR_LEN > cap)
            return -1;
        out[o++] = 0x00; //root name
        wr16(out + o, DNS_TYPE_OPT); o += 2;
        wr16(out + o, edns_udp_payload); o += 2;

        wr32(out + o, 0); //ext-rcode/version/flags = 0
        o += 4;
        wr16(out + o, 0); //rdlength: no options
        o += 2;
    }
}

```

```

        *out_len = 0;
        return 0;
}

```

dns_backend.c

```

c
int dns_resolve_query(const uint8_t *query, size_t query_len,
                      uint8_t *resp, size_t resp_cap, size_t *resp_len,
                      bool is_udp)
{
    struct dns_query_info q;
    struct dns_answer ans[DNS_MAX_ANSWERS];
    size_t nans = 0;
    int rcode = DNS_RCODE_SERVFAIL;

    if (dns_parse_query(query, query_len, &q))
        return DNS_DROP;

    if (q.q.qclass != DNS_CLASS_IN)
        rcode = DNS_RCODE_NOTIMP;
    else
        dns_resolve(q.q.name, q.q.qtype, ans, DNS_MAX_ANSWERS,
                   &nans, &rcode);

    if (dns_build_response(query, query_len, &q, ans, nans, rcode,
                           is_udp,
                           resp, resp_cap, resp_len))
        return DNS_DROP;

    return DNS_OK;
}

```

resolver_stub.c

Just needs the matching signature (it doesn't use `dns_msg.c` at all, so it never builds an OPT record — documented as-is, it exists purely for transport isolation):

```

c
int dns_resolve_query(const uint8_t *query, size_t query_len,
                      uint8_t *resp, size_t resp_cap, size_t *resp_len,
                      bool is_udp)
{
    (void)is_udp;    /* the stub answers identically either way -- it
                     * isolates the transport layer, it doesn't

```

```

        * exercise EDNS/TCP size negotiation. */
    ... /* unchanged body */
}

```

af_xdp_user.c — worker_main()

```

c
    rc = dns_resolve_query(job->query, job->query_len,
                           job->resp, sizeof(job->resp), &out_len,
                           true);    /* UDP */

```

(no other change needed — dns_job.query[]/resp[] are already sized off

DNS_MAX_MSG_LEN, which just grew from 512 to 1500 in dns_iface.h; 42-byte

headers + 1500 = 1542 < FRAME_SIZE(4096), so it's still safely inside one UMEM frame.)

server.c — worker_main()

```

c
    rc = dns_resolve_query(job->msg, job->len, resp, sizeof(resp),
                           &resp_len, true);    /* UDP */

```

dns_tcp.c — conn_main()

```

c
    int rc = dns_resolve_query(query, mlen, resp,
                                DNS_MAX_TCP_MSG_LEN, &resp_len,
                                false); /* TCP: no UDP size cap */

```

resolver.c — send EDNSo upstream too

```

c
static int query_server(const char *server_ip, const char *name, uint16_t
qtype,
                        uint8_t *resp, size_t resp_cap)
{
    ...
    if (dns_build_query(txid, name, qtype, false,
DNS_EDNS_OUR_UDP_PAYLOAD,
                        qbuf, sizeof(qbuf), &qlen))
        return -1;
    ...
}

```

Nothing else in `resolver.c` needs to change: `dissect()`'s additional-section loop already only keeps A glue records and silently continues past anything else (including OPT), so upstream OPT records were already handled gracefully.

Unit tests — `test_dns.c`

Update the query-builder helper and add an EDNS variant:

```
c
static size_t make_query(uint8_t *b, const char *name, uint16_t qtype)
{
    size_t n;

    if (dns_build_query(0x1234, name, qtype, true, 0, b, 512, &n))
        return 0;
    return n;
}

static size_t make_query_edns(uint8_t *b, const char *name, uint16_t
qtype,
                               uint16_t edns_payload)
{
    size_t n;

    if (dns_build_query(0x1234, name, qtype, true, edns_payload, b, 512,
&n))
        return 0;
    return n;
}
```

In `test_build_query()`, add 0 as the new argument to both existing `dns_build_query` calls, then append:

```
c
    CHECK(h.arcount == 0, "arcount %u when no EDNS requested",
h.arcount);
    ...
    //EDNS0: an OPT record appears in the additional section
    CHECK(dns_build_query(0xCAFE, "example.com", DNS_TYPE_A, true,
        DNS_EDNS_OUR_UDP_PAYLOAD, b, sizeof(b), &n) == 0,
        "edns build_query failed");
    CHECK(dns_parse_header(b, n, &h) == 0, "edns header unparseable");
    CHECK(h.arcount == 1, "edns arcount %u, want 1", h.arcount);
    CHECK(n == 12 + 13 + 4 + DNS_OPT_RR_LEN,
        "edns query length %zu", n);
```

```

    struct dns_rr rr;
    size_t off;

    CHECK(dns_skip_question(b, n, 12, &off) == 0, "edns question
unparseable");
    CHECK(dns_parse_rr(b, n, off, &rr, &off) == 0, "edns opt
unparseable");
    CHECK(rr.type == DNS_TYPE_OPT, "opt type %u", rr.type);
    CHECK(rr.name[0] == '\\0', "opt name not root: '%s'", rr.name);
    CHECK(rr.rclass == DNS_EDNS_OUR_UDP_PAYLOAD, "opt payload %u",
rr.rclass);

    struct dns_query_info qi;

    CHECK(dns_parse_query(b, n, &qi) == 0, "edns self-parse failed");
    CHECK(qi.has_edns, "self-parsed query missing has_edns");
    CHECK(qi.edns_udp_payload == DNS_EDNS_OUR_UDP_PAYLOAD, "self-parsed
payload");
    CHECK(!qi.edns_do, "self-parsed DO bit set unexpectedly");

    CHECK(dns_build_query(1, "example.com", DNS_TYPE_A, false,
DNS_EDNS_OUR_UDP_PAYLOAD, b, 12 + 13 + 4 + 2, &n)
!= 0,
        "undersized edns buffer accepted");

```

In test_parse_query(), add:

```

c
    //no additional section at all: has_edns stays false
    {
        uint8_t pq[512];
        size_t pn = make_query(pq, "example.com", DNS_TYPE_A);
        struct dns_query_info pinfo;

        CHECK(dns_parse_query(pq, pn, &pinfo) == 0, "plain reparse
failed");
        CHECK(!pinfo.has_edns, "plain query should not have has_edns
set");
    }

    //EDNS0 with the DO bit set
    {
        uint8_t eb[512];
        size_t en;
        struct dns_query_info einfo;

```

```

        CHECK(dns_build_query(0x2222, "example.com", DNS_TYPE_A, true,
                               4096, eb, sizeof(eb), &en) == 0,
              "do-bit query build failed");

        /* DO is bit 15 of the OPT RR's TTL field, which starts
         * 12(hdr)+13(qname)+4(qtype/qclass)+1(root name)+2(type)
         * +2(class) = 34 bytes in. */
        eb[34] |= 0x80;

        CHECK(dns_parse_query(eb, en, &einfo) == 0, "do-bit parse
failed");

        CHECK(einfo.has_edns, "do-bit query missing has_edns");
        CHECK(einfo.edns_do, "DO bit not observed");
        CHECK(einfo.edns_udp_payload == 4096, "do-bit payload");
    }

```

In `test_build_response()`, every existing call to `dns_build_response(...)` gains `true`, right before the output buffer argument (9 call sites — e.g. `dns_build_response(q, qlen, &info, ans, 1, DNS_RCODE_NOERROR, true, out, sizeof(out), &olen)`), preserving the old classic-512 behavior since none of those `info` structs have `has_edns` set.

New dedicated test function — add and call from `main()`:

```

c
static void test_edns(void)
{
    uint8_t eq[512], eout[2048];
    struct dns_query_info einfo, classic_info;
    struct dns_answer eans[3];
    size_t eqlen, eolen;
    char la[64], lb[64], lc[64], ld[64], name1[160], name2[160];

    /* Same trick as fake_hierarchy.py's BIG_HOP chain: two max-length
     * (63 byte) labels per hop so the chain is ~130-140 bytes per
     * name and ~600 bytes total across two CNAME hops -- comfortably
     * over the classic 512 ceiling, comfortably under 1232. */
    memset(la, 'a', 63); la[63] = '\0';
    memset(lb, 'b', 63); lb[63] = '\0';
    memset(lc, 'c', 63); lc[63] = '\0';
    memset(ld, 'd', 63); ld[63] = '\0';
    snprintf(name1, sizeof(name1), "%s.%s.test", la, lb);
    snprintf(name2, sizeof(name2), "%s.%s.test", lc, ld);

```



```

    eqlen = make_query_edns(eq, name1, DNS_TYPE_A,
DNS_EDNS_OUR_UDP_PAYLOAD);
    CHECK(eqlen > 0, "edns make_query failed");
    CHECK(dns_parse_query(eq, eqlen, &einfo) == 0, "edns query parse
failed");
    CHECK(einfo.has_edns, "has_edns not set");
    CHECK(einfo.edns_udp_payload == DNS_EDNS_OUR_UDP_PAYLOAD, "edns
payload");
    CHECK(!einfo.edns_do, "DO bit set when not requested");

    memset(eans, 0, sizeof(eans));
    eans[0].type = DNS_TYPE_CNAME;
    eans[0].ttl = 300;
    snprintf(eans[0].owner, sizeof(eans[0].owner), "%s", name1);
    snprintf(eans[0].rdata.cname, sizeof(eans[0].rdata.cname), "%s",
name2);
    eans[1].type = DNS_TYPE_CNAME;

    eans[1].ttl = 300;
    snprintf(eans[1].owner, sizeof(eans[1].owner), "%s", name2);
    snprintf(eans[1].rdata.cname, sizeof(eans[1].rdata.cname),
"final.test");
    eans[2].type = DNS_TYPE_A;
    eans[2].ttl = 300;
    snprintf(eans[2].owner, sizeof(eans[2].owner), "final.test");
    inet_pton(AF_INET, "10.0.0.9", &eans[2].rdata.a);

    classic_info = einfo;
    classic_info.has_edns = false;

    //without EDNS (classic 512), this chain must not fit
    memset(eout, 0xFF, sizeof(eout));
    CHECK(dns_build_response(eq, eqlen, &classic_info, eans, 3,
DNS_RCODE_NOERROR, true, eout, sizeof(eout),
&eolen) == 0, "classic chain build failed");
    CHECK(rdl6(eout + 2) & DNS_FLAG_TC, "classic chain should
truncate");
    CHECK(rdl6(eout + 6) < 3, "classic chain ancourt %u", rdl6(eout +
6));
    CHECK(rdl6(eout + 10) == 0, "classic response should carry no OPT");

    //with EDNS advertised at 1232, the same chain fits whole
    memset(eout, 0xFF, sizeof(eout));
    CHECK(dns_build_response(eq, eqlen, &einfo, eans, 3,
DNS_RCODE_NOERROR,
true, eout, sizeof(eout), &eolen) == 0,
"edns chain build failed");

```

```

CHECK(!(rd16(eout + 2) & DNS_FLAG_TC), "edns chain should not
truncate");
CHECK(rd16(eout + 6) == 3, "edns chain ancount %u", rd16(eout + 6));
CHECK(rd16(eout + 10) == 1, "edns chain arcount (OPT) %u", rd16(eout
+ 10));

//walk to the OPT record and check its shape
{
    struct dns_rr rr;

    size_t off;

    CHECK(dns_skip_question(eout, eolen, 12, &off) == 0,
          "edns response question unparseable");
    for (int i = 0; i < 3; i++) {
        struct dns_rr skip_rr;

        CHECK(dns_parse_rr(eout, eolen, off, &skip_rr, &off) ==
0,
              "edns answer %d unparseable", i);
    }
    CHECK(dns_parse_rr(eout, eolen, off, &rr, &off) == 0,
          "opt record unparseable");
    CHECK(rr.type == DNS_TYPE_OPT, "opt type %u", rr.type);
    CHECK(rr.name[0] == '\\0', "opt name not root: '%s'", rr.name);
    CHECK(rr.rclass == DNS_EDNS_OUR_UDP_PAYLOAD,
          "opt advertised payload %u", rr.rclass);
    CHECK(rr.rdlength == 0, "opt rdlength %u, want 0",
rr.rdlength);
}

//a client claiming only 700 bytes is held to 700, not 1232
{
    struct dns_query_info small_edns = einfo;

    small_edns.edns_udp_payload = 700;
    memset(eout, 0xFF, sizeof(eout));
    CHECK(dns_build_response(eq, eqlen, &small_edns, eans, 3,
        DNS_RCODE_NOERROR, true, eout,
        sizeof(eout), &eolen) == 0,
          "small edns chain build failed");
    CHECK(eolen <= 700, "small edns response %zu bytes, want
<=700",
          eolen);
}

```

```

//TCP ignores EDNS size negotiation entirely: no cap even without
EDNS...
memset(eout, 0xFF, sizeof(eout));
CHECK(dns_build_response(eq, eqlen, &classic_info, eans, 3,
                        DNS_RCODE_NOERROR, false, eout, sizeof(eout),
                        &eolen) == 0, "tcp chain build failed");
CHECK(!(rd16(eout + 2) & DNS_FLAG_TC), "tcp chain should not
truncate");
CHECK(rd16(eout + 6) == 3, "tcp chain amount %u", rd16(eout + 6));
CHECK(rd16(eout + 10) == 0, "tcp response should carry no OPT");

//...but OPT is still echoed over TCP if the client sent one
(version ack)
memset(eout, 0xFF, sizeof(eout));
CHECK(dns_build_response(eq, eqlen, &einfo, eans, 3,
DNS_RCODE_NOERROR,
                        false, eout, sizeof(eout), &eolen) == 0,
      "tcp+edns chain build failed");
CHECK(rd16(eout + 10) == 1, "tcp+edns response should carry OPT");
}

```

And in main():

c

```

test_names();
test_compression();
test_parse_query();
test_parse_rr();
test_build_response();
test_build_query();
test_edns();
test_cache();

```

Integration tests — test_resolver.py

Add near the top (after decode_name, before the query helpers):

python

```

def encode_query_edns(name, qid, qtype=1, payload=1232, do_bit=False):
    """Same as encode_query(), plus an OPT record in the additional
    section advertising `payload` as our UDP receive capacity."""
    q = encode_query(name, qid, qtype)
    q = q[:10] + struct.pack(">H", 1) + q[12:]      # ARCOUNT 0 -> 1
    ttl = 0x8000 if do_bit else 0
    opt = b"\x00" + struct.pack(">HHIH", 41, payload, ttl, 0)
    return q + opt

```

```

def get_edns_opt(msg):
    """(payload_size, rdlength) of the OPT record in the additional
    section, or None. Assumes NSCOUNT == 0, always true for this
    resolver's own responses."""
    _, qd, an = struct.unpack(">HHH", msg[2:8])
    arcount = struct.unpack(">H", msg[10:12])[0]
    off = 12
    for _ in range(qd):
        _, off = decode_name(msg, off)
        off += 4
    for _ in range(an):
        _, off = decode_name(msg, off)
        _, _, _, rdlen = struct.unpack(">HHIH", msg[off:off + 10])
        off += 10 + rdlen
    if arcount == 0:
        return None
    rtype, rclass, _, rdlen = struct.unpack(">HHIH", msg[off + 1:off +
11])
    return (rclass, rdlen) if rtype == 41 else None

```

Add a new section (e.g. right before the "TCP" section, or after it):

```

python
# =====
# EDNS0
# =====

# --- the same big chain that forced TCP fallback now fits over UDP
# once the client advertises a bigger payload ---
r, _ = ask("x", 0x3001, raw=encode_query_edns(BIG_HOP1, 0x3001,
payload=1232))
check(r is not None, "edns big chain got a UDP response")
if r:
    flags = struct.unpack(">H", r[2:4])[0]
    rcode, ans = parse_answers(r)
    check(not (flags & 0x0200), "edns big chain does not set TC over UDP",
          f"flags {flags:#06x}")
    check([a[1] for a in ans] == ["CNAME", "CNAME", "A"],
          "edns big chain returns the full 3-record chain over UDP",
          f"got {[a[1] for a in ans]}")
    opt = get_edns_opt(r)
    check(opt is not None, "edns response carries an OPT record")
    if opt:
        check(opt[0] == 1232, "opt advertises our payload size", f"got
{opt}")

```

```

        check(opt[1] == 0, "opt rdlength is 0 (no options)", f"got {opt}")

# --- a client claiming a small payload is held to it ---
r, _ = ask("x", 0x3002, raw=encode_query_edns(BIG_HOP1, 0x3002,
payload=600))
check(r is not None, "edns small-payload query got a response")
if r:
    check(len(r) <= 600, "response honours the smaller advertised
payload",
        f"got {len(r)} bytes")

# --- a plain query (no OPT) gets no OPT back ---
r, _ = ask("www.example.com", 0x3003)
check(r is not None, "plain query still answered")

if r:
    check(get_edns_opt(r) is None, "plain query gets no OPT record back")

```

README.md

In "Known limitations", replace:

- No EDNS0, so responses are capped at the classic 512 bytes.

with:

- EDNS0 (RFC 6891) is supported: a client that sends an OPT record gets UDP responses up to 1232 bytes (the 2020 DNS-flag-day value) instead of the classic 512, and gets an OPT record echoed back. We hold ourselves to 1232 regardless of what a client claims to accept, to stay clear of IP fragmentation. EDNS options (cookies, NSID, ...) and DNSSEC (the DO bit) are parsed but not acted on. A client requesting protocol version > 0 is not sent BADVERS; it's silently treated the same as version 0.

Commands to test everything

sh

make clean

```

make                                # rebuilds af_xdp_kern.o + all 4 binaries with the
                                    # new dns_iface.h ABI (dns_resolve_query gained a
                                    # bool param, so this is a hard rebuild, not
                                    # incremental)

```

```

make test                            # unit tests, ASan+UBSan, includes test_edns()

```

```
./test_dns          # same thing, run directly if you want to add -v
etc.

make offline        # unit tests + integration tests against the fake
                    # root/TLD/auth hierarchy, includes the new
                    # EDNS0 checks in test_resolver.py
```

Manual/live spot-check with dig (after make offline's harness, or your own

```
./server -p 5354 ... +DNS_ROOT_HINTS=...):
sh
# find the engineered long-name for the truncation demo
python3 -c "from fake_hierarchy import BIG_HOP1; print(BIG_HOP1)"

# modern dig sends EDNS0 (bufsize 1232) by default -- full answer, no TC
dig @127.0.0.1 -p 5354 <BIG_HOP1 name>

# force classic no-EDNS behaviour -- TC set, dig auto-retries over TCP
dig @127.0.0.1 -p 5354 +noedns <BIG_HOP1 name>

# ask for more than we'll give -- reply's OPT still says udp: 1232
dig @127.0.0.1 -p 5354 +bufsize=4096 www.example.com

# see the OPT pseudosection directly
dig @127.0.0.1 -p 5354 +qr www.example.com | grep -A2 "OPT PSEUDOSECTION"
```

If you want to re-run the full AF_XDP benchmark (`run_tests.sh`) afterward, nothing there needs changing — it already just drives `dnssperf/loadgen`, which don't send EDNS, so the timing comparisons are apples-to-apples with before; the EDNS behavior shows up only when you query with an EDNS-aware client like modern `dig`

explain me like, suppose our browser uses the DNS query to get the IP of any domain right?

Explain exactly considering the browser request to be Application layer request hwo is this routed differntly accroding to you

Explaining how AF_XDP reroutes DNS traffic around the kernel stack.

Good question — let's trace one query end-to-end and see exactly where the two paths diverge.

The normal (non-XDP) path

When your browser wants `example.com`'s IP, here's the actual layer-by-layer journey:

Browser (Application layer)

|
| calls `getaddrinfo()` / `gethostbyname()`

↓

OS resolver library (stub resolver)

|
| builds a DNS query, wraps it, hands to socket API

↓

`socket(AF_INET, SOCK_DGRAM) → sendto()`

|
| Transport layer: OS kernel adds UDP header (src port = random ephemeral, dst port = 53)

↓

Kernel IP layer: adds IPv4 header (src = your IP, dst = resolver IP, e.g. 8.8.8.8)

|

↓

Kernel adds Ethernet header (src MAC = your NIC, dst MAC = your gateway)

|

↓

NIC driver → wire → routers → DNS resolver's NIC

|

↓

DNS resolver's kernel: Ethernet → IP → UDP → delivered to a UDP socket

|

↓

DNS server *application* (e.g. `bind`, or Google's resolver software) reads the socket, processes, replies

The key thing: **every single packet, in both directions, passes through the full kernel networking stack** — Ethernet parsing, IP routing/checksums, UDP demux to the right socket — before any application code ever sees the bytes. This is normal, and it's what `socket()` + `sendto()/recvfrom()` give you for free.

The AF_XDP path (what this project asks for)

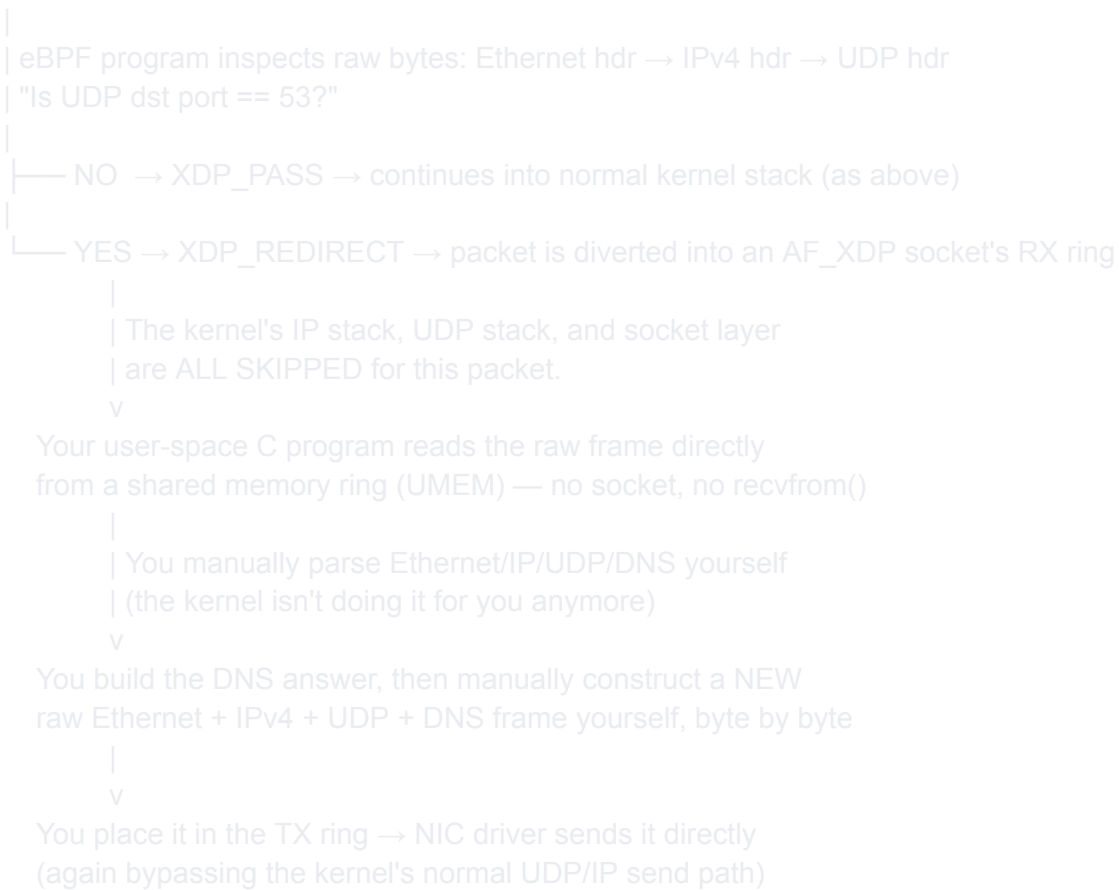
Now here's the divergence — it happens **on the machine acting as the DNS resolver**, at the very moment a packet arrives on the NIC, *before* it would normally enter the kernel's IP/UDP stack:

Client's query arrives at resolver's NIC

|

↓

XDP hook fires (this runs **INSIDE THE NIC DRIVER**, earliest possible point)



So what's actually different, concretely?

	Normal UDP socket path	AF_XDP path
Who parses Ethernet/IP/UDP headers on receive?	Kernel	You, in user-space code
Who builds Ethernet/IP/UDP headers on send?	Kernel (you just call <code>sendto()</code> with a payload)	You, manually crafting every header + checksum
Where does the packet "become visible" to your app?	After full kernel stack, at socket <code>recvfrom()</code>	Right at the NIC driver, via eBPF, before kernel stack
Copies of packet data	Kernel copies into socket buffer, then to your buffer	Ideally zero-copy — you read straight from shared UMEM memory
Who decides "does this app get this packet"?	Kernel's port-based socket lookup	Your own eBPF filter logic (<code>XDP_REDIRECT</code> on dst port 53)

Bringing it back to the browser

The browser itself, and its DNS query construction, are *identical* in both cases — it's still just an application-layer UDP request to port 53. **Nothing changes on the client/browser side.** The divergence is entirely on the *resolver machine's* receive/send path: normally the kernel silently does Ethernet→IP→UDP→socket demuxing for you; with AF_XDP, you rip that stack out for port-53 traffic and reimplement the header parsing, DNS logic, and header construction yourself in user space, trading kernel convenience for lower latency/overhead — which is exactly the performance tradeoff your project is meant to measure.

explain both nonXDP and XDP path in detail with very detailed description with each and every step explained in proper chain

Explaining the full technical process step by step.

Let's go step by step, layer by layer, with nothing skipped. I'll do the non-XDP path first since it's the baseline everything else is measured against, then the AF_XDP path, calling out exactly where and why it diverges.

PATH 1: Normal UDP Socket Resolver (Baseline)

Step 0 — Setup (happens once, before any query)

The resolver program does:

```
c
int sock = socket(AF_INET, SOCK_DGRAM, 0);
bind(sock, {0.0.0.0, port 53}, ...);
```

This tells the **kernel**: "any UDP packet arriving with destination port 53 belongs to *this* socket's receive queue." The kernel now maintains an internal table (a socket lookup table, keyed by protocol+port+address) so it knows where to deliver future port-53 packets.

Step 1 — Client sends the query (browser side, not our focus, but for completeness)

Browser → OS stub resolver → `sendto()` → kernel builds UDP header (src port = random ephemeral, e.g. 51041; dst port = 53) → kernel builds IPv4 header (src = client IP, dst = resolver IP) → kernel builds Ethernet header (dst MAC = default gateway) → NIC transmits bits onto the wire.

Step 2 — Packet arrives at the resolver's NIC (hardware)

The NIC receives the raw electrical/optical signal, reconstructs it into a frame, and — if using standard networking — DMAs (copies) the raw bytes into a kernel-managed **ring buffer in RAM** (this is the NIC driver's RX ring, separate from AF_XDP's ring). This triggers a hardware interrupt (or the driver is polling, via NAPI).

Step 3 — Kernel driver layer

The `sk_buff` (socket buffer) is allocated by the kernel to represent this packet. The raw bytes are attached to it. This is **copy #1**: bytes moved from NIC DMA buffer into kernel packet buffer.

Step 4 — Ethernet (Layer 2) processing

Kernel's `netif_receive_skb()` path parses the Ethernet header:

- Checks destination MAC matches this NIC (or is broadcast/multicast).
- Reads EtherType field (0x0800 = IPv4).
- Strips Ethernet header, hands the rest up to the IP layer.

Step 5 — IPv4 (Layer 3) processing

Kernel's IP receive function:

- Validates IP header checksum.
- Checks destination IP matches a local address (or needs forwarding — not our case).
- Reads Protocol field (17 = UDP).
- Reassembles fragments if needed (DNS queries usually aren't fragmented).
- Strips IP header, hands payload to UDP layer.

Step 6 — UDP (Layer 4) processing

Kernel's UDP receive function:

- Validates UDP checksum (optional but typically present).
- Reads destination port (53).
- **Looks up the socket table** built in Step 0 to find which socket owns port 53.
- Copies the UDP payload (the raw DNS message bytes) into that socket's **receive queue** (a buffer associated with the socket, not the app yet). This is **copy #2**.
- Wakes up any process blocked on `recvfrom()`.

Step 7 — Application reads it

Your resolver program was blocked here:

```
c
recvfrom(sock, buf, sizeof(buf), 0, &client_addr, &addr_len);
```

The kernel copies bytes from the socket's receive queue into your userspace buffer `buf`. This is **copy #3**. Now, finally, your application-layer code has:

- The raw DNS message bytes (already stripped of Ethernet/IP/UDP — the kernel did all that parsing for you).
- `client_addr` filled in automatically by the kernel (client's IP and source port), extracted from the headers it already parsed.

Step 8 — DNS logic (your code)

You parse the DNS header + question section (transaction ID, QNAME, QTYPE), look up or forward, build a DNS response message (just the DNS layer — question echoed back + answer records).

Step 9 — Sending the reply

C

```
sendto(sock, response_buf, response_len, 0, &client_addr, addr_len);
```

You only supply the **payload** and destination address. The kernel does everything else:

- **UDP layer:** builds UDP header (src port = 53, dst port = client's original source port, computes checksum).
- **IP layer:** builds IPv4 header (src = resolver IP, dst = client IP, TTL, checksum), consults the routing table to decide the next hop.
- **ARP/neighbor lookup:** kernel checks its ARP cache for the next hop's MAC address (may trigger an ARP request if not cached).
- **Ethernet layer:** builds Ethernet header (dst MAC = next hop's MAC, src MAC = resolver's NIC MAC).
- Hands the fully-formed frame to the NIC driver, which DMAs it out to the wire.

Full chain, non-XDP:

Client sendto()

→ kernel UDP hdr → kernel IP hdr → kernel Eth hdr → NIC → wire

→ Resolver NIC → DMA copy → sk_buff

→ kernel parses Eth → strips

→ kernel parses IP → strips

→ kernel parses UDP, socket lookup → copies to socket recv queue

→ app recvfrom() → copies to app buffer [app sees ONLY DNS bytes]

→ app builds DNS response

→ app sendto() with payload only

→ kernel builds UDP hdr → kernel builds IP hdr → ARP lookup → kernel builds Eth hdr

→ NIC → wire → client

Note: **three data copies** on receive path alone (DMA→skb, skb→socket queue, socket queue→userspace), and the application never sees or touches Ethernet/IP/UDP headers — the kernel abstracts all of it away via the socket API.

PATH 2: AF_XDP Resolver

Step 0 — Setup (happens once, before any query — much more involved)

This is where it gets heavier than a plain socket:

1. **Allocate UMEM**: you `mmap()` a chunk of memory in your user-space process. This becomes the *shared* packet buffer region between kernel and your app — no kernel-private copy exists separately from this.
2. **Register UMEM with the kernel** via `setsockopt(XDP_UMEM_REG)`, dividing it into fixed-size frames (e.g. 2048 or 4096 bytes each).
3. **Create the four rings**:
 - **Fill ring** — you push addresses of empty UMEM frames into this, telling the kernel "here are buffers you can DMA incoming packets into."
 - **RX ring** — kernel pushes descriptors here (frame address + length) when a packet arrives.
 - **TX ring** — you push descriptors here when you want to transmit a frame.
 - **Completion ring** — kernel pushes back the addresses of frames it has finished transmitting, so you can reuse them.
4. **Create the AF_XDP socket** (`AF_XDP`, `SOCK_RAW`-like) and `bind()` it to a specific interface + queue ID.
5. **Load and attach the XDP eBPF program** to the NIC via `bpf(BPF_PROG_LOAD)` + attaching it to the interface's XDP hook. This program contains hand-written logic (in eBPF bytecode, verified by the kernel verifier) to parse headers *inside the driver*.
6. **Populate an XSKMAP** (a special eBPF map) that maps RX queue index → your AF_XDP socket, so the eBPF program knows which socket to redirect matching packets to.
7. Push some initial empty frame addresses into the **fill ring** so there's somewhere for incoming packets to land.

Step 1 — Client sends the query

Identical to before — client-side is unchanged. Raw frame arrives at the wire, destined for the resolver's NIC.

Step 2 — Packet arrives at the resolver's NIC (hardware)

NIC receives the frame. Normally it would DMA into the kernel's default RX ring and generate an skb. **But** because an XDP program is attached, the driver instead invokes the XDP hook

before any skb is allocated — this is the earliest possible interception point, literally inside the NIC driver's receive routine.

Step 3 — XDP program execution (in-kernel, but pre-stack)

Your eBPF program runs, given a pointer to the raw frame bytes still sitting in the NIC's DMA buffer. It manually walks the headers *itself*, using raw pointer arithmetic (with the eBPF verifier enforcing bounds checks):

```
data = ctx->data
data_end = ctx->data_end

eth = (struct ethhdr *)data
if data + sizeof(eth) > data_end: bounds-fail
if eth->h_proto != ETH_P_IP: return XDP_PASS // not our concern

ip = (struct iphdr *)(eth + 1)
if bounds check fails: return XDP_PASS
if ip->protocol != IPPROTO_UDP: return XDP_PASS

udp = (struct udphdr *)(ip + 1)
if bounds check fails: return XDP_PASS
if udp->dest != htons(53): return XDP_PASS

// It's a DNS query — redirect it
return bpf_redirect_map(&xsk_map, queue_index, 0) // → XDP_REDIRECT
```

- If it's **not** UDP/53 traffic → **XDP_PASS** → the packet falls through to the *normal* path (Steps 3–7 of Path 1 above) and behaves exactly like a regular socket-based flow. This is important: XDP doesn't touch non-matching traffic at all.
- If it **is** UDP/53 → **XDP_REDIRECT** tells the driver: don't build an skb, don't hand this to the IP stack — instead hand it to the AF_XDP socket registered in the **XSKMAP**.

Step 4 — Redirect into AF_XDP socket

The driver takes a free frame address you'd previously placed in the **fill ring**, DMA's the packet bytes directly into that UMEM frame (this is the **only copy** — or in true zero-copy mode, the NIC may DMA directly into the UMEM frame with no copy at all, if the driver supports zero-copy AF_XDP), and places a descriptor **{addr, len}** into the **RX ring**.

Critically: **no sk_buff was allocated, no IP layer touched it, no UDP layer touched it, no socket lookup by port happened** — your eBPF program's own logic was the only thing that decided this packet's fate.

Step 5 — Your user-space program reads the RX ring

Your app loop:

```
c
while (1) {
    idx = poll RX ring for new descriptors;
    for each descriptor:
        addr = desc.addr;    // offset into UMEM
        len = desc.len;
        packet_ptr = umem_base + addr; // directly point into shared memory
        // packet_ptr now points at the RAW Ethernet frame — untouched
    }
}
```

There is **no `recvfrom()`**, **no socket read**, **no kernel copy into your buffer** — you're reading directly out of the same memory region the NIC DMA'd into. This is the "zero-copy" performance win.

Step 6 — YOU manually parse Ethernet/IP/UDP/DNS (application-layer + everything below, all in user-space now)

Since the kernel didn't strip any headers for you, your code must do what the kernel used to do in Steps 4–6 of Path 1:

- Parse Ethernet header → extract client's source MAC (needed later for the reply), verify EtherType.
- Parse IPv4 header → extract client's source IP, verify checksum yourself if you care, note header length (may include options).
- Parse UDP header → extract client's source port, verify UDP length/checksum.
- What's left is the raw DNS message bytes — *now* you're at the same logical point Path 1 reached automatically after `recvfrom()`.

Step 7 — DNS logic (your code) — identical in spirit to Path 1

Parse DNS header (transaction ID), question section, decide answer, build DNS response bytes.

Step 8 — YOU manually construct the reply frame (reverse of Step 6, and this is work the kernel did silently in Path 1's `sendto()`)

You grab a free UMEM frame (from the completion ring, or a fresh fill-ring frame) and write into it, byte by byte:

1. **Ethernet header:** dst MAC = client's MAC (captured in Step 6), src MAC = your NIC's MAC, EtherType = IPv4.

2. **IPv4 header:** src IP = resolver IP, dst IP = client IP, protocol = UDP, TTL, total length, then **compute the IPv4 header checksum yourself** (one's-complement sum — the kernel isn't doing this for you anymore).
3. **UDP header:** src port = 53, dst port = client's original source port, length, then **compute the UDP checksum yourself** (which for IPv4/UDP involves building a pseudo-header from the IP src/dst + protocol + UDP length — kernel used to handle this transparently).
4. **DNS payload:** your response bytes from Step 7.

Step 9 — Transmit

You place a descriptor `{addr of the frame you just built, length}` into the **TX ring**, then call a syscall (`sendto()` on the `AF_XDP` socket, used here just as a kick/notify mechanism, or the kernel may poll automatically) to tell the kernel "go transmit what's in the TX ring." The driver DMA's that frame straight from UMEM out to the NIC — no IP stack, no UDP stack involved on the way out either.

Step 10 — Recycle the frame

Once transmission completes, the kernel places that frame's address into the **completion ring** so you know it's safe to reuse for a future packet (RX or TX). You're responsible for this bookkeeping — in Path 1, the kernel managed buffer lifecycle for you invisibly.

Full chain, `AF_XDP`:

Client `sendto()` → kernel builds all headers → NIC → wire
 → Resolver NIC → XDP hook fires **INSIDE DRIVER**, before `skb/IP/UDP` stack
 → eBPF program manually parses `Eth→IP→UDP` headers itself
 → matches UDP dst port 53 → `XDP_REDIRECT`
 → frame DMA'd into UMEM (fill-ring frame) → descriptor pushed to RX ring
 → app reads RX ring, points directly at UMEM (no socket copy)
 → app manually parses `Eth/IP/UDP` itself (kernel didn't)
 → app extracts DNS bytes, runs DNS logic, builds DNS answer bytes
 → app manually builds **NEW** `Eth+IP+UDP` headers itself, computes checksums itself
 → app writes frame into a UMEM frame, pushes descriptor to TX ring
 → kernel DMA's straight from UMEM to NIC → wire → client
 → completion ring returns frame address for reuse

Side-by-side summary of what moved where

Responsibility	Path 1 (UDP socket)	Path 2 (<code>AF_XDP</code>)
Ethernet parse (RX)	Kernel	eBPF program + your app
IP parse/checksum (RX)	Kernel	eBPF program + your app

UDP parse/checksum (RX)	Kernel	eBPF program + your app
Socket/port demux decision	Kernel's socket table	Your eBPF filter logic + XSKMAP
Data copies on receive	3 (DMA→skb, skb→sockbuf, sockbuf→userbuf)	1, or 0 in true zero-copy mode
Ethernet/IP/UDP header construction (TX)	Kernel (from <code>sendto()</code> payload)	Your app, manually, byte-by-byte
Checksum computation (TX)	Kernel	Your app
ARP/routing lookup (TX)	Kernel	Your app must handle or pre-resolve
Buffer lifecycle management	Kernel (hidden)	Your app (fill/RX/TX/completion rings)
Where app-visible processing begins	After full network stack, at <code>recvfrom()</code>	The instant a raw frame lands in UMEM

The one-sentence version: **in Path 1, four layers of kernel machinery (driver→Ethernet→IP→UDP→socket) stand between the wire and your DNS logic, doing parsing, checksums, and buffer copies for you automatically; in Path 2, your eBPF program intercepts the frame at the driver itself and your user-space code re-implements everything the kernel used to do for those layers — in exchange for skipping the socket/stack overhead entirely.**

AF_XDP Egress Path — Key Points

1. Buffer reuse

- Reply is written into the **same UMEM buffer** that held the incoming request.
- Already DMA-mapped → no need to register a new buffer, no extra copy.

2. Userspace prepares the packet

- App writes the response payload (headers + DNS data) directly into a UMEM frame.
- App pushes that frame's address into the **TX ring** → signals "ready to send."

3. Trigger the kernel

- A cheap **zero-length `sendto()`** call notifies the kernel to process the TX ring.
- This syscall carries no data — it's just a "go look" signal.

4. Kernel driver hands off to NIC

- Driver dequeues the TX ring entries.
- Writes them into the NIC's own **TX descriptor ring**, giving the NIC the DMA address + length.

5. NIC transmits via DMA

- NIC does a **DMA read** straight from UMEM into its internal buffer, then sends on the wire.
- **No CPU involvement** in moving the data.

6. Completion signaling

- NIC signals done via **interrupt or NAPI poll**.
- Driver places the UMEM frame's address into the **Completion ring**.

7. Buffer recycling

- App polls the Completion ring to see which buffers are free.
- Pushes those addresses back into the **Fill ring** → ready for future incoming packets.

8. What's eliminated vs. normal stack

- No `sk_buff` creation.
- No traversal of kernel networking layers (UDP→IP→Ethernet).
- No user-to-kernel or kernel-to-NIC data copies.

9. Net effect

- Same UMEM buffer cycles through RX → processing → TX → completion → Fill, reused indefinitely.
- Minimizes kernel involvement and memory copies → significantly lower per-packet overhead, while userspace keeps full control of buffer lifetime.



explain this pointwise similarly

This is a shift from theory to the actual **implementation** — real functions from their `af_xdp_client.c` code. Let's break it into stages.

Filtering logic (continuation from before)

- Uses `bpf_printk` to log source/destination ports (debug tool).
- If destination port is **53** → redirect to userspace socket via `xsk_map`.
- Everything else → `XDP_PASS` (goes through normal kernel path).

3.2 The XDP Client Program — Setup Phase

Step 1: Load & attach the eBPF hook

- Uses `xdp_program__create` → loads the compiled eBPF hook file.
- Uses `xdp_program__attach` → attaches it to the NIC.

Step 2: Get access to the XSKMAP

- After attaching, retrieves the `xsk_map` using:
`bpf_object__find_map_by_name(...)`
- This is how the userspace program gets a handle to the same map the eBPF code uses to pick a target socket.

Step 3: Prepare memory

- Removes locked-memory limits (`setrlimit`) — needed because UMEM must be **pinned** (non-swappable) memory, which usually requires elevated limits.
- Allocates a **page-aligned buffer** (`packet_buffer`) — page-alignment is required for DMA to work correctly.
- Registers this buffer as **UMEM** via `configure_xsk_umem` — this is the shared kernel↔userspace region we've discussed extensively.

Step 4: Create & bind the AF_XDP socket

- `xsk_configure_socket` — creates the socket, binds it to a specific NIC + queue.

Step 5: Thread pool for concurrency

- Initializes a **pool of DNS worker threads** so multiple queries can be processed concurrently instead of one-at-a-time.

Step 6: Main receive loop

- `rx_and_process()` — the core loop that continuously receives + processes packets.
- On shutdown: cleans up — destroys thread pool, closes socket, frees memory.

`rx_and_process()` — How packets actually get pulled out

Step 1: Poll for readiness

- Continuously polls the XDP socket file descriptor (`xsk_socket__fd()`).
- Uses `poll()` syscall **only if `xsk_poll_mode` is enabled** (i.e., this is optional — busy-polling avoids even this syscall).

Step 2: Read packets from RX ring

- Once packets are available, calls `handle_receive_packets()`.
- This retrieves entries using `xsk_ring_cons__peek()` — reads from the RX ring (as we covered — just addresses/offsets into UMEM).

Step 3: Refill the Fill ring (keep pipeline moving)

- To avoid running out of buffers, the code proactively refills the **Fill ring**:
 - `xsk_prod_nb_free()` — checks how many free slots are available.
 - `xsk_ring_prod__reserve()` — reserves entries to write into.
 - `xsk_ring_prod__fill_addr()` — gets frame addresses to offer up.
 - `xsk_ring_prod__submit()` — commits them, making them visible to the kernel.
- This is the exact Fill-ring mechanism from our earlier "ownership" discussion — done here explicitly in code.

Step 4: Extract and dispatch each packet

- For each received packet: extract its **memory address and length** from the RX ring descriptor.
- Pass it to `process_packet()` for actual parsing/handling.

Worker Threads — Concurrent DNS Processing

Step 1: Threads pull work from a shared queue

- `dns_worker_thread()` — each thread runs concurrently.
- Dequeues tasks using `work_queue_pop()`.

Step 2: Resolve the DNS query

- Calls `resolve_dns()` — this is the actual "ask further server" logic we discussed earlier (the DNS resolution step).

Step 3: Build the response

- On success: `craft_response_with_ip()`.

- On failure (NXDOMAIN/SERVFAIL): `craft_error_response()`.
- (NXDOMAIN = domain doesn't exist; SERVFAIL = resolver-side failure — standard DNS error codes.)

Step 4: Transmit the response

- `send_dns_response()` — pushes the reply through the AF_XDP TX path (the egress flow we covered: write to UMEM → TX ring → kick kernel).
- `complete_tx()` — notifies the kernel that transmission is done (this is the completion-ring handling).

Step 5: Cleanup per request

- `xsk_free_umem_frame()` — releases/recycles the UMEM frame (back into the Fill ring for reuse, per our earlier discussion).
- `free()` — frees any other dynamically allocated memory (unrelated to UMEM, just normal heap cleanup).

Big-picture takeaway (their own words)

This combination —

- low-level XDP socket operations,
- efficient UMEM-based memory management,
- multithreaded/concurrent DNS resolution —

is what enables **high-throughput, zero-copy packet handling**, suitable for a low-latency DNS server. This is literally the real code implementing every concept we've walked through conceptually: eBPF filtering → XSKMAP redirect → UMEM/rings → your own header parsing/DNS logic → manual reply construction → TX/completion cycle — now tied to actual function names in their codebase.

Section 4 preview

The next section says they built their **own custom DNS resolver logic in userspace** (rather than using an existing stub resolver library) — meaning `resolve_dns()` is their own implementation, not a call into some pre-built DNS library. This aligns with what we discussed: the actual "ask a further server" logic is custom application code, separate from the AF_XDP machinery.

i want to implement a couple of features:

- TCP fallback for truncated responses — DNS over UDP has a 512-byte (or EDNS0-negotiated) size limit; when a response is too big, the resolver should set the TC (truncated) bit and let the client retry over TCP. This is explicitly called out as a gap in the

doc's "minimal iterative resolver" limitations list — implementing it (even partially) is a natural, well-scoped extension.

- EDNS(0) support — lets you negotiate larger UDP payload sizes (avoiding unnecessary truncation) and is a genuinely modern DNS feature that's usually skipped in a first pass. Also improves realism of your benchmark since real-world resolvers almost all send EDNS0.
- Retry/timeout logic for upstream queries (if you're doing the forwarding design) — real resolvers retry a query to a different upstream server (or resend) if no answer comes within some timeout. Good for demonstrating resilience under packet loss.

i want to implement the TCP thing first\

Good pick — this is a well-scoped feature. Before I start coding, I want to nail down *which* side of "TCP fallback" you mean, because the two are pretty different pieces of work and the README hints at both:

1. **Client-facing:** right now `dns_build_response()` already sets the TC bit correctly when a response doesn't fit in 512 bytes — but nothing in `server.c` (or the AF_XDP path) actually listens on TCP/53, so a client that gets TC=1 has nowhere to retry to. This means adding a TCP listener that speaks the 2-byte-length-prefixed DNS-over-TCP framing and returns the *full*, uncapped response.
2. **Upstream-facing** (the one the README explicitly calls out as unimplemented): in `resolver.c`, `query_server()` only ever asks upstream servers over UDP. If an upstream response comes back with TC set, `dissect()` currently just bails (`return -1`) and the resolver falls through to trying the *next* server instead of correctly re-querying the *same* server over TCP to get the complete answer.

isn't what i'm making a server. wait, is this an recursive resolver or is can it act as both? does client side on server make sense?

Good question — let's clarify the architecture, because "client-facing" was a confusing label on my part.

Your project is a **recursive/iterative resolver**, not a forwarder. It plays *two* distinct network roles simultaneously:

1. **Downstream role — you're the server.** A stub resolver or `dig` sends a query to your resolver on port 53 (either via `server.c`'s plain UDP socket, or via the AF_XDP fast path). You compute the answer and send it back. This is the direction `loadgen.c` and `test_resolver.py` exercise.
2. **Upstream role — you're the client.** To *compute* that answer, `resolver.c` doesn't ask some other recursive resolver for the final answer — it walks the delegation chain itself: query the root hints, get a referral to `.com`, query that, get a referral to the authoritative

server, query that, get the actual record. In each of those hops, *your resolver* is the one sending the query and the root/TLD/auth server is the one answering. That's `query_server()` in `resolver.c`.

So "TCP fallback" genuinely applies twice, in opposite directions:

- **Downstream direction:** a client asks *you* something whose answer doesn't fit in 512 bytes (e.g., a name with a long CNAME chain or many records). You set TC=1 on the UDP reply. For the client to actually get the full answer, *you* need to be listening on TCP/53 too, so it can reconnect there. This is 100% "does it make sense for a server to accept TCP" — yes, every real DNS server (BIND, Unbound, PowerDNS, your home router) listens on both UDP/53 and TCP/53 for exactly this reason.
- **Upstream direction:** when *you* query a root/TLD/auth server over UDP and *it* replies with TC=1 (its answer didn't fit in 512), your resolver currently just gives up and tries a different server (per the README gap). Correct behavior is to re-ask *that same server* over TCP.

Given that, which do you want first?

Good context to have — this paper is describing the same NAPI/AF_XDP machinery from a slightly different angle than we've been discussing, so let me tie the two together and be precise about which "busy polling" it means, since (as we covered last turn) there are actually two unrelated things that share that name.

What NAPI is

The paper's framing is right: once a packet is DMA'd into a ring buffer descriptor, something has to notice it's there and hand it up the stack. The naive approach — one hardware interrupt per packet — collapses under DNS-scale traffic, since tiny UDP packets at high rate mean the CPU spends more time entering/exiting interrupt context than doing actual work.

NAPI (the driver-level mechanism, standard on every mainline Linux NIC driver) fixes this with a hybrid scheme:

1. First packet arrives → one hardware interrupt fires.
2. The interrupt handler does almost nothing: it **disables further RX interrupts on that queue** and schedules the driver's poll function via `NET_RX_SOFTIRQ`.
3. That softirq calls the driver's `napi->poll()` — this is the function pointer your XDP program actually runs inside of, per the paper's own point about the XDP hook being invoked from here, before any `skb` exists.
4. `poll()` drains up to a fixed budget (traditionally 64 packets) off the ring in one pass.
5. If the ring goes empty before the budget runs out → re-arm the hardware interrupt, go back to sleep.
6. If the budget gets fully used (meaning more traffic is still queued) → **don't** re-arm interrupts, just reschedule another softirq pass immediately.

That last branch is the important one: under sustained heavy load, step 6 keeps firing over and over, and interrupts effectively stay suppressed the whole time — the CPU converges into continuous polling *without anyone asking for it*. This is what the paper calls "NAPI poll mode," and it's a purely reactive, kernel-scheduled behavior: at low traffic you get occasional interrupts + short poll bursts; at high traffic it smoothly turns into a tight softirq loop on its own.

What "AF_XDP busy-poll mode" means (and why it's a different mechanism)

This is where I want to be careful, because the paper's phrasing — "NAPI poll mode **or** AF_XDP busy-poll mode" — presents these as two parallel options, and they are, but for a different reason than "poll mode just needs more traffic." Busy-poll mode is **explicitly requested**, not something that emerges from load:

- `SO_BUSY_POLL` (general sockets) / `SO_PREFER_BUSY_POLL` (kernel ≥5.10, AF_XDP-aware) is a socket option. When set, a thread calling into that socket (`recvfrom`, `poll`, etc.) directly and synchronously invokes `napi_busy_loop()` — which calls the *exact same* `napi->poll()` function from step 3 above — from inside its own syscall, instead of waiting for the softirq scheduler to get around to it.
- `SO_PREFER_BUSY_POLL` additionally tells the softirq machinery to back off scheduling that NAPI instance itself while a busy-poller is active, so the two invocation paths don't fight over the same ring.

So the correct relationship is: **NAPI's poll function is the one and only thing that ever drains the ring — "poll mode" and "busy-poll mode" are two different ways of getting that same function invoked**, not two different draining mechanisms. Poll mode is the kernel deciding to keep calling it based on how full the ring stays; busy-poll mode is your thread forcing the call to happen every time it touches the socket, independent of load.

Mapping this onto your own project

This is the piece worth being explicit about for your report/viva, because there are now **three** layers total and your code only implements one of them:

Layer	What it does	Controlled by	Your project
NAPI's <code>poll()</code> function	Actually drains the ring, runs your XDP program per packet	— always exists, never optional	Runs <code>xdp_dns_redirec t()</code> inline
How NAPI gets <i>scheduled</i>	interrupt→softirq (default, becomes continuous "poll mode" under load) vs.	the paper's two bullets	Not set — left at kernel default

SO_BUSY_POLL/SO_PREFER_BUSY_POLL (thread forces it)

Your userspace RX/TX loop	Whether <i>your thread</i> sleeps in <code>poll()</code> on the AF_XDP fd, or spins	your <code>-b / opt.busy_poll</code> <code>ll</code>	Implemented — this is <code>run_loop()</code>
------------------------------------	--	---	--

So your `-b` flag is a third, separate thing from either of the paper's two bullets — it only decides whether your own thread blocks or spins *once packets are already sitting in the AF_XDP ring*. It has no effect on whether getting them into that ring in the first place was interrupt-driven or busy-poll-driven at the NIC/NAPI level. Concretely: with `-b` set today, under low-to-moderate traffic you're still paying one hardware interrupt + one softirq dispatch per burst to get packets from the wire into the AF_XDP ring — `-b` only removes the *second* layer of latency (your thread's own scheduler wakeup) on top of that. Adding `SO_PREFER_BUSY_POLL` on the AF_XDP socket in `xsk_setup.c` would close that remaining gap by making your `-b` spin loop *also* the thing pulling packets off the wire, collapsing all three layers into one continuous loop on one core — which is the actual mechanism the paper's "AF_XDP busy-poll mode" bullet is referring to, and still the piece missing from `run_tests.sh`'s comparison matrix if you want a fully apples-to-apples "every polling mode this stack supports" benchmark.

the ASAN/UBSAN was tested where? for `af_xdp_kern` or `user` or `resolver`?

Precise answer: it covered `resolver.c`, `dns_backend.c`, `dns_msg.c`, `dns_cache.c`, `dns_tcp.c`, and `server.c` — the backend logic and the plain-UDP transport. It did **not** cover `af_xdp_user.c`, `xsk_setup.c`, or `af_xdp_kern.c`. Here's exactly where each sanitizer run actually pointed, and why the AF_XDP-specific files were excluded both times.

What the project's own `make test` covers (before this session)

`make`

```
test_dns: test_dns.c dns_msg.c dns_cache.c
$(CC) -O1 -g -Wall -Wextra -I. -fsanitize=address,undefined -o $@ $^ -lpthread
```

Only `dns_msg.c` (wire parsing/building) and `dns_cache.c` (the LRU). That's it — no `resolver.c`, no transport code at all. This matches the README's own framing: these two files are "pure — no sockets, no globals, no allocation" by design, specifically so they *can* be exhaustively unit-tested this way.

What I added and ran in this session

`bash`

```
gcc -O1 -g -Wall -Wextra -I. -fsanitize=address,undefined -pthread \
```



```
-o server_asan server.c dns_tcp.c dns_backend.c dns_msg.c dns_cache.c resolver.c  
-lpthread
```

and the ThreadSanitizer equivalent with the same file list but `-fsanitize=thread`. So this extended coverage to:

- **resolver.c** — the iterative resolution logic (root→TLD→auth walking, CNAME chasing, referral handling) — this was genuinely new coverage; it was never under any sanitizer before, in the original project or in this session's TCP/EDNS0 unit tests.
- **dns_backend.c** — the glue function (`dns_resolve_query()`) that both transports call, including the new EDNS0 negotiation logic added this session.
- **dns_tcp.c** — the new TCP listener, exercised under real socket/thread activity for the first time (this is exactly the module `test_dns.c`'s unit tests can't reach, since it's almost entirely sockets/threads with no pure logic to isolate).
- **server.c** — the plain-UDP control-arm transport.

I deliberately picked `server.c` as the driver, **not** `af_xdp_user.c`, specifically to sidestep AF_XDP's requirements (root/CAP_NET_RAW, a real NIC or veth pair, `--privileged` if in Docker) so the sanitizer runs could go straight into `test_resolver.py`'s existing offline harness with no extra setup.

What was never sanitized, and why each one specifically can't be

File	Why ASan/UBSan doesn't apply
<code>af_xdp_kern.c</code>	This compiles with <code>clang -target bpf</code> , not <code>gcc/native</code> — it becomes eBPF bytecode loaded into the kernel and checked by the in-kernel BPF verifier , not linked against any userspace sanitizer runtime. There's no equivalent of <code>-fsanitize=address</code> for the BPF target; the closest thing this project already has is <code>make verify</code> , which runs the actual verifier (a different, narrower kind of safety check: bounds-on-every-access, no unbounded loops, no unreachable code — not "undefined behavior" in the C sense).
<code>af_xdp_user.c</code> / <code>xsk_setup.c</code>	Technically <i>could</i> be compiled with <code>-fsanitize=address,undefined</code> — nothing architecturally blocks it — but doing so meaningfully requires actually running the AF_XDP path (a bound XSK socket, UMEM, a real or veth NIC), which is exactly the setup complexity <code>server.c</code> exists to let you skip for iteration/testing. I chose not to bother standing up a veth pair just to sanitizer-test this file, since <code>dns_tcp.c/dns_backend.c/resolver.c</code> — the actually new code this session — are identical whichever transport drives them (that's the whole "one independent variable" design your project already leans on).

If you want full coverage

The one gap worth closing, if you want ASan/UBSan on literally everything reachable from userspace: build an `af_xdp_user_asan` the same way, using the veth setup from the README's own "Testing without a network, a NIC, or root" section:

bash

```
gcc -O1 -g -Wall -Wextra -I. -fsanitize=address,undefined -pthread \
-o af_xdp_user_asan af_xdp_user.c xsk_setup.c dns_tcp.c dns_backend.c dns_msg.c
dns_cache.c resolver.c \
-lxdp -lbpf -lelf -lz -lpthread
```

then drive it with `dig` over the veth pair the README already documents. `af_xdp_kern.c` itself stays permanently out of reach of ASan/UBSan regardless — the BPF verifier is the only automated correctness check that will ever apply to it.

Layer 1 — `test_dns_msg.c`: unit tests for the wire-format and cache code

This is a plain C test binary (no network, no threads, no root privileges — the header comment says it runs under ASan+UBSan via `make test`), exercising `dns_msg.c` and `dns_cache.c` directly by calling their functions and inspecting buffers.

Harness: a tiny homemade one — `CHECK(cond, ...)` increments a global checks counter, and on failure prints the file/line plus a formatted message and increments failures. `main()` just calls each `test_*`() function in sequence and exits nonzero if anything failed. No framework, no dependencies — appropriate for something meant to run in CI under sanitizers.

`test_names()` — round-trips `dns_name_encode/dns_name_decode`:

Encodes "www.example.com", checks the exact byte length (17) and that label-length bytes are where expected (`buf[0]==3`, `buf[4]==7`), then decodes it back and checks it matches, including that `next_off` lands exactly after the name.

Trailing-dot form ("example.com.") and the root name ("") are checked as their own cases.

Checks that decoding lowercases mixed-case input, using a hand-built raw label buffer.

Negative cases: empty label ("a..b"), a label over 63 bytes, and a name over 255 bytes must all be rejected by the encoder.

`dns_name_equal` is checked for case-insensitivity, mismatch, and that it doesn't do a prefix match (example.com vs example.com.uk).

`test_compression()` — targets the pointer-following logic specifically, since that's the classic place for parsing bugs to hide:

A normal case: builds a message where a second name is a compression pointer back into the first, and checks it decodes correctly and `next_off` is past the pointer (not past the jump target).

Then a series of adversarial inputs, each expected to be rejected: a self-pointer ($0 \rightarrow 0$), a forward pointer ($0 \rightarrow 2$), a ping-pong pair ($4 \rightarrow 0$, $0 \rightarrow 4$), a pointer truncated to one byte, a reserved label-type bit pattern (0x40), and a label whose declared length runs past the end of the buffer. These map directly onto the defenses described in `dns_name_decode()` (backwards-only jumps, jump-count cap, bounds checks).

`test_parse_query()` — checks that a built query round-trips through `dns_parse_query()` correctly (id, name, qtype, qclass, RD flag, raw offset/length), then a set of rejections: header-only (11-byte) message, QR=1 (a response isn't a valid query), non-zero opcode, qdcount=2, qdcount=0, and a question truncated before its type/class fields. It also checks the DNS-0x20 property: a mixed-case query name gets lowercased in the parsed struct, but the raw bytes in the original buffer are untouched (`mixed[13] == 'W'`) — this matters because the response builder later needs to echo the original casing back verbatim.

`test_parse_rr()` — builds a raw A record by hand, checks it parses correctly, then corrupts the `rlength` field to claim more bytes than are actually present and checks that's rejected.

`test_build_response()` — the most detailed test, covering `dns_build_response()`:

Poisons the output buffer with 0xFF before calling, specifically to catch a real bug class: since the header (including the TC flag) is written last, anything that reads `out[]` before it's written would read garbage — the comment says this is literally what caught a "spurious TC bit" bug during development.

Checks a normal single-answer response: exact output length, ID echoed, QR/RA set, RD mirrored, TC not set, rcode NOERROR, counts correct, the question section echoed byte-for-byte, the answer's owner name compressed to a pointer at offset 12, and the rdata bytes correct.

Checks the mixed-case/0x20 case again at the response layer: the question must be echoed with its original casing preserved.

Checks NXDOMAIN: rcode set correctly, zero answers even though answers were passed in, output length equals just the question (no answer section written), TC not set.

Checks a CNAME chain: two answers where the second's owner isn't the queried name — verifies both are present, in order; that the first (matching) owner is compressed but the second (non-matching) owner is written out in full rather than pointer-compressed; and separately re-parses the built response with `dns_parse_rr/dns_name_decode` to confirm the CNAME target is actually recoverable from the bytes produced, not just internally consistent.

Checks the truncation path: a buffer sized to fit the question but not a full answer record must come back with TC set, ancourt 0, and length equal to just the question — i.e., no half-written RR leaks into the output.

Checks the absolute floor: a buffer too small even for the 12-byte header is rejected outright.

`test_build_query()` — checks the outbound query builder: ID, RD flag off when not requested, qdcount, exact length, and that an undersized buffer is rejected.

`test_cache()` — exercises `dns_cache.c` directly:

Miss on empty cache, then hit after `dns_cache_put`, checking the answer content and that TTL served is remaining-time-based (≤ 300 && > 290 , since a little time will have passed).

Case-insensitive lookup (`WWW.EXAMPLE.COM` matches an entry stored as `www.example.com`), and that qtype is part of the key (an A lookup doesn't match a cached AAAA entry).

A negative (NXDOMAIN) entry with NULL/0 answers — this specifically exercises the `memcpy` NULL-guard mentioned in `dns_cache.c`.

TTL clamping: a 1-second TTL comes back at least `CACHE_MIN_TTL`.

Multi-answer TTL: a set with a 900s and a 60s answer gets served at the 60s ceiling (minimum wins).

Overwrite-in-place: re-putting the same key doesn't grow the entry count, and leaves exactly the newly-put answers (not a merge).

Eviction under pressure: inserts `CACHE_CAPACITY + 100` distinct entries and checks the live count never exceeds capacity and that evictions actually happened (not just silently dropped or unbounded growth).

Output truncation: requesting into a 1-slot buffer when 2 answers are cached returns exactly 1, without overrunning the caller's buffer.

This layer never touches sockets, threads, or the resolver's actual upstream logic — it's purely "does the codec and cache do what they claim."

SIMPLIFY AND EXPLAIN

This test suite (`test_dns_msg.c`) isolates and verifies the core data structures of a DNS resolver—specifically the wire-format parsing, message construction, and caching logic—without requiring a network, threads, or elevated privileges. It uses a lightweight, custom test harness optimized for Continuous Integration (CI) memory sanitizers.

Here is the breakdown of what the suite validates:

Wire-Format Parsing & Security

Name Encoding/Decoding: Ensures DNS names are properly translated to and from the wire format (e.g., converting "www.example.com" into length-prefixed labels). It verifies case-insensitivity and strictly enforces protocol limits (rejecting labels over 63 bytes or names over 255 bytes).

Compression Pointer Defenses: Targets the pointer-following logic used to compress DNS names, a classic hiding spot for parsing vulnerabilities. It feeds the parser adversarial inputs—like infinite loops, forward jumps, truncated pointers, and out-of-bounds lengths—to ensure they are safely rejected.

Record Parsing: Validates that incoming queries and resource records (RRs) are read correctly, throwing out malformed packets (e.g., truncated questions, invalid opcodes, or buffer overflows).

Message Construction

Response Building: Uses a "poisoned" (0xFF-filled) buffer to catch uninitialized memory reads. It verifies that single answers, NXDOMAIN (not-found) responses, and complex CNAME chains are built accurately.

DNS-0x20 Preservation: Confirms that the parser reads mixed-case queries (e.g., "WwW.eXaMPlE.com") internally as lowercase, but the response builder accurately echoes the exact original casing back to the client as a security measure.

Safe Truncation: Ensures that if a response is larger than the available buffer, it safely sets the Truncated (TC) flag and omits the answer section entirely, preventing corrupted, half-written records from leaking to the network.

Cache Logic

Retrieval & Matching: Confirms cache hits and misses, ensuring lookups are case-insensitive but strictly type-matched (an IPv4 A record query won't return a cached IPv6 AAAA record).

TTL (Time-To-Live) Management: Verifies that cached items age dynamically based on the system clock, enforces minimum TTL limits to prevent cache thrashing, and ensures that a multi-record answer inherits the lowest TTL in the set.

Eviction & Safety Limits: Tests the cache under pressure to confirm that old entries are properly evicted once the maximum capacity is reached, validates negative (NXDOMAIN) caching guards, and ensures the cache won't overrun a client's buffer if there are more cached answers than requested slots.

Layer 2 — the Python script: integration test of the live iterative resolver

This is a black-box test: it sends real UDP DNS queries over the wire to a running resolver process and checks the wire-format responses, so it's testing resolver.c's actual behavior (referrals, glue, CNAME chains, NXDOMAIN, caching) end-to-end, not just isolated functions.

Setup assumption (stated in the docstring): a companion `fake_hierarchy.py` is running a synthetic root/TLD/authoritative hierarchy, and the resolver under test is listening on 127.0.0.1:5354 with `DNS_ROOT_HINTS=127.0.0.11` pointing at that fake hierarchy instead of the real internet — this is exactly the override mechanism built into `dns_resolver_init()` in `resolver.c`. make offline presumably wires both processes up together. This lets the test exercise deep delegation/glue/CNAME logic deterministically and fast, rather than depending on the real DNS system's current state and multi-second round trips.

Helpers: `encode_query()` builds a raw query by hand (independent of the C encoder, which matters — it means the test isn't just checking that the C code agrees with itself).

`decode_name()` is a small hand-rolled name decoder that follows compression pointers (with its own depth guard against loops). `ask()` sends a UDP packet and waits for a reply, returning (None, elapsed) on timeout rather than raising, so callers can assert "no reply" as an expected outcome. `parse_answers()` decodes the answer section into (owner, type, value) tuples.

Test cases, each targeting a specific scenario from `resolver.c`'s logic:

Plain referral chain with glue at every level (`www.example.com`) — checks the transaction ID is echoed, QR set, TC not set, rcode NOERROR, and the single answer is exactly the expected A record.

Referral with no glue (`host.example.net`) — per the comment, the `.net` delegation names an NS (`ns1.example.com`) without a glue A record, forcing the resolver down the `resolve_ns_addr()` path in `resolver.c`. Checks it still resolves to the right answer.

Single CNAME (`alias.example.com`) — checks the client gets both the CNAME record and the terminal A record, in order, matching the "return every hop" requirement noted in `resolver.c`'s top comment.

Two-hop CNAME chain (`deep.example.com`) — checks exactly 3 records come back in the order CNAME, CNAME, A.

Authoritative NXDOMAIN (`missing.example.com`) — checks rcode is specifically 3 (NXDOMAIN), not SERVFAIL, and zero answers — this exercises the "an authoritative no is not our failure" distinction from `dns_resolve()`.

Cache hit — repeats the first query and checks it comes back in under 50ms, i.e. fast enough that it could only have been served from `dns_cache.c` rather than walked through the hierarchy again.

DNS-0x20 mixed case — sends a query with randomized casing and checks the question section comes back byte-for-byte identical, then that it still resolves correctly — this is the response-echo behavior tested at the unit level in `test_build_response()`, now verified against the live server.

Malformed input — a 3-byte garbage packet and a query with its QNAME chopped off mid-label must both produce no reply at all (matching the `DNS_DROP` behavior in `dns_backend.c`, where an unparseable query gets nothing back rather than a garbage response), and critically, a final well-formed query afterward must still succeed — proving the malformed input didn't crash or wedge the server.

The script accumulates failures in a list and exits nonzero if any occurred, printing ok/FAIL per check as it goes — a lightweight version of the same pattern as the C harness.

This Python script performs "black-box" integration testing by sending real network traffic (UDP packets) to a running instance of the DNS resolver. Unlike the unit tests that check isolated functions, this script tests the entire system end-to-end to verify how it handles complex, real-world internet routing.

To make the tests fast and 100% predictable, the script does not use the real internet. Instead, it pairs with a companion script (`fake_hierarchy.py`) that simulates a miniature, private version of the global DNS system. The resolver is forced to ask this fake internet for answers.

Crucially, the Python script builds and decodes the raw DNS messages itself from scratch. Because it does not reuse the C code being tested, it guarantees a truly independent verification—the C program isn't just grading its own homework.

The script tests the resolver against several specific network scenarios:

Standard Resolution: Successfully walking the DNS tree from the root servers down to the final answer.

Missing "Glue": Forcing the resolver to pause and look up the IP address of a nameserver because it wasn't provided alongside the delegation instructions.

Alias Chains (CNAMEs): Requesting a domain that is just an alias for another domain, and verifying the resolver tracks down the final IP address while returning the complete paper trail of jumps.

Definitive "Not Found" (NXDOMAIN): Checking that when an authoritative server says a domain absolutely does not exist, the resolver correctly reports that specific status rather than a generic server failure.

Cache Performance: Repeating a query and measuring the response time. It must return in under 50 milliseconds, proving the answer came directly from memory rather than the network.

Case-Sensitivity (DNS-0x20): Sending a query with randomized capital letters (e.g., `wWw.eXaMpLe.CoM`) to verify the resolver echoes the exact casing back to the client, a vital security feature to prevent forged responses.

Crash Resistance: Throwing severely broken or incomplete packets at the server. The resolver must silently ignore the garbage data without crashing or freezing, and immediately answer the next valid query.

Want me to explain what DNS glue records are?

Yes

Layer 3 — `run_tests.sh`: the benchmark harness

This one isn't correctness testing at all — it's a performance-measurement harness comparing the `AF_XDP` transport against the plain-UDP transport, and its own comments are candid about two methodology traps it fell into and fixed.

Two-phase design, and the script's comment is explicit about why both phases matter:

Phase 1 — transport isolation. Uses stub binaries (`af_xdp_user_stub`, `server_stub` — presumably a build using a fake zero-latency backend instead of the real resolver) at high concurrency (500 outstanding). With upstream latency removed, any difference between the AF_XDP and UDP arms can only come from the packet path itself — this is described as "where the project's claim actually lives."

Phase 2 — end to end. Uses the real resolver binaries, so both arms are now dominated by the same upstream DNS round trips. The comment sets an explicit expectation: the two arms should land close together here, and if there's a large gap, that's suspicious rather than a win — worth checking cache hit rates first, since a lucky cache-warm run could fake a speedup that has nothing to do with the transport.

Two bugs the script's own comments call out as previously wrong, now fixed:

Profiling the wrong process. `perf -p 1` under `bash -c "CMD"` only gets the actual target process if CMD is a simple command, because bash only `exec`-replaces itself (keeping the same PID) for simple commands — a compound command like `gcc x.c && ./x` leaves PID 1 as bash itself, idling, while the real process runs under a different PID. The fix: everything is precompiled ahead of time (no compile step in the timed path), and `perf_wrap()` finds the PID by process name via `pgrep -x`, not by assuming PID 1.

Insufficient load. `-q 10` (10 outstanding queries) against a resolver with ~1-second upstream latency works out to about 2 queries/second — at that rate the packet-handling path (which the whole project is trying to measure) is microseconds inside a second-long transaction, so the measurement is dominated by root-server response-time variance instead. Hence `CONC=500` for phase 1's transport-isolation runs (dropped to `CONC=50` for phase 2, since there it's fine — even expected — for real upstream latency to dominate).

Mechanics:

`perf_wrap()` runs `perf stat -t <pid> ... -o out -- sleep 5` in the background for 5 seconds against the located PID, using `-t` specifically so worker threads are counted too, not just the main thread.

`load()` prefers `dnsperv` if installed, falling back to a custom `./loadgen` binary otherwise; both are run inside a network namespace (`ip netns exec "$CLIENT_NS"`) to simulate a separate client host talking over `veth0`.

run_xdp()/run_udp() each: start the server binary in the background, sleep briefly for startup, kick off the perf capture, run the load generator (capturing output to \$OUT/\$label.client), then send SIGINT and wait for graceful shutdown — which is what triggers each binary's own stats printout (print_stats() in af_xdp_user.c, the equivalent block in server.c) into \$OUT/\$label.server.

Phase 1 runs four configurations: XDP in poll mode, XDP in busy-poll mode, XDP forced into copy mode (not zero-copy), and the plain UDP stub — giving a matrix that separates the poll-vs-busy-poll and zerocopy-vs-copy variables from the transport choice itself.

Phase 2 runs just one XDP config and the UDP baseline, both against the real resolver.

All results land in a timestamped results/ subdirectory (.server output,

This shell script (run_tests.sh) is not about checking for bugs or correctness; it is a dedicated performance benchmarking tool. Its primary goal is to measure how much faster the software processes network traffic using a high-performance framework called AF_XDP compared to standard UDP networking.

The script is divided into two distinct testing phases to ensure accurate measurements:

Phase 1: The Raw Speed Test (Transport Isolation)

This phase measures the absolute maximum speed of the network code. It swaps out the real DNS logic for a "stub" (a fake, instant-reply backend) and hits the server with massive concurrency (500 simultaneous requests). By removing the slow process of actually looking up DNS records on the internet, any difference in speed is guaranteed to be purely from the network transport layer (AF_XDP vs. UDP). The script tests four variations here: standard UDP, XDP in "copy" mode, XDP in "zero-copy" poll mode, and XDP in ultra-fast "busy-poll" mode.

Phase 2: The Real-World Test (End-to-End)

This phase tests the real, fully-functional DNS resolver under normal conditions (50 simultaneous requests). Here, the script actually expects AF_XDP and UDP to perform very similarly. Why? Because the time it takes to process a packet is microscopic compared to the time it takes to ask a root server across the country for a DNS record. The script notes that if AF_XDP suddenly looks vastly faster in this phase, it is likely a flawed test (such as one run getting lucky with cached memory) rather than a true network speedup.

Methodology Fixes (Lessons Learned)

The script's comments explicitly detail two major mistakes that were fixed to make the benchmarks trustworthy:

Targeting the wrong process: Previously, the Linux performance monitor (perf) was accidentally measuring the background bash shell instead of the actual DNS server. This was fixed by

separating the compilation step from the running step, and using exact process names (pgrep -x) to attach the profiler to the correct target and its worker threads.

Testing internet lag instead of code speed: Initially, the script only sent 10 concurrent requests. Because internet lookups take time, this translated to a measly 2 queries per second. At that low volume, the test was completely blind to network layer efficiency. Pumping the concurrency to 500 in Phase 1 solved this.

How the Script Works Under the Hood:

Simulated Network: It uses Linux network namespaces to pretend the traffic generator is a completely separate computer communicating over a virtual cable (veth0).

Traffic Generation: It prefers using the industry-standard dnsperf tool to blast the server with traffic, falling back to a custom script if necessary.

Data Collection: For each test, it boots the server, attaches the perf profiler, runs the heavy load, and then sends a graceful stop signal (SIGINT). This clean shutdown tells the server to dump its own internal performance statistics to a timestamped results/ directory for analysis.

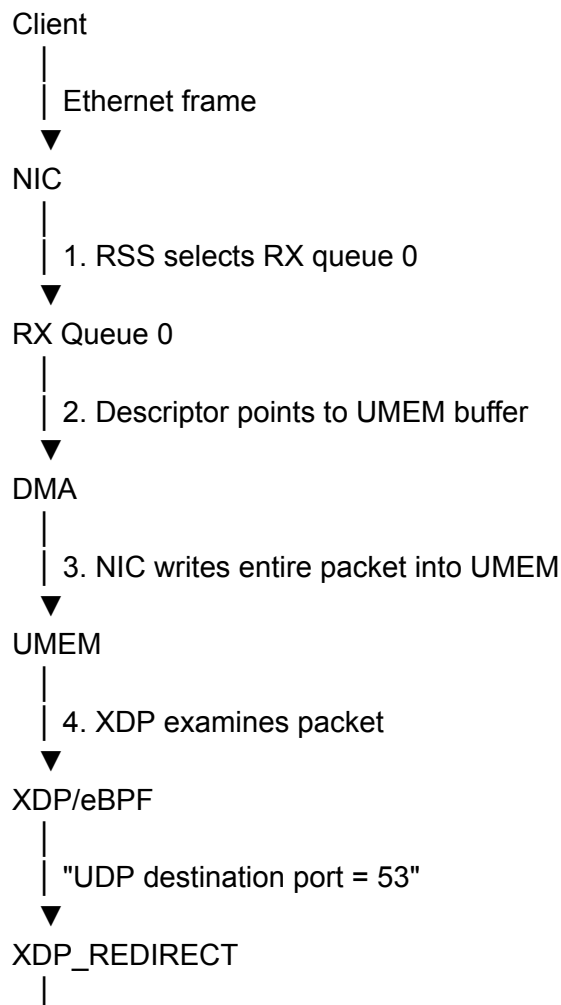
Same section we covered before (RSS/NAPI/XDP hook) — here's the short-points version. Step 1: Packet arrives, NIC picks a queue NIC computes an RSS hash (from IP/port info) to decide which RX queue gets the packet. Your project uses just 1 RX queue — so this step barely matters here, but it's how real multi-core NICs spread load. Step 2: RX queue = ring of DMA descriptors Each RX queue is a hardware ring of descriptors. Each descriptor = a pre-set DMA address pointing to a buffer. In zero-copy AF_XDP, that address points into UMEM (your shared memory), not kernel memory. Step 3: NIC writes packet via DMA NIC writes the packet directly to that pre-set DMA address (into UMEM). CPU does nothing during this write. Step 4: No per-packet interrupts Instead of interrupting the CPU for every packet, drivers use: NAPI poll mode, or AF_XDP busy-poll mode CPU checks periodically instead of being interrupted constantly → much more efficient at high packet rates. Step 5: XDP hook runs Once a batch of packets is ready, kernel calls your XDP hook function (a function pointer set up when you attached your eBPF program). Your program gets an xdp_md struct = just offsets into the UMEM buffer (start/end of this packet) — no data copied to give you this. Step 6: Your program decides — DNS packet? If it's UDP dst port 53, your program calls: `c bpf_redirect_map(&xsks_map, rx_queue_index, 0);` xsks_map = a table mapping RX queue → target AF_XDP socket. Your eBPF code does not move data, copy memory, or touch the socket itself — it just picks a target and returns XDP_REDIRECT. Step 7: Kernel (not your code) does the actual redirect Takes the same UMEM address the NIC already DMA'd into. Pushes that address into the target socket's RX ring. Key facts: Packet is not copied. Only the address goes into a lock-free ring. NIC's job ends right after DMA — it's not involved anymore. Step 8: Userspace picks it up Your app polls the AF_XDP socket's RX ring. New address appears →

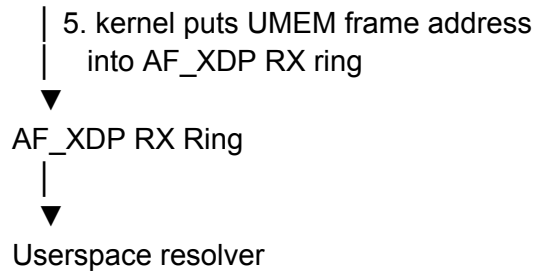
you read/process the packet directly from UMEM. Contrast with normal sockets Normal path: NIC → DMA into kernel memory → wrapped in sk_buff → passed through IP/UDP layers → then copied into userspace. AF_XDP zero-copy skips: sk_buff creation/traversal across layers Per-packet syscalls / kernel↔user transitions Final kernel→user memory copy in this once the packet is identified as DNS packet, still it is DMA'd like any other query and then since it is DNS it is also stored address in RX ring ? is it the way it works? \# 1. Build the image docker build -t dns_ax . # 2. Run the server on a host port (5300 is usually free) docker run -d -p 5300:53/udp --name dns_server dns_ax ./server -p 53 -w 4 # 3. Query from your host (outside the container) dig @127.0.0.1 -p 5300 www.google.com dig @127.0.0.1 -p 5300 github.com dig @127.0.0.1 -p 5300 example.com # 4. Stop when done docker stop dns_server docker rm dns_server how to test this by setting up

Yes — **your understanding is essentially correct**, with one important distinction about the two RX rings.

What happens to a DNS packet

Suppose a DNS request arrives:





So **yes**:

The NIC does **not know that it is a DNS packet**. It DMA-writes the packet into the buffer regardless of its contents.

Only **after the packet is in memory** does XDP inspect it and determine:

Ethernet → IPv4 → UDP → destination port 53

If it is DNS, your eBPF program returns:

```
bpf_redirect_map(&xsk_map, queue_index, 0);
```

Then the kernel redirects that **same UMEM frame** to the AF_XDP socket.

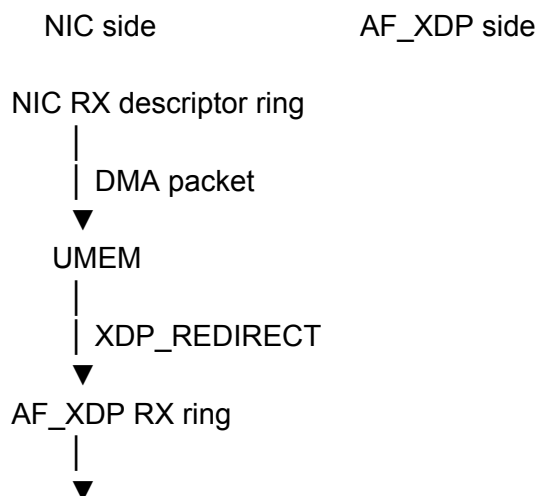
One subtle correction

When you say:

"it is also stored address in RX ring"

That's correct, but don't confuse the **NIC RX ring** with the **AF_XDP RX ring**.

There are effectively two stages:



userspace

The NIC's RX descriptor already contained the UMEM address **before the packet arrived**.

After XDP decides to redirect, the kernel places the corresponding frame/address into the **AF_XDP RX ring**.

So:

NIC RX ring:

"Where should I DMA this incoming packet?"

AF_XDP RX ring:

"Which UMEM frame is ready for userspace to process?"

Now, how to test your Docker DNS server

Your commands are basically the right sequence. Assuming your project directory contains the `Dockerfile`, `server`, source code, etc., let's do it step by step.

1. Go to your project directory

In WSL:

```
cd /path/to/your/project
```

Check:

```
ls
```

You should see something like:

```
Dockerfile
```

```
server
```

```
...
```

If `server` isn't already built, that's okay **if your Dockerfile builds it**.

2. Build the Docker image

Run:

```
docker build -t dns_ax .
```

You should eventually see something similar to:

Successfully built ...

Successfully tagged dns_ax:latest

Check:

```
docker images
```

You should see:

```
dns_ax
```

3. Start the DNS server container

Run:

```
docker run -d -p 5300:53/udp --name dns_server dns_ax ./server -p 53 -w 4
```

Breaking this down:

```
docker run
├── -d
│   └── run in background
├── -p 5300:53/udp
│   ├── host UDP port 5300
│   └── ▼
│       └── container UDP port 53
├── --name dns_server
└── dns_ax ./server -p 53 -w 4
    ├── your image
    └── listen on port 53
```

Check that it is running:

```
docker ps
```

You should see something like:

CONTAINER ID	IMAGE	COMMAND	STATUS
xxxxxx	dns_ax	"/server -p 53 -w 4"	Up ...

4. Check the server logs

This is important before testing DNS.

Run:

```
docker logs dns_server
```

If your server prints initialization information, you should see it there.

You can also watch logs continuously:

```
docker logs -f dns_server
```

Press:

Ctrl+C

to stop following the logs. This **doesn't stop the container**.

5. Send an actual DNS query

From your **host/WSL terminal**, run:

```
dig @127.0.0.1 -p 5300 www.google.com
```

You should get something like:

```
:: ANSWER SECTION:  
www.google.com.    ...  A    ...
```

Then test:


```
dig @127.0.0.1 -p 5300 github.com
```

and:

```
dig @127.0.0.1 -p 5300 example.com
```

The important part is:

```
@127.0.0.1
```

means:

Send the DNS query to localhost.

And:

```
-p 5300
```

means:

Use UDP port 5300.

Docker then translates:

Host

127.0.0.1:5300



Docker port mapping

Container

:53

6. Test whether TCP is also supported

Your Docker mapping is:

```
-p 5300:53/udp
```

So **only UDP** is exposed.

Your normal DNS test:

```
dig @127.0.0.1 -p 5300 google.com
```

will normally use UDP.

You can explicitly force UDP:

```
dig +notcp @127.0.0.1 -p 5300 google.com
```

This is useful because your AF_XDP project is specifically about **UDP DNS packets**.

7. The most important test for your project

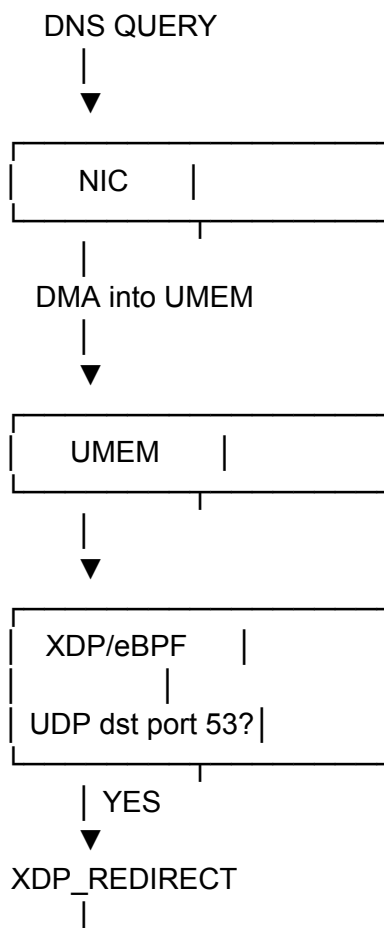
Since you're studying the AF_XDP path, don't just verify:

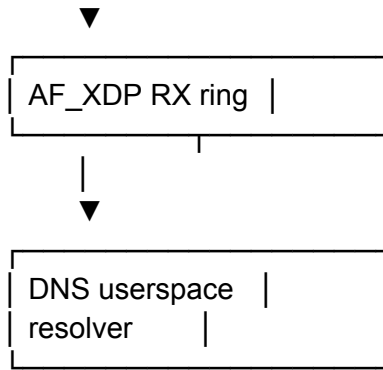
"Did I get a DNS answer?"

You want to verify:

Did the packet actually go through the AF_XDP/XDP path?

Your architecture should look approximately like:





So ideally your server should print/debug something when it receives a packet through XSK.

For example, if your code has logging, you might see:

```
Received packet
queue = 0
length = 73
DNS query received
```

Then you can correlate that with:

```
dig @127.0.0.1 -p 5300 example.com
```

! But there is a major Docker consideration

This is the part I would check **before assuming your Docker test exercises AF_XDP**.

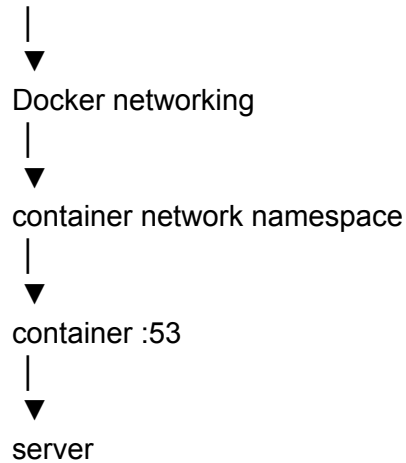
AF_XDP/XDP normally attaches to a **network interface/NIC inside the host/network namespace**.

When you run:

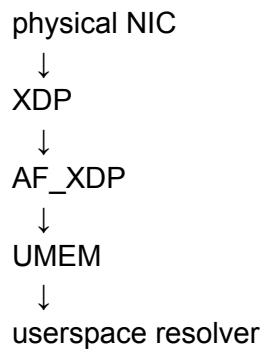
```
docker run -p 5300:53/udp ...
```

the packet path is roughly:

```
dig
|
▼
127.0.0.1:5300
```



That is **not automatically the same thing as:**



In other words:

Docker port forwarding can test that your DNS server works, but it does not necessarily test your XDP/AF_XDP datapath.

This distinction is extremely important for your project.

If your goal is simply:

"Does my Dockerized DNS server respond?"

Then your commands are perfect:

```
docker build -t dns_ax .
```

```
docker run -d \
  -p 5300:53/udp \
  --name dns_server \
  dns_ax ./server -p 53 -w 4
```

```
dig @127.0.0.1 -p 5300 example.com
```

If your goal is:

"Does my XDP → AF_XDP → UMEM → userspace pipeline work?"

Then we need to inspect **how your project attaches the XDP program and creates the AF_XDP socket**, because Docker's normal bridge/port-publishing setup may bypass the exact interface you're expecting.

If you show me your project's **Dockerfile + XDP setup/server startup code**, I can trace the exact packet path and give you the correct test setup.



how to understand this. codebase form where quickly so that we have understood the architecture, Now how to do it

Excellent — **your DNS resolver itself is now passing the entire integration suite.** 🎉

The important part is:

0 failures

and all the previously failing cases now pass:

- A resolution ✓
- Missing-glue resolution ✓
- CNAME → A chain ✓
- Deep CNAME → CNAME → A chain ✓
- NXDOMAIN ✓
- Cache hit ✓
- Case-insensitive DNS resolution ✓
- AAAA resolution ✓
- CNAME → AAAA ✓
- NODATA ✓
- Malformed/truncated queries ✓

What happened this time?

Interestingly, although you still got:

PermissionError: [Errno 13] Permission denied

the fake hierarchy **was already running from the previous attempt.**

So this invocation:

make offline

↓

tries to start fake_hierarchy.py

↓

cannot bind 127.0.0.11:53 because it is already occupied

↓

PermissionError

did **not** mean that the fake hierarchy was absent.

The already-running fake hierarchy continued serving the resolver, which is why everything passed.

You can see that from:

```
upstream sent 40
upstream t/o 0
```

and all the correct DNS answers.

The only remaining problem

This:

```
pkill: killing pid 2629 failed: Operation not permitted
make: *** [Makefile:98: offline] Terminated
```

is a **test-harness cleanup problem**, not a DNS resolver problem.

Your `make offline` script probably starts processes and then tries to clean them up with something like:

```
pkill ...
```

But the fake hierarchy was started with a different privilege level / user, so your normal shell cannot kill its process.

In other words:

DNS implementation	✓
Unit tests	✓ 146/146
Integration tests	✓ 28/28
Resolver behavior	✓
Fake hierarchy	✓ (but manually/previously running)
Cleanup script	⚠ permission issue

Confirm the fake hierarchy is still running

Run:

```
sudo ss -ltnp | grep ':53'
```

You should see the three listeners:

```
127.0.0.11:53
127.0.0.12:53
127.0.0.13:53
```

Then clean them up with:

```
sudo pkill -f fake_hierarchy.py
```

You may also have the baseline server left behind. Check:

```
ps aux | grep -E 'fake_hierarchy|server'
```

Your test result is actually very good

Your final resolver statistics:

```
cache hits    3
cache misses  8
hit rate      27.3%
evictions     0
expired       0
live entries   7
upstream sent 40
upstream t/o  0
```

are consistent with the test exercising multiple resolution paths.

Most importantly:

```
upstream t/o  0
```

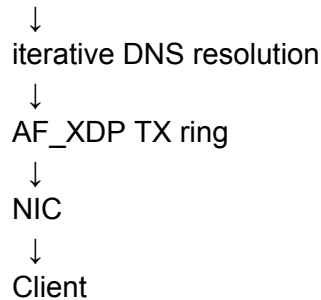
means none of the iterative upstream requests timed out.

So at this point, **I would not modify `resolver.c` or the DNS logic based on this output.**

Next step

Now that the conventional resolver is validated, you can move to the **AF_XDP** path:

```
Client
↓
NIC
↓
XDP/eBPF classifier
↓
AF_XDP RX ring
↓
your userspace resolver
```

The `make offline` result establishes that your **DNS backend/resolver logic works independently of the transport**. That's exactly what you want before testing AF_XDP, because then failures in the AF_XDP experiment can be attributed much more confidently to the packet transport rather than DNS resolution.

in step 4: verify the XDP program: `> make verify bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test && \ echo "verifier OK" && rm -f /sys/fs/bpf/dns_test /bin/sh: 1: bpftool: not found make: *** [Makefile:104: verify] Error 127 /mnt/c/Users/Adil/Desktop/dns_ax/dns_ax | on master !7 ?14 ..... at 08:36:16 PM >`

This is straightforward: **bpftool is not installed**.

Your XDP object was already compiled successfully, because `make verify` reached:

```
bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

and then failed with:

```
bpftool: not found
```

1. Install **bpftool**

Run:

```
sudo apt update  
sudo apt install bpftool
```

Then verify:

```
bpftool version
```

You should get something like:

```
bpftool v...
```

2. Run verification again

make verify

Expected:

verifier OK

What this does is important:

af_xdp_kern.o



bpftool prog load



Linux eBPF verifier



"Is this eBPF program safe and valid?"



verifier OK

So this is **not testing whether your DNS resolver works**. Your `make offline` already tested the DNS logic.

`make verify` specifically checks whether the **kernel accepts your XDP/eBPF program**.

If `bpftool` still isn't found

Check:

which bpftool

and:

bpftool version

If Ubuntu's package isn't available for your particular WSL setup, tell me the output of:

uname -a

lsb_release -a

and I'll give you the appropriate installation route.

After `make verify` passes, you're basically ready to move from **DNS correctness testing** → **actual AF_XDP packet-path testing**.

Thursday 11:47 AM

```
make verify make: *** No rule to make target 'verify'. Stop. > cd
/mnt/c/Users/Adil/Desktop/dns_ax/dns_ax make verify bpftool prog load af_xdp_kern.o
/sys/fs/bpf/dns_test && \ echo "verifier OK" && rm -f /sys/fs/bpf/dns_test libbpf: Failed to bump
RLIMIT_MEMLOCK (err = -EPERM), you might need to do it explicitly! libbpf: Error in
bpf_object__probe_loading(): -EPERM. Couldn't load trivial BPF program. Make sure your
kernel supports BPF (CONFIG_BPF_SYSCALL=y) and/or that RLIMIT_MEMLOCK is set to big
enough value. Error: failed to load object file make: *** [Makefile:104: verify] Error 255
/mnt/c/Users/Adil/Desktop/dns_ax/dns_ax | on master !14 ?3
..... at 06:17:30 AM > how to
resolve this error
```

The **Makefile** problem is already fixed. The **current error is different**:

```
libbpf: Failed to bump RLIMIT_MEMLOCK (err = -EPERM)
libbpf: Error in bpf_object__probe_loading(): -EPERM
Error: failed to load object file
```

This means your **af_xdp_kern.o** compiled successfully, but when **bpftool** tries to load the eBPF program, the environment **doesn't allow the required BPF memory/resource limit**.

Since you're running this under **WSL2**, there are two likely issues: **RLIMIT_MEMLOCK** and/or WSL's BPF capabilities.

1. First check your current limits

Run inside Ubuntu/WSL:

```
ulimit -l
```

Also:

```
uname -a
```

and:

```
bpftool feature probe kernel | head -30
```

2. Try increasing the memlock limit

Run:

```
ulimit -l unlimited
```

Then verify:

```
ulimit -l
```

It should print:

unlimited

Now retry:

make verify

3. If `ulimit -l unlimited` itself fails

If you get something like:

```
bash: ulimit: max locked memory: cannot modify limit: Operation not permitted
```

then this isn't something you can fix just inside the shell. Your WSL environment is restricting the resource limit.

Check:

```
cat /proc/sys/kernel/unprivileged_bpf_disabled
```

and:

```
cat /sys/kernel/security/lockdown 2>/dev/null || echo "no lockdown"
```

Also check:

```
id
```

4. Important: you're using WSL2

Your project needs to do:

NIC

↓

XDP

↓

eBPF program

↓

XSKMAP

↓

AF_XDP socket

↓

your userspace C program

For the `make verify` step, you're specifically doing:

```
bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

That requires the **BPF syscall** and **permission to load BPF programs**.

WSL2 can support many Linux kernel features, but depending on your WSL/kernel configuration, BPF operations can be restricted.

5. Check whether BPF itself works

Run:

```
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

If that works, clean it:

```
sudo rm -f /sys/fs/bpf/dns_test
```

Then:

```
make verify
```

If **sudo** makes it work, the issue is primarily **permissions/resource limits for your normal user**.

6. Check BPF filesystem

Run:

```
mount | grep bpf
```

You ideally want something like:

```
bpf on /sys/fs/bpf type bpf ...
```

If nothing appears, try:

```
sudo mount -t bpf bpf /sys/fs/bpf
```

Then:

```
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

But there's one important thing

Before changing a bunch of WSL settings, run these **exact commands** and send me the output:

```
ulimit -l
uname -r
cat /proc/sys/kernel/unprivileged_bpf_disabled
mount | grep bpf
id
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

That will tell us whether we're dealing with:

1. **memlock limit**
2. **BPF permission**
3. **BPF filesystem**
4. **WSL kernel restriction**
5. or a combination.

Your **Makefile** itself is now fine—the failure is happening specifically when Linux tries to **load the eBPF object into the kernel**.

This output is actually telling us that **most of the network namespace setup already exists**. You are not starting from a clean state.

The important part is:

```
veth1@if4 ... UP, LOWER_UP
inet 10.11.0.2/24
```

So **ns1** already exists, and **veth1** is already inside it with the correct IP.

What happened

These errors:

RTNETLINK answers: File exists

and:

Cannot create namespace file "/run/netns/ns1": File exists

mean you already created **veth0/veth1** and **ns1** in an earlier attempt.

Then:

Cannot find device "veth1"

happened because **you were trying to move veth1 into ns1 again**. It's already there.

And:

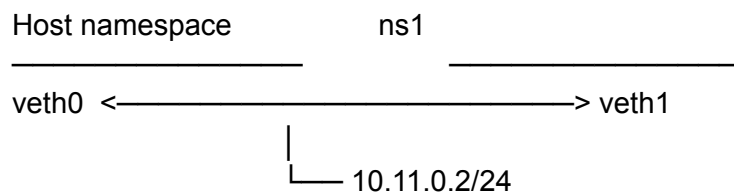
Error: ipv4: Address already assigned.

means **10.11.0.2/24** is already assigned—which your final **ip addr** output confirms.

So don't repeat those setup commands.

Your current state

You currently have:



And:

veth1 = UP

10.11.0.2/24 = configured

Check the host side

Run:

```
ip addr show veth0
```

You should see **veth0**.

Then give the host side an IP if it doesn't already have one:

```
sudo ip addr add 10.11.0.1/24 dev veth0
```

```
sudo ip link set veth0 up
```

If you get:

RTNETLINK answers: File exists

that's fine—the address is already configured.

Check:

```
ip addr show veth0
```

You want:

```
inet 10.11.0.1/24
```

Then test connectivity

From the host:

```
ping -c 3 10.11.0.2
```

From **ns1**:

```
sudo ip netns exec ns1 ping -c 3 10.11.0.1
```

If both work, your veth pair is correctly connected.

Check the namespace

Run:

```
sudo ip netns exec ns1 ip addr
```

You already have:

```
veth1@if4  
inet 10.11.0.2/24
```

You should also bring loopback up:

```
sudo ip netns exec ns1 ip link set lo up
```

So your next commands should be

Don't recreate anything. Just run:

```
ip addr show veth0
```


then:

```
sudo ip addr add 10.11.0.1/24 dev veth0
sudo ip link set veth0 up
```

and test:

```
ping -c 3 10.11.0.2
```

If that works, you're ready for the next part of the AF_XDP test setup.

```
> sudo ./af_xdp_user \ -d veth0 \ --filename ./af_xdp_kern.o \ -S sudo: './af_xdp_user': command
not found ~
```

```
.....
..... at 06:28:47 AM >
```

This one is simpler: **the `af_xdp_user` executable does not exist in your current directory (or isn't built yet).**

Your command is correct in principle:

```
sudo ./af_xdp_user \
  -d veth0 \
  --filename ./af_xdp_kern.o \
  -S
```

but `./af_xdp_user` means “run the executable named `af_xdp_user` from the current directory.”

1. Check whether it exists

Run:

```
ls -lh af_xdp_user*
```

Also:

```
ls -lh af_xdp_kern.o
```

If you see `af_xdp_kern.o` but **not** `af_xdp_user`, then you need to build it.

2. Build the project

From:

```
cd /mnt/c/Users/Adil/Desktop/dns_ax/dns_ax
```

run:

make

Then check:

ls -lh af_xdp_user

If the build succeeds, you should have something like:

af_xdp_kern.o

af_xdp_user

3. Then run your command again

```
sudo ./af_xdp_user \  
-d veth0 \  
--filename ./af_xdp_kern.o \  
-S
```

If **make** fails

Don't try random fixes yet. Run:

make

and send me the **full error**, especially the first compiler/linker error.

Also, because you're testing AF_XDP on **veth0**, there may be another issue after the executable starts: **AF_XDP support depends on how the XDP program attaches to the veth interface and how the XSK/UMEM is configured**. We'll handle that after **af_xdp_user** actually launches.

```
> sudo xdp-loader status [sudo: authenticate] Password: sudo: 'xdp-loader': command not found  
~
```

```
..... took 4s | at 06:30:53 AM >
```

That's another **missing tool**, not an AF_XDP program error.

sudo: 'xdp-loader': command not found

means **xdp-loader** is not installed, or it isn't in your **\$PATH**.

1. Check whether it's installed under another path

Run:

```
which xdp-loader  
find /usr -name xdp-loader 2>/dev/null
```

If both return nothing, install/build the **xdp-tools** package.

2. On Ubuntu/WSL, try

```
sudo apt update  
sudo apt install xdp-tools
```

Then:

```
which xdp-loader
```

You should get something like:

```
/usr/sbin/xdp-loader
```

Then:

```
sudo xdp-loader status
```

3. If **apt** says the package isn't available

Check your Ubuntu version:

```
cat /etc/os-release
```

and:

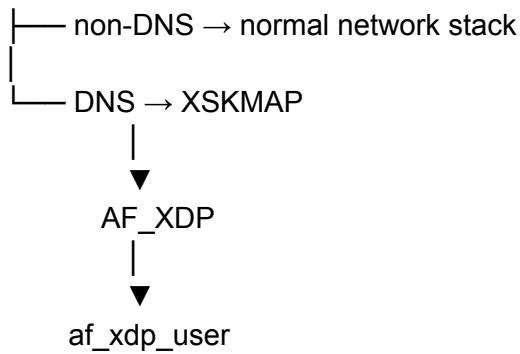
```
apt-cache search xdp-tools
```

Send me the output if **xdp-tools** isn't found.

What **xdp-loader status** is for

In your project, you're trying to get:

```
veth0  
|  
▼  
XDP program  
|
```



`xdp-loader status` lets you check whether an XDP program is currently attached to `veth0`.

Once `xdp-loader` is installed, you can check specifically:

```
sudo xdp-loader status veth0
```

But **don't worry about `xdp-loader` yet if your project already has `af_xdp_user`**. Your immediate previous error showed that `af_xdp_user` itself wasn't built/found. I'd get `make` producing the executable first, then attach the XDP program.

Yes — the **root cause is very clear** from this output. Your resolver itself is probably not the problem yet.

What is actually happening

The first error is:

```
starting fake root/TLD/auth hierarchy...
Traceback ...
self.sock.bind((addr, 53))
PermissionError: [Errno 13] Permission denied
```

Your offline test is designed to start three fake DNS servers:

```
127.0.0.11:53 → fake root
127.0.0.12:53 → fake .com
127.0.0.13:53 → fake authoritative server
```

The project documentation confirms that `make offline` is supposed to create exactly those servers and point `DNS_ROOT_HINTS` at `127.0.0.11`.

But **port 53 is a privileged port (<1024)**. Your WSL user doesn't have permission to bind it.

So the fake DNS hierarchy never starts.

Then your resolver does:

resolver → 127.0.0.11:53

but there is nothing listening there.

That explains almost every subsequent failure:

rcode NOERROR (got 2)

2 = **SERVFAIL**.

And therefore:

one answer (got 0)

correct A record (got [])

CNAME + A returned (got 0)

NXDOMAIN (got 2)

AAAA ... (got 2)

These are **cascading failures**, not 20 independent bugs.

Fix 1 — quickest solution

First check your current privileged-port setting:

```
sysctl net.ipv4.ip_unprivileged_port_start
```

You'll probably get:

```
net.ipv4.ip_unprivileged_port_start = 1024
```

For this offline test environment, you can allow unprivileged processes to bind low ports:

```
sudo sysctl -w net.ipv4.ip_unprivileged_port_start=0
```

Then verify:

```
sysctl net.ipv4.ip_unprivileged_port_start
```

It should say:

```
net.ipv4.ip_unprivileged_port_start = 0
```

Now run:

```
make clean
make
make offline
```

Expected beginning

Instead of:

```
starting fake root/TLD/auth hierarchy...
PermissionError: [Errno 13] Permission denied
```

you should get something like:

```
starting fake root/TLD/auth hierarchy...
resolver: using 1 override root hint(s)
...
iterative resolver integration tests
```

```
ok  www.example.com got a response
ok  transaction id echoed
ok  QR set
ok  TC not set
ok  rcode NOERROR
ok  one answer
...
```

Fix 2 — test the actual problem first

Before rerunning the entire test, you can verify whether your user can bind port 53.

Run:

```
python3 - <<'PY'
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("127.0.0.11", 53))
print("SUCCESS: can bind 127.0.0.11:53")
s.close()
PY
```

Currently, you'll probably get:

PermissionError: [Errno 13] Permission denied

After:

```
sudo sysctl -w net.ipv4.ip_unprivileged_port_start=0
```

run it again.

You should get:

SUCCESS: can bind 127.0.0.11:53

Then:

make offline

Why I don't recommend changing your resolver

Your resolver explicitly uses port 53 for upstream DNS:

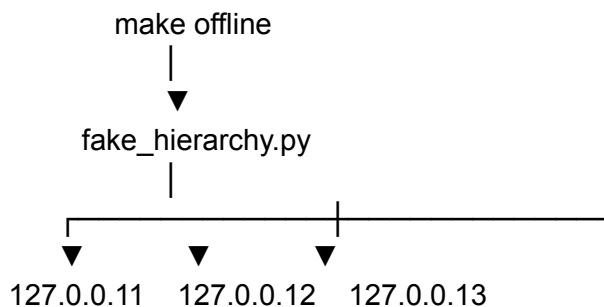
```
dst.sin_port = htons(53);
```

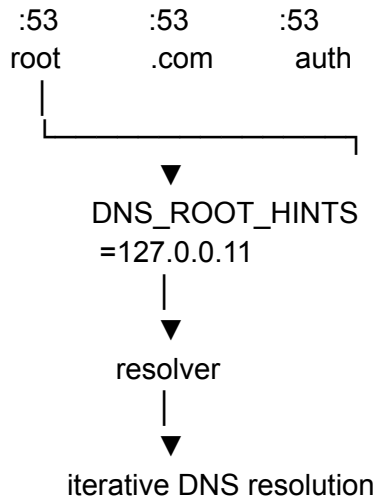
Your code confirms that.

And your `DNS_ROOT_HINTS` mechanism is specifically designed to replace the real root IPs with local fake root IPs.

So **don't change this to 5300 just to make the test pass** unless you also modify the fake hierarchy and resolver consistently.

The intended architecture is:





That's exactly what the offline test is intended to exercise: referrals, glue, missing glue, CNAME chains and NXDOMAIN.

There is another clue in your output

At the end:

```

--- dns backend ---
cache hits    0
cache misses  17
hit rate      0.0%
evictions     0
expired       0
live entries  0
upstream sent 17
upstream t/o  0
  
```

The important part is:

```

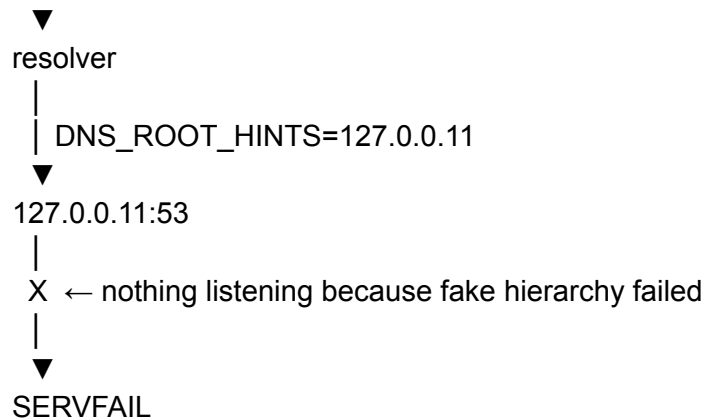
upstream sent 17
live entries  0
  
```

Your resolver is trying to resolve things, but the fake hierarchy isn't providing answers.

So the flow is effectively:

```

Client
|
  
```

What about this **IndexError**?

At the very end:

IndexError: list index out of range

This:

```
check(ans[-1] == (BIG_HOP3, "A", "10.0.0.2"),  
      ...
```

is **also a secondary failure**.

The test expects:

[CNAME, CNAME, A]

but because the resolver got:

SERVFAIL

it has:

```
ans = []
```

so:

```
ans[-1]
```

crashes.

Don't fix this first. Once the fake hierarchy starts working, this may disappear automatically.

Do this now

Run these **in order**:

```
sysctl net.ipv4.ip_unprivileged_port_start
```

then:

```
sudo sysctl -w net.ipv4.ip_unprivileged_port_start=0
```

then verify:

```
sysctl net.ipv4.ip_unprivileged_port_start
```

then:

```
python3 - <<'PY'
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("127.0.0.11", 53))
print("SUCCESS: port 53 works")
s.close()
PY
```

If that says:

```
SUCCESS: port 53 works
```

finally:

make offline

Do not change `resolver.c`, `dns_backend.c`, or the DNS logic yet. Your 146 unit tests already pass, and the integration failures are overwhelmingly explained by the fake DNS servers failing to start.

One note: this `sysctl` change is a system-wide WSL setting. If you don't want to change it, the simpler temporary workaround is to run the fake hierarchy with sufficient privilege, but that requires adjusting the `make offline` launch rather than just running the resolver as root.

```
> > make verify bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test && \ echo "verifier OK" &&
rm -f /sys/fs/bpf/dns_test libbpf: Failed to bump RLIMIT_MEMLOCK (err = -EPERM), you might
need to do it explicitly! libbpf: Error in bpf_object__probe_loading(): -EPERM. Couldn't load
trivial BPF program. Make sure your kernel supports BPF (CONFIG_BPF_SYSCALL=y) and/or
```

that RLIMIT_MEMLOCK is set to big enough value. Error: failed to load object file make: ***
[Makefile:104: verify] Error 255 /mnt/c/Users/Adil/Desktop/dns_ax/dns_ax | on master !14 ?3
..... at 06:12:38 AM >

This is a **different issue from make offline**. Your BPF object compiled successfully, but the kernel is refusing to load it because the process cannot raise its **BPF memory lock limit (RLIMIT_MEMLOCK)**.

The key line is:

```
libbpf: Failed to bump RLIMIT_MEMLOCK (err = -EPERM)
```

and then:

Error: failed to load object file

First try this

In your WSL Ubuntu terminal:

```
ulimit -l
```

If it prints something small like:

```
64
```

or:

```
8192
```

try:

```
sudo prlimit --memlock=unlimited:unlimited --pid $$
```

Then check:

```
ulimit -l
```

and run:

```
make verify
```

If **sudo prlimit** doesn't work

Try:

```
sudo sh -c 'ulimit -l unlimited; exec bash'
```

Then in the **new root shell**:

```
cd /mnt/c/Users/Adil/Desktop/dns_ax/dns_ax
make verify
```

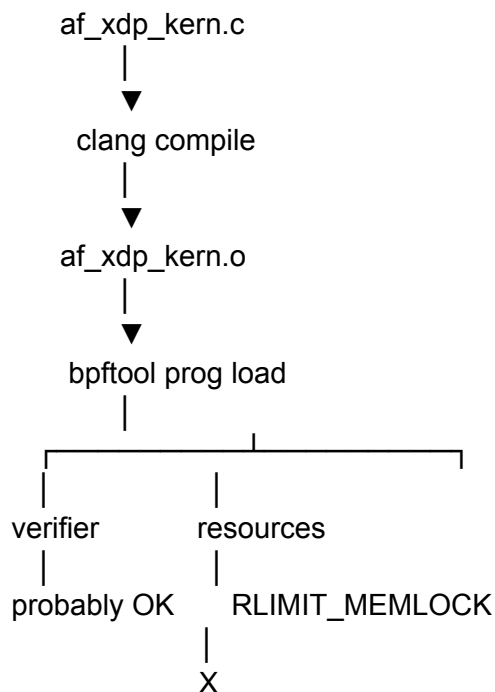
If this works, you'll see:

```
verifier OK
```

But there's an important WSL2 complication

You're running this project under **WSL2**, and your earlier environment is WSL 2.x. BPF/XDP support in WSL can have restrictions around capabilities and resource limits. Your README explicitly expects `make verify` to check that the BPF verifier accepts `af_xdp_kern.o`; the project itself is designed to use BPF/XDP.

So let's distinguish two things:



Your failure is currently occurring **before we can meaningfully test whether your BPF program passes verification**.

Check these 5 things

Run:

```
uname -a
bpftool version
ulimit -l
cat /proc/sys/kernel/unprivileged_bpf_disabled
capsh --print | grep -E 'Current[cap_bpf|cap_sys_admin']
```

Paste the output of those five commands.

Especially important

If you see:

```
unprivileged_bpf_disabled = 1
```

that can be another restriction.

Also check:

```
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

If that succeeds, immediately clean up:

```
sudo rm -f /sys/fs/bpf/dns_test
```

Then the problem is simply that your normal shell lacks the privileges required by [bpftool](#).

Don't modify your BPF code yet

This error does **not** indicate a problem with:

```
SEC("xdp")
int ...
```

or your DNS classification logic.

Your current failure is:

RLIMIT_MEMLOCK → EPERM → BPF object cannot be loaded

So let's fix the **environment/permissions first**, then see whether the verifier itself reports an actual BPF-program error.

```
> cd /mnt/c/Users/Adil/Desktop/dns_ax/dns_ax make verify bpftool prog load af_xdp_kern.o
/sys/fs/bpf/dns_test && \ echo "verifier OK" && rm -f /sys/fs/bpf/dns_test libbpf: Failed to bump
RLIMIT_MEMLOCK (err = -EPERM), you might need to do it explicitly! libbpf: Error in
bpf_object__probe_loading(): -EPERM. Couldn't load trivial BPF program. Make sure your
kernel supports BPF (CONFIG_BPF_SYSCALL=y) and/or that RLIMIT_MEMLOCK is set to big
enough value. Error: failed to load object file make: *** [Makefile:104: verify] Error 255
/mnt/c/Users/Adil/Desktop/dns_ax/dns_ax | on master !14 ?3
..... at 06:14:08 AM >
```

This is no longer a **bpftool not found** problem. **bpftool** is running now; the failure is happening before your XDP program reaches the verifier.

The important line is:

```
libbpf: Failed to bump RLIMIT_MEMLOCK (err = -EPERM)
```

So first check your current locked-memory limit:

```
ulimit -l
```

Then try loading the program as root directly:

```
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

If that works, your BPF object is probably fine and **make verify** is simply running **bpftool** without sufficient privilege. Clean up with:

```
sudo rm -f /sys/fs/bpf/dns_test
```

Then run the verification from a root shell with an unlimited memlock limit:

```
sudo bash
ulimit -l unlimited
```

```
cd /mnt/c/Users/Adil/Desktop/dns_ax/dns_ax
make verify
```

You want to eventually see:

```
verifier OK
```

If `ulimit -l unlimited` says "Operation not permitted"

Run:

```
sudo prlimit --pid $$ --memlock=unlimited:unlimited  
ulimit -l
```

Then retry:

```
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

If **sudo bpftool** still gives **EPERM**

Because you're on WSL2, check whether the WSL kernel actually permits the required BPF operations:

```
uname -a  
bpftool version  
ulimit -l  
cat /proc/sys/kernel/unprivileged_bpf_disabled  
sudo bpftool feature probe kernel
```

Also check the kernel config:

```
zcat /proc/config.gz | grep -E 'CONFIG_BPF|CONFIG_BPF_SYSCALL|CONFIG_XDP'
```

If **/proc/config.gz** doesn't exist:

```
grep -E 'CONFIG_BPF|CONFIG_BPF_SYSCALL|CONFIG_XDP' /boot/config-$(uname -r)
```

The key options should include something like:

```
CONFIG_BPF=y  
CONFIG_BPF_SYSCALL=y  
CONFIG_BPF_JIT=y
```

For your immediate case, run these **three commands first**:

```
ulimit -l  
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test  
sudo bpftool feature probe kernel
```

The output of those will tell us whether this is just the memlock/privilege issue or a WSL kernel restriction. Your current error does **not yet mean there is anything wrong with **af_xdp_kern.c** itself**.

Thursday 2:08 AM

```
> make verify bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test && \ echo "verifier OK" && rm
-f /sys/fs/bpf/dns_test libbpf: Failed to bump RLIMIT_MEMLOCK (err = -EPERM), you might
need to do it explicitly! libbpf: Error in bpf_object__probe_loading(): -EPERM. Couldn't load
trivial BPF program. Make sure your kernel supports BPF (CONFIG_BPF_SYSCALL=y) and/or
that RLIMIT_MEMLOCK is set to big enough value. Error: failed to load object file make: ***
[Makefile:104: verify] Error 255 how to resolve this issue
```

This is a **Linux BPF permission / RLIMIT_MEMLOCK issue**, not a problem with your `af_xdp_kern.c` code.

The important line is:

```
libbpf: Failed to bump RLIMIT_MEMLOCK (err = -EPERM)
```

Your `bpftool` is trying to load the eBPF program, but the environment doesn't allow it to increase the amount of memory that can be locked.

Since you're running this inside **WSL**, there are a few things to check.

1. First check your current limits

Run:

```
ulimit -l
```

and:

```
ulimit -a | grep locked
```

If you get something like:

```
64
```

then your locked-memory limit is only 64 KB, which is usually too small for BPF/AF_XDP work.

2. Try increasing the limit for the current shell

Run:

```
ulimit -l unlimited
```

Then verify:

```
ulimit -l
```


You should see:

unlimited

Now retry:

make verify

3. If **ulimit** itself gives **Operation not permitted**

For example:

bash: ulimit: max locked memory: cannot modify limit: Operation not permitted

then your WSL environment is preventing you from raising the limit.

Check:

```
cat /proc/self/limits | grep "Max locked memory"
```

You may see:

Max locked memory	65536	65536	bytes
-------------------	-------	-------	-------

This means your shell is capped at 64 KB.

4. Check whether BPF is actually supported

Run:

```
uname -r
```

and:

```
grep CONFIG_BPF_SYSCALL /boot/config-$(uname -r) 2>/dev/null
```

Ideally you should get:

```
CONFIG_BPF_SYSCALL=y
```

Also:

```
bpftool version
```

If `bpftool` now exists, that's good—you already got past the previous `bpftool: not found` problem.

5. Check whether you're running as root

Try:

```
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

If that works, the problem is primarily your user's BPF/memory-lock permissions.

However, **don't permanently run the whole resolver with `sudo` yet**. For AF_XDP, we want to configure the environment correctly rather than simply hiding permission problems with root.

Important WSL-specific point

Because you're doing an **AF_XDP/XDP project inside WSL**, there is another potential issue.

Run:

```
uname -a
```

and:

```
ip link
```

Your project requires:

NIC

↓

XDP

↓

AF_XDP socket

↓

your userspace DNS resolver

WSL's networking/virtual NIC setup can affect whether XDP/AF_XDP actually works, even after `make verify` succeeds.

But **your current error happens before that stage**. Right now the immediate failure is:

libbpf
↓
tries to load BPF object
↓
tries to raise RLIMIT_MEMLOCK
↓
EPERM
↓
BPF program not loaded

Try these commands now

Run them in this exact order:

```
ulimit -l  
ulimit -l unlimited  
ulimit -l
```

Then:

```
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

If that succeeds, clean it up:

```
sudo rm -f /sys/fs/bpf/dns_test
```

Then try:

```
make verify
```

If **ulimit -l unlimited** fails

Send me the output of these four commands:

```
uname -a  
ulimit -l  
cat /proc/self/limits | grep "Max locked memory"  
sudo bpftool prog load af_xdp_kern.o /sys/fs/bpf/dns_test
```

From those outputs, we can determine whether this is **just RLIMIT_MEMLOCK**, a **WSL kernel configuration issue**, or a **BPF privilege/capability issue**.

Pasted markdown(1).md
File

these are the testing steps i am follownig. i am on step 5. this is the error. > ./server -p 5300 -w
4 bind(5300): Address already in use >

Aha — **we found the cause.**

Windows itself has a UDP listener on port **5300**:

```
LocalAddress  LocalPort  
::           5300
```

:: means **all IPv6 interfaces**. Because you're using WSL2, this Windows listener can make port 5300 appear unavailable from your WSL environment.

1. Find which Windows process owns it

In PowerShell, run:

```
Get-NetUDPEndpoint -LocalPort 5300 | Select-Object LocalAddress,LocalPort,OwningProcess
```

You should now get something like:

```
LocalAddress LocalPort OwningProcess  
-----  
::          5300    1234
```

Then:

```
Get-Process -Id 1234
```

Replace **1234** with whatever PID you get.

2. If it's an old process you started

You can stop it with:

```
Stop-Process -Id 1234
```

Then check:

```
Get-NetUDPEndpoint -LocalPort 5300
```

If nothing is returned, go back to WSL and run:

```
sudo ss -ltnp | grep ':5300'
```

Then:

```
./server -p 5300 -w 4
```

If you don't recognize the process

Don't kill it yet. Paste the output of:

```
Get-NetUDPEndpoint -LocalPort 5300 | Select-Object LocalAddress,LocalPort,OwningProcess
```

and:

```
Get-Process -Id <PID>
```

I'll tell you what it is and whether it's safe to stop.

The important thing is that **your server code isn't necessarily the problem**. We've now established that port **5300** is already occupied on the Windows side, which explains why your WSL server gets:

```
bind(5300): Address already in use
(base) PS C:\Users\Adil> Get-Process -Id 5124 Handles NPM(K) PM(K) WS(K) CPU(s) Id SI
ProcessName -----
com.docker.backend (base) PS C:\Users\Adil> i think this is an old docker container i ran.
maybe\
```

Yes — **very likely**. **com.docker.backend** is part of Docker Desktop, and your old Docker-based DNS/server setup may have caused Docker to publish/hold UDP port **5300**.

But don't kill **com.docker.backend** directly. That's the Docker Desktop backend, so stopping PID **5124** may disrupt **all of Docker Desktop**.

First, check Docker's published ports

In **PowerShell**, run:

```
docker ps
```

Then:

```
docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Ports}}"
```

Look for something containing:

```
5300/udp
```

For example:

CONTAINER ID	NAMES	PORTS
abc123	dns-server	0.0.0.0:5300->5300/udp

If you find your old DNS container, stop it:

```
docker stop <container-name-or-id>
```

Then check:

```
Get-NetUDPEndpoint -LocalPort 5300
```

If it returns nothing, go back to WSL:

```
sudo ss -ltnp | grep ':5300'
```

and finally:

```
./server -p 5300 -w 4
```

If **docker ps** shows nothing

Also check stopped containers:

```
docker ps -a
```

If you recognize the old DNS container, you can stop/remove it as appropriate.

Don't uninstall/restart Docker Desktop yet. Let's identify the container that owns **5300** first. This is especially likely given that your project README previously used Docker for the conventional UDP server, while your current Step 5 is intentionally running **./server** directly.

Run:

```
docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Ports}}"
```

and paste the output.

```
(base) PS C:\Users\Adil> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
4d718835fbc0   dns_ax    "./server -p 53 -w 4"    28 minutes ago Up 28 minutes  0.0.0.0:5300->53/udp, [::]:5300->53/udp   dns_server
(base) PS C:\Users\Adil>
```

Exactly. **We found the culprit.** 🎯

Your Docker container:

```
dns_server
```

is currently running:

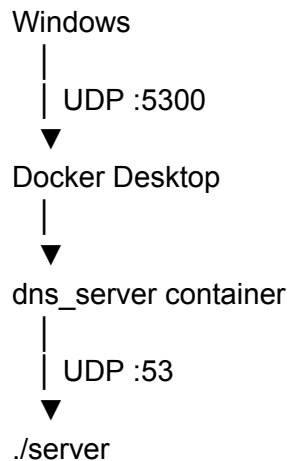
```
./server -p 53 -w 4
```

and Docker is publishing:

```
0.0.0.0:5300 -> 53/udp
```

```
:::5300 -> 53/udp
```

So the situation is:



Therefore when you run your **native WSL server**:

```
./server -p 5300 -w 4
```

it asks the kernel for UDP **5300**, but Docker already has that port published.

That's why you got:

```
bind(5300): Address already in use
```

Stop the old Docker container

Since this is apparently the old baseline container, run in PowerShell:

```
docker stop dns_server
```

You should get:

```
dns_server
```

Then verify:

```
Get-NetUDPEndpoint -LocalPort 5300
```

It should return **nothing**.

Then go back to WSL and check:

```
sudo ss -ltnp | grep ':5300'
```

Again, ideally **no output**.

Now run Step 5

```
./server -p 5300 -w 4
```

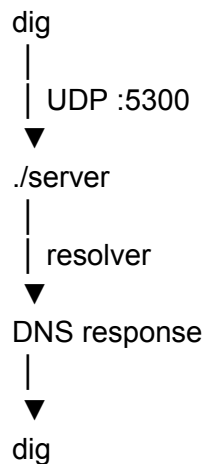
It should now start successfully.

You'll probably see your server's startup output and it will remain running. **Leave that terminal open.**

Then open another WSL terminal and run the test from your README:

```
dig @127.0.0.1 -p 5300 www.google.com
```

The expected flow is:



This is your **baseline UDP test**, before we touch the AF_XDP/veth setup.

And you don't need to remove the Docker container; just **stop** it for now:

```
docker stop dns_server
```

You can start it again later with:

```
docker start dns_server
```

if needed.

Tuesday 10:13 PM

Same section we covered before (RSS/NAPI/XDP hook) — here's the short-points version.

Step 1: Packet arrives, NIC picks a queue

- NIC computes an **RSS hash** (from IP/port info) to decide which RX queue gets the packet.
- Your project uses just **1 RX queue** — so this step barely matters here, but it's how real multi-core NICs spread load.

Step 2: RX queue = ring of DMA descriptors

- Each RX queue is a hardware ring of **descriptors**.
- Each descriptor = a pre-set DMA address pointing to a buffer.
- In zero-copy AF_XDP, that address points **into UMEM** (your shared memory), not kernel memory.

Step 3: NIC writes packet via DMA

- NIC writes the packet directly to that pre-set DMA address (into UMEM).
- **CPU does nothing** during this write.

Step 4: No per-packet interrupts

- Instead of interrupting the CPU for every packet, drivers use:
 - **NAPI poll mode**, or
 - **AF_XDP busy-poll mode**
- CPU checks periodically instead of being interrupted constantly → much more efficient at high packet rates.

Step 5: XDP hook runs

- Once a batch of packets is ready, kernel calls your **XDP hook function** (a function pointer set up when you attached your eBPF program).
- Your program gets an `xdp_md` struct = just **offsets** into the UMEM buffer (start/end of this packet) — no data copied to give you this.

Step 6: Your program decides — DNS packet?

- If it's UDP dst port 53, your program calls:

c

```
bpf_redirect_map(&xsk_map, rx_queue_index, 0);
```

- **xsks_map** = a table mapping RX queue → target AF_XDP socket.
- Your eBPF code does **not** move data, copy memory, or touch the socket itself — it just picks a target and returns **XDP_REDIRECT**.

Step 7: Kernel (not your code) does the actual redirect

- Takes the same UMEM address the NIC already DMA'd into.
- Pushes that address into the target socket's **RX ring**.
- Key facts:
 - Packet is **not copied**.
 - Only the **address** goes into a lock-free ring.
 - NIC's job ends right after DMA — it's not involved anymore.

Step 8: Userspace picks it up

- Your app polls the AF_XDP socket's RX ring.
- New address appears → you read/process the packet **directly from UMEM**.

Contrast with normal sockets

- Normal path: NIC → DMA into **kernel** memory → wrapped in **sk_buff** → passed through IP/UDP layers → **then** copied into userspace.
- AF_XDP zero-copy skips:
 1. **sk_buff** creation/traversal across layers
 2. Per-packet syscalls / kernel↔user transitions
 3. Final kernel→user memory copy

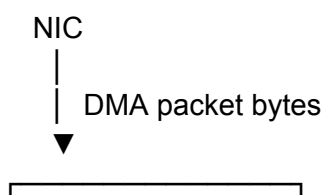
in this once the packet is identified as DNS packet, still it is DMA'd like any other query and then since it is DNS it is also stored address in RX ring ? is it the way it works?

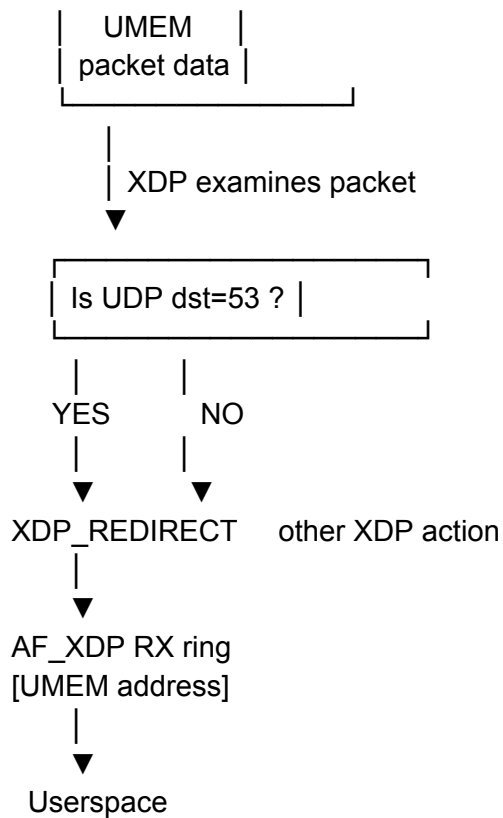
Yes — **that is essentially correct**, with one important timing detail.

The key idea is:

The packet is DMA'd into UMEM first. Only after that does the XDP program inspect it and decide whether its address should be redirected to the AF_XDP RX ring.

So for a DNS packet, the flow is:





The subtle but important point

The **DNS identification happens after DMA**, not before it.

Suppose the NIC receives:

Ethernet | IPv4 | UDP | DNS

The NIC doesn't normally say:

"Ah, this is DNS, I'll put it in the AF_XDP socket."

Instead:

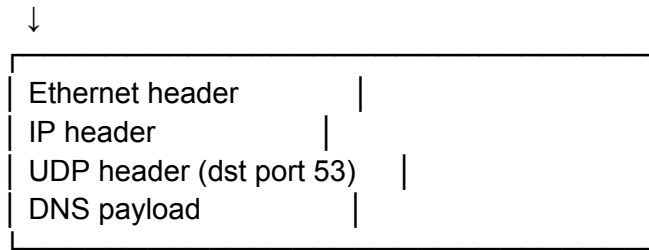
1. NIC receives packet

The NIC has an RX descriptor saying roughly:

"Put the incoming packet at UMEM address 0xABC000"

So it DMA-writes:

UMEM
0xABC000



2. XDP runs

Now the kernel invokes your eBPF/XDP program.

It looks at the **same packet bytes already sitting in UMEM**:

```
if (udp->dest == 53) {
    return bpf_redirect_map(&xsks_map, queue, 0);
}
```

It determines:

This is DNS

↓
 redirect this packet
 ↓
 XDP_REDIRECT

3. Kernel puts the address into the AF_XDP RX ring

This is the part you're asking about.

The kernel does **not** copy:

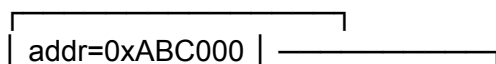
DNS packet bytes
 ↓
 another buffer

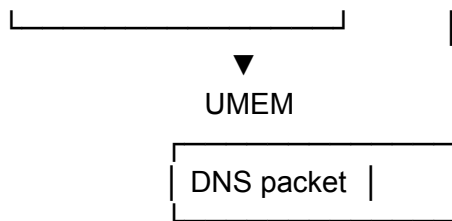
Instead, it effectively tells the AF_XDP socket:

"Packet is at UMEM address 0xABC000"

So the RX ring contains something conceptually like:

AF_XDP RX ring





Userspace reads the address from the RX ring and then accesses the packet **at that address** in **UMEM**.

So your statement can be made more precise

You said:

"once the packet is identified as DNS packet, still it is DMA'd like any other query and then since it is DNS it is also stored address in RX ring?"

Exactly.

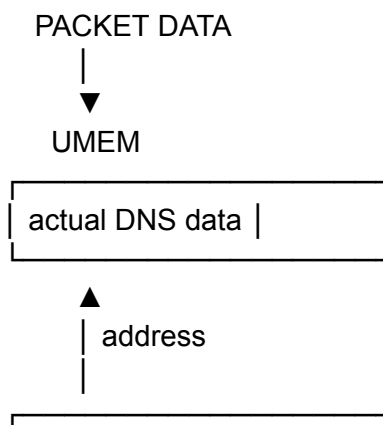
More precisely:

Every incoming packet is first DMA'd into the buffer specified by the RX descriptor. XDP then examines that packet. If it is a DNS packet, `bpf_redirect_map()` causes the kernel to place a descriptor/address referring to that same UMEM buffer into the AF_XDP RX ring. No packet data is copied into the RX ring.

And importantly:

RX ring $\neq$ packet storage.

The RX ring stores **metadata/address information**.



```
| AF_XDP RX ring |  
| addr = 0xABC000 |  
└────────────────┘
```

That's the central zero-copy idea: **the packet stays where the NIC put it; the rings mostly pass ownership/references to that memory around.**

Wednesday 6:29 PM

22.8 Approximating LRU As it turns out, the answer is yes: approximating LRU is more feasible from a computational-overhead standpoint, and indeed it is what many modern systems do. The idea requires some hardware support, in the form of a use bit (sometimes called the reference bit), the first of which was implemented in the first system with paging, the Atlas onelevel store [KE+62]. There is one use bit per page of the system, and the use bits live in memory somewhere (they could be in the per-process page tables, for example, or just in an array somewhere). Whenever a page is referenced (i.e., read or written), the use bit is set by hardware to 1. The hardware never clears the bit, though (i.e., sets it to 0); that is the responsibility of the OS. How does the OS employ the use bit to approximate LRU? Well, there could be a lot of ways, but with the clock algorithm [C69], one simple approach was suggested. Imagine all the pages of the system arranged in a circular list. A clock hand points to some particular page to begin with (it doesn't really matter which). When a replacement must occur, the OS checks if the currently-pointed to page P has a use bit of 1 or 0. If 1, this implies that page P was recently used and thus is not a good candidate for replacement. Thus, the use bit for P is set to 0 (cleared), and the clock hand is incremented to the next page ($P + 1$). The algorithm continues until it finds a use bit that is set to 0, implying this page has not been recently used (or, in the worst case, that all pages have been and that we have now searched through the entire set of pages, clearing all the bits). Note that this approach is not the only way to employ a use bit to approximate LRU. Indeed, any approach which periodically clears the use bits and then differentiates between which pages have use bits of 1 versus 0 to decide which to replace would be fine. The clock algorithm of Corbato's was just one early approach which met with some success, and had the nice property of not repeatedly scanning through all of memory looking for an unused page.

OPERATING SYSTEMS [VERSION 1.10] WWW.OSTEP.ORG
BEYOND PHYSICAL MEMORY: POLICIES 13 0 20 40 60 80 100 0% 20% 40% 60% 80% 100%
The 80-20 Workload Cache Size (Blocks) Hit Rate OPT LRU FIFO RAND Clock
Figure 22.9: The 80-20 Workload With Clock The behavior of a clock algorithm variant is shown in Figure 22.9. This variant randomly scans pages when doing a replacement; when it encounters a page with a reference bit set to 1, it clears the bit (i.e., sets it to 0); when it finds a page with the reference bit set to 0, it chooses it as its victim. As you can see, although it doesn't do quite as well as perfect LRU, it does better than approaches that don't consider history at all.

22.9 Considering Dirty Pages One small modification to the clock algorithm (also originally suggested by Corbato [C69]) that is commonly made is the additional consideration of whether a page has been modified or not while in memory. The reason for this: if a page has been modified and is thus dirty, it must be written back to disk to evict it, which is expensive. If it has not been modified (and is thus clean), the eviction is free; the physical frame can simply be reused for other purposes without additional I/O. Thus, some VM systems prefer to evict clean pages over dirty pages. To support this behavior, the hardware should include a modified bit (a.k.a. dirty bit).

This bit is set any time a page is written, and thus can be incorporated into the page-replacement algorithm. The clock algorithm, for example, could be changed to scan for pages that are both unused and clean to evict first; failing to find those, then for unused pages that are dirty, and so forth Explain this

Absolutely. This section is explaining **how operating systems approximate LRU (Least Recently Used)** when doing page replacement.

The key problem is:

True LRU is expensive to implement, so the OS uses hardware-maintained bits + the Clock algorithm to get a cheap approximation.

1. First: What is LRU?

Suppose physical memory has 4 pages:

A B C D

And the recent access history is:

$A \rightarrow C \rightarrow B \rightarrow A$

The most recently used page is:

A

The least recently used is:

D

So if we need to evict a page, **LRU chooses D**.

The problem is: how does the OS know the exact ordering?

It would need to track every page access:

A accessed $\rightarrow$ move A to front

C accessed $\rightarrow$ move C to front

B accessed $\rightarrow$ move B to front

A accessed $\rightarrow$ move A to front

Doing this for every memory access would be extremely expensive because programs perform **millions/billions of memory accesses**.

So instead, we approximate LRU.

2. Hardware gives us a Use/Reference bit

Each page gets a **use bit**:

Page	Use bit

A	1
B	0
C	1
D	0

The hardware automatically sets this bit to **1** whenever the page is accessed.

For example:

CPU accesses A



Hardware sets A.use = 1

Similarly:

CPU reads/writes C



Hardware sets C.use = 1

So:

A → use = 1

B → use = 0

C → use = 1

D → use = 0

Important:

Hardware sets the bit, but hardware doesn't clear it.

The OS is responsible for clearing it.

3. Why is this useful?

The use bit doesn't tell us exactly **when** a page was used.

It only tells us:

"Has this page been used since the OS last cleared its bit?"

For example, suppose the OS clears everything:

A = 0

B = 0

C = 0

D = 0

Then the program runs:

A → C → A → C → A

Hardware sets:

A = 1

C = 1

B = 0

D = 0

Now the OS knows:

A → recently used

C → recently used

B → not recently used

D → not recently used

It doesn't know whether:

A was used 1 ns ago

C was used 1 ms ago

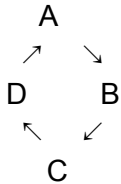
But it knows that **A and C have been used during the observation period**, while B and D haven't.

That's enough to make a reasonably good replacement decision.

4. Clock Algorithm

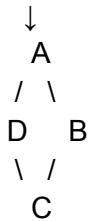
Now comes the clever part.

Imagine all pages arranged in a circle:



And there is a **clock hand** pointing at one page.

For example:



The clock hand is essentially a pointer telling the OS:

"Start looking for a victim from here."

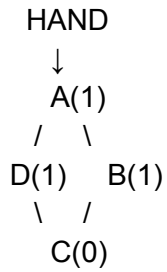
5. Suppose we need to evict a page

Imagine:

Page	Use bit

A	1
B	1
C	0
D	1

Clock hand points to A:



The OS checks A.

A has use bit = 1

That means:

A has been used recently.

So:

A: $1 \rightarrow 0$

And move the clock hand:

A $\rightarrow$ B

6. Check B

B has:

B = 1

So B was recently used.

Therefore:

B: $1 \rightarrow 0$

Move the hand:

B $\rightarrow$ C

7. Check C

C has:

$C = 0$

This means:

C has not been accessed since the OS last cleared its bit.

Therefore:

$C = \text{victim}$

Evict C.

8. Why does this approximate LRU?

Suppose:

$A = 1$

$B = 0$

$C = 1$

$D = 0$

The OS interprets:

$A \rightarrow \text{recently used}$

$B \rightarrow \text{not recently used}$

$C \rightarrow \text{recently used}$

$D \rightarrow \text{not recently used}$

So it prefers:

B or D

rather than:

A or C

That's approximately what LRU wants.

But notice the difference:

True LRU

Knows exact ordering:

A → most recent

C → second

D → third

B → oldest

Clock

Only knows:

A → used recently

C → used recently

B → not recently used

D → not recently used

So Clock doesn't know exactly which of B and D is older.

That's why it's **approximate LRU**, not exact LRU.

9. Why clearing the bits matters

This is extremely important.

Suppose we never cleared the bits.

Eventually:

A = 1

B = 1

C = 1

D = 1

Then every page looks recently used forever.

That would be useless.

So the OS periodically resets the history.

For example:

Initially:

A 0
B 0
C 0
D 0

Program accesses:

$A \rightarrow C \rightarrow A$

Hardware:

A 1
B 0
C 1
D 0

Then Clock comes around:

$A = 1 \rightarrow \text{clear} \rightarrow 0$
 $C = 1 \rightarrow \text{clear} \rightarrow 0$

Now the system starts a **new observation period**.

If the program subsequently accesses:

C

then:

A = 0
B = 0
C = 1
D = 0

Now C is clearly favored over the others.

So the use bit provides a kind of **recent-history window**.

10. Why is Clock better than scanning everything?

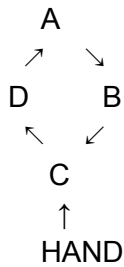
Imagine there are:

1 million pages

A naive algorithm might repeatedly scan all 1 million pages looking for an unused page.

Clock doesn't necessarily need to repeatedly scan the entire set.

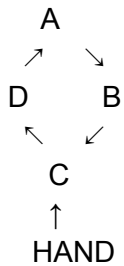
It keeps a pointer:



After examining a page, the hand moves forward.

So the OS remembers where it left off.

Next time:



It continues from there.

This avoids repeatedly starting from the beginning and rescanning everything.

11. Now the Dirty Bit

The next section adds another piece of information.

A page can be:

Clean

It hasn't been modified.

RAM:

Page A

Disk:

Page A

They contain the same data.

Therefore, if the OS wants to evict A:

delete A from RAM

That's enough.

It doesn't need to write anything to disk.

Dirty

Suppose the process modifies the page:

CPU writes to A

↓

RAM's A changes

↓

Disk's A is now outdated

For example:

Disk:

A = "Hello"

RAM:

A = "Hello World"

Now A is **dirty**.

If the OS wants to evict it, it must first:

RAM

↓

write modified page

↓

Disk

That requires disk I/O, which is expensive.

12. Hardware maintains the Dirty bit

Just like the use bit, hardware can maintain a **dirty/modified bit**.

Whenever the CPU writes to a page:

CPU writes Page A

↓

hardware

↓

A.dirty = 1

So now we have two pieces of information:

	Use	Dirty
Page A	1	1
Page B	0	0
Page C	1	0
Page D	0	1

13. Now Clock can make an even smarter decision

The OS would ideally prefer:

unused + clean

because that's the cheapest page to evict.

So we can think of the preference roughly as:

1. Unused + clean ← best
2. Unused + dirty
3. Recently used + clean
4. Recently used + dirty

The exact implementation can vary, but the general principle is:

Prefer pages that haven't been used recently and don't need to be written back.

14. Complete picture

Think of every page as having two important bits:

Page		
Use bit		← Has it been accessed?
Dirty bit		← Has it been modified?

For example:

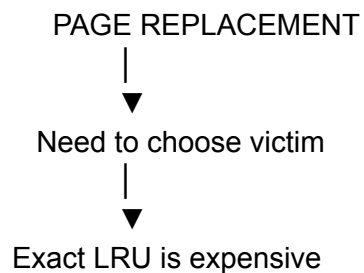
Page	Use	Dirty	

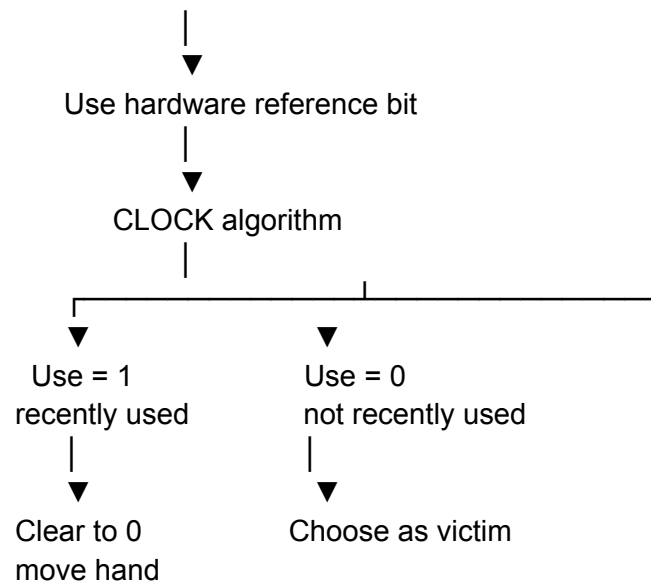
A	1	0	
B	0	0	← excellent victim
C	1	1	
D	0	1	← possible victim, but expensive

The Clock algorithm can use these bits to make a cheap replacement decision.

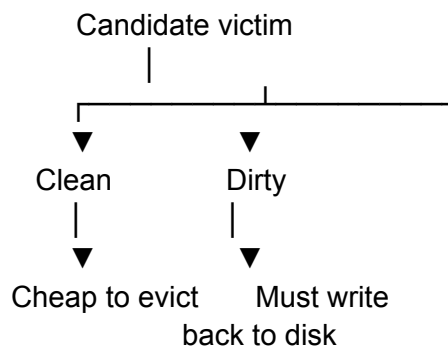
15. The biggest idea to remember

You can summarize the entire section as:





Then add the dirty bit:



In one sentence:

Clock approximates LRU by using a hardware reference/use bit to distinguish recently accessed pages from pages that haven't been accessed recently, while the dirty bit lets the OS prefer victims that can be evicted without expensive disk writes.

And one subtle point from OSTEP: **Clock is not "LRU with a circular list."** The circular structure is what makes the search efficient; the **use bit is what provides the recent-history information** that makes it approximate LRU.

Tuesday 7:35 PM

1.3. XDP(Express Data Path) When a packet arrives at the NIC, the NIC computes an RSS (Receive Side Scaling) hash-typically over a 2–5 tuple (commonly 5-tuple for UDP/TCP, depending on NIC configuration)-to select an RX queue. Note that in our implementation, we only have one RX queue for the NIC. Each RX queue is a hardware ring buffer of DMA descriptors, where each descriptor contains a pre-programmed DMA address pointing to a packet buffer. In zero-copy AF XDP mode, these DMA buffer addresses belong to a region of

user memory called UMEM, which has been pre-registered and pinned by the driver. The NIC writes the packet into the next available descriptor's associated DMA address using DMA. During this step, the CPU is not involved. In modern high-performance drivers, RX processing commonly occurs in NAPI poll mode or AF XDP busy-poll mode, where packet processing does not trigger an interrupt per packet. Once a batch of packets is ready, the driver invokes the XDP hook via a function pointer previously installed when the XDP program was attached (rather than dynamically checking for NULL). The XDP program receives packet boundaries through the xdp_md metadata struct, which provides offsets into the driver-owned buffer-backed by UMEM when zero-copy is enabled. In the DNS server case, if the XDP hook program (which we attach) identifies a packet destined for port 53, it calls: `bpf_redirect_map(&xsk_map, rx_queue_index, 0)`; The XSKMAP holds kernel-space references to AF XDP socket objects (`struct xdp_sock *`), indexed by NIC RX queue ID. The XDP program does not itself deliver the packet, copy memory, or interact with the socket directly. It only selects a target socket and returns XDP REDIRECT. After the XDP program returns, the kernel performs the redirect. It takes the same UMEM buffer address that the NIC used for DMA and writes it into the selected socket's RX ring. Importantly:

- The packet is not copied.
- Only the UMEM buffer address is enqueued in a lock free ring.
- The NIC plays no role after DMA completes. Userspace continuously polls the AF XDP socket. When a new buffer address appears in the socket's RX ring, userspace directly accesses and processes the packet from UMEM. In contrast, traditional kernel sockets do not support zero copy in the general case. Packets are first DMA-written into kernel memory, wrapped in multiple `sk_buff` structures as they traverse the network stack, and finally copied into userspace before being made available to the application. Thus, AF XDP in zero-copy mode eliminates:

1. Allocation and traversal of sk network stack layers, `buff` structures across
2. Per-packet system calls and kernel/userspace transitions,
3. And the final memory copy from kernel space to user space.

Instead, AF XDP delivers packets by publishing UMEM buffer addresses through a shared ring, enabling high throughput, low-latency packet processing for workloads such as DNS servers. Explain this in system

Yes. The easiest way to understand this section is to follow **one DNS packet from the NIC all the way to your userspace DNS resolver**.

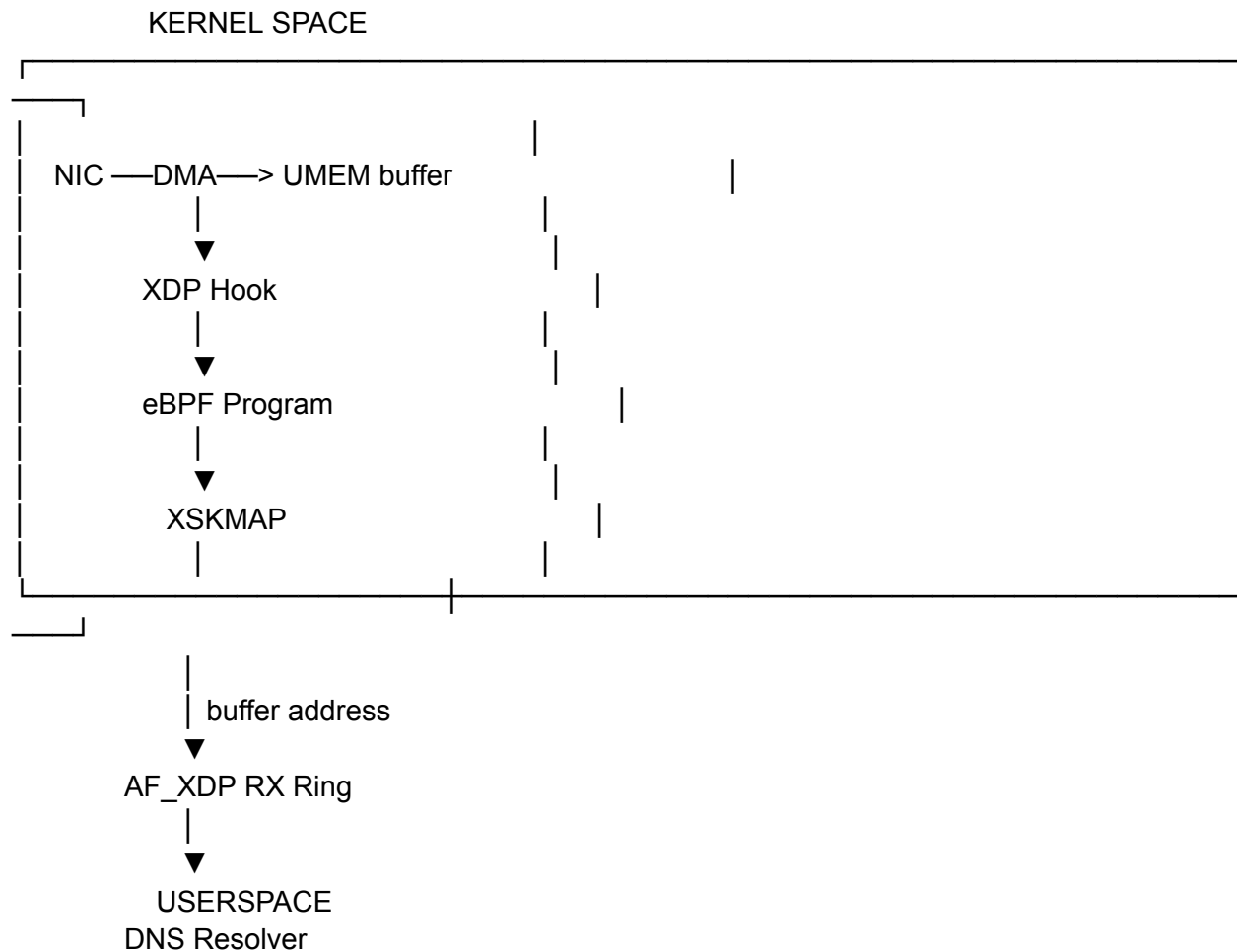
The key idea is:

NIC → DMA into UMEM → XDP hook → eBPF program → XSKMAP → AF_XDP RX ring → userspace reads the same buffer

There is **no packet copy** in the zero-copy AF_XDP path.

1. First understand the components

Before following the packet, identify the pieces involved:



There are four especially important things:

1. NIC

The network card receives Ethernet frames from the network.

2. UMEM

A region of memory that userspace has registered for AF_XDP.

In zero-copy mode, this memory is also usable by the NIC for DMA.

3. XDP/eBPF program

Runs very early in the packet-receive path.

Your program can inspect the packet and decide:

DNS packet? → redirect to AF_XDP

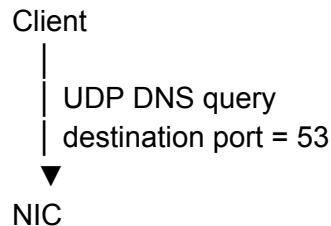
Not DNS? → normal processing / drop / etc.

4. AF_XDP socket

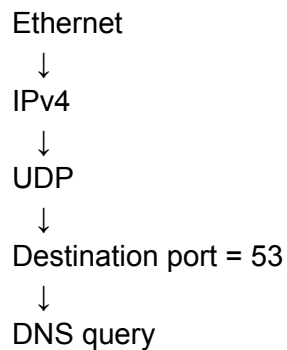
The socket provides rings through which the kernel and userspace communicate using **buffer addresses**, rather than copying packet contents.

2. Step 1 — Packet arrives at NIC

Suppose a DNS query arrives:



For example:

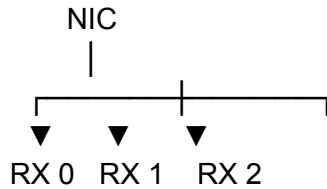


The NIC receives the packet.

3. Step 2 — NIC uses RSS to choose an RX queue

Modern NICs usually have multiple receive queues.

For example:



The NIC computes an **RSS hash** from packet fields.

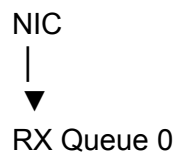
For TCP/UDP this commonly involves the 5-tuple:

source IP
destination IP
source port
destination port
protocol

Then:

RSS hash
↓
select RX queue

Your implementation has:



because you are using only one RX queue.

4. What exactly is an RX queue?

An RX queue has a hardware ring containing descriptors.

Conceptually:

RX Ring

Descriptor 0 —> buffer address A

Descriptor 1 —> buffer address B

Descriptor 2 —> buffer address C

Descriptor 3 —> buffer address D

A descriptor essentially tells the NIC:

"When you receive a packet, put it at this memory address."

5. Where do these buffers come from?

This is where **AF_XDP UMEM** becomes important.

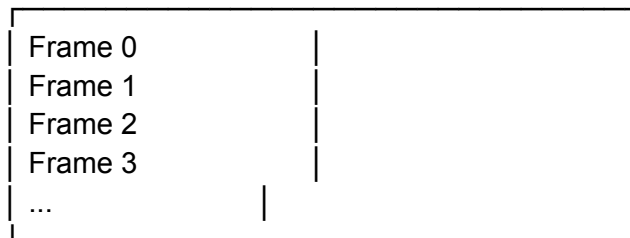
In zero-copy AF_XDP:

Userspace

|
| creates/registers UMEM



UMEM



The driver/NIC is given DMA addresses corresponding to these buffers.

So the RX descriptor might effectively say:

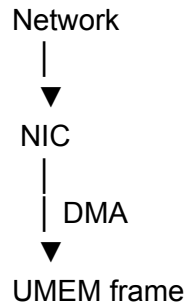
Descriptor 0

|
└──> DMA address corresponding to UMEM frame 0

6. Step 3 — NIC performs DMA

Now the packet arrives.

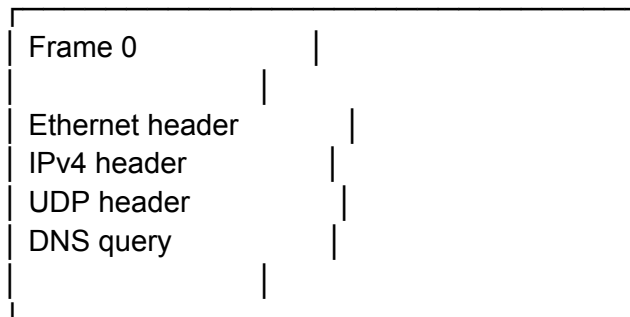
The NIC does:



The NIC writes the packet directly into the memory buffer.

For example:

UMEM



Important

The CPU doesn't have to execute instructions for every byte being received.

The NIC's DMA engine performs the memory write.

So:

NIC ———DMA————> Memory

rather than:

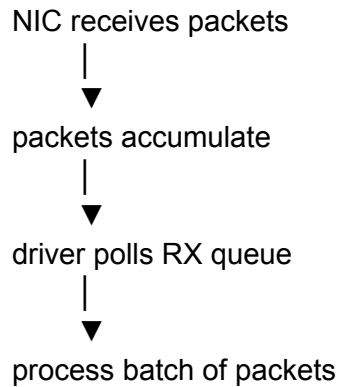
NIC → CPU → copy packet → memory

7. Step 4 — CPU/driver starts processing the packet

Eventually the driver processes the received packet.

Modern Linux networking commonly uses **NAPI polling** rather than generating an interrupt for every packet.

Conceptually:



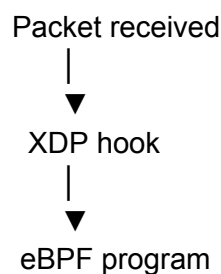
This is important for high performance because handling one interrupt per packet would be expensive.

8. Step 5 — XDP hook is invoked

Now we reach the most important part.

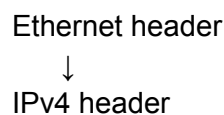
The driver invokes the **XDP hook**.

Think of XDP as an early decision point:



Your eBPF program gets information about the packet and can inspect its contents.

For example:

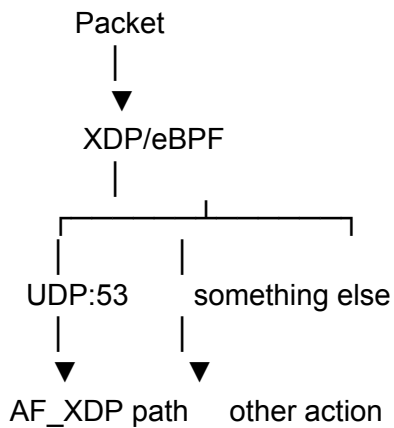


↓
UDP header
↓
destination port

It checks:

```
if (protocol == UDP && destination_port == 53)
```

Conceptually:



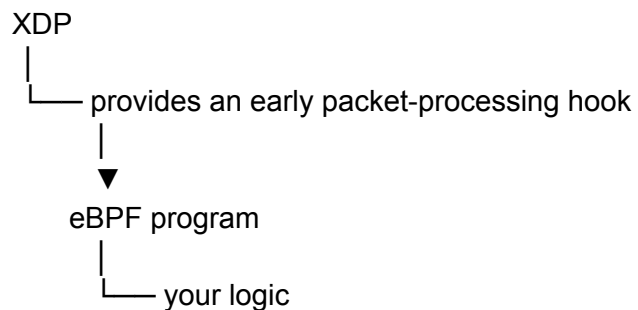
9. Is XDP itself the eBPF program?

This distinction is important.

XDP is the framework/hook point.

eBPF is the program you attach to that hook.

Think:



So when you say:

"XDP checks whether this is DNS"

more precisely:

The eBPF program attached to the XDP hook checks whether the packet is DNS traffic.

10. Step 6 — eBPF sees a DNS packet

Suppose the packet is:

Ethernet
↓
IPv4
↓
UDP
↓
destination port = 53
↓
DNS

Your eBPF program determines:

This is a DNS packet.

Now you want it to go to your AF_XDP socket.

You call:

```
bpf_redirect_map(&xsk_map, rx_queue_index, 0);
```

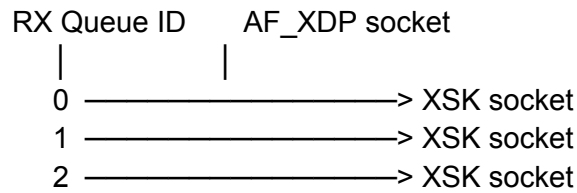
This is the critical line.

11. What is `xsk_map`?

`xsk_map` is an **XSKMAP**.

Conceptually:

XSKMAP



In your case:

RX queue = 0

XSKMAP[0]



your AF_XDP socket

So:

```
bpf_redirect_map(&xsk_map, 0, 0);
```

means roughly:

"Redirect this packet associated with RX queue 0 to the AF_XDP socket stored at XSKMAP[0]."

12. What does the eBPF program actually do?

This is another very important point.

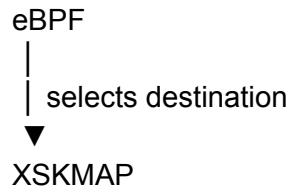
The eBPF program **does NOT**:

- copy packet
- send packet to userspace
- write packet into socket

Instead, it simply makes a decision:

"Send this packet to this AF_XDP socket."

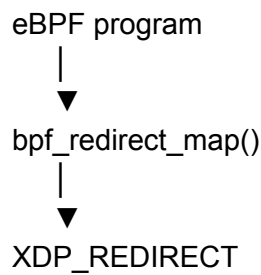
So:



Then the kernel performs the actual redirect.

13. Step 7 — XDP returns **XDP_REDIRECT**

Conceptually:



This tells the XDP subsystem:

"Redirect this packet to the selected AF_XDP socket."

14. Step 8 — Kernel performs the redirect

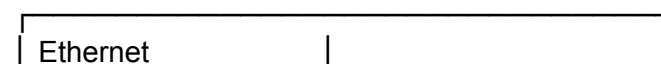
Now the kernel looks at the selected XSK.

Remember something extremely important:

The packet is **already sitting in UMEM**.

For example:

UMEM frame #42





The kernel does **not** take those bytes and copy them somewhere else.

Instead, it effectively says:

"Buffer #42 contains the packet."

and publishes that buffer address into the AF_XDP RX ring.

Conceptually:

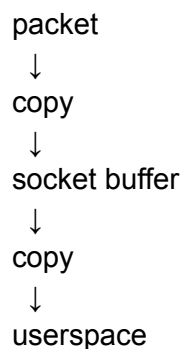
UMEM frame #42



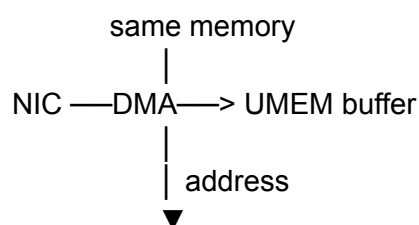
AF_XDP RX Ring

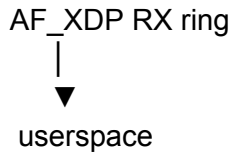
15. This is the heart of zero-copy

Traditional thinking might be:



AF_XDP zero-copy is closer to:





The ring contains something like:

buffer address = 0x1234000

not the entire packet.

16. What does "only the buffer address is enqueued" mean?

Suppose the DNS packet occupies:

150 bytes

The kernel doesn't put 150 bytes into the RX ring.

Instead it puts something conceptually like:

RX ring:

0x1234000	
0x1235000	
0x1236000	

Each address identifies a buffer in UMEM.

Userspace sees:

0x1234000

and knows:

"The packet is located at this UMEM address."

Then userspace accesses that memory directly.

17. Step 9 — Userspace polls the AF_XDP RX ring

Your DNS resolver is running in userspace.

It continuously checks the RX ring:

```
while (running) {  
    check AF_XDP RX ring;  
  
    if (packet available) {  
        get buffer address;  
        access packet from UMEM;  
        parse DNS;  
    }  
}
```

Suppose it receives:

buffer address = 0x1234000

It accesses:

UMEM + offset

and finds:

Ethernet
IPv4
UDP
DNS

Then your resolver parses the DNS request.

18. Notice what DIDN'T happen

In this entire path:

NIC
↓
DMA
↓
UMEM
↓
XDP
↓
eBPF
↓
XSKMAP
↓
RX ring
↓
userspace

there wasn't a packet copy from kernel memory into userspace.

The same underlying buffer is being used.

That's the major advantage of AF_XDP zero-copy.

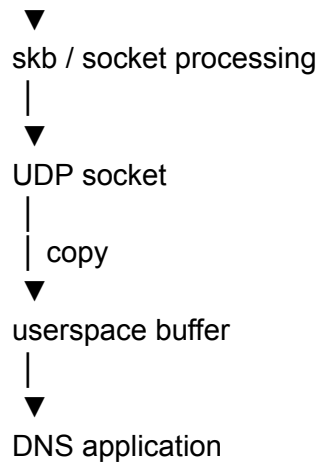
19. Compare this with normal UDP sockets

Now let's compare.

Traditional UDP

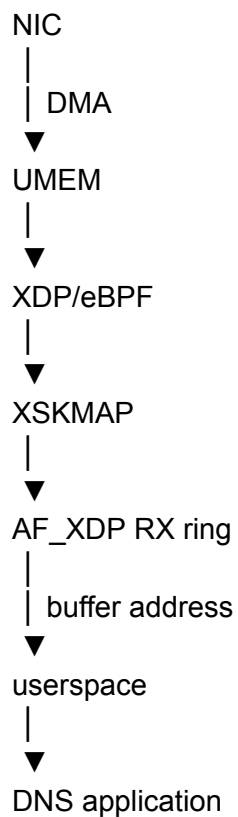
Conceptually:

NIC
|
| DMA
▼
kernel memory
|
▼
network stack
|



There are many kernel networking layers involved.

AF_XDP zero-copy



Much shorter.

20. What exactly does AF_XDP eliminate?

Your text lists three major things.

1. `sk_buff` overhead

Traditional Linux networking heavily uses `struct sk_buff` (`skb`) to represent packets while they travel through the network stack.

AF_XDP can bypass much of that processing for packets redirected to the XSK.

So you avoid a lot of:

packet
↓
skb
↓
network stack
↓
more processing

2. Repeated kernel/userspace transitions

A normal socket application interacts with the kernel through socket operations and associated system calls.

AF_XDP is designed around shared rings.

Userspace can poll the rings and process packets without doing a system call for every packet.

So instead of:

packet
↓
kernel
↓
syscall
↓
userspace

repeatedly, you can have:

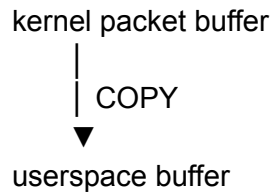
kernel —shared ring— userspace

with batching/polling.

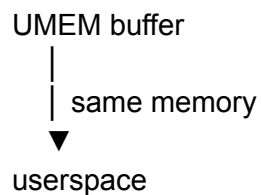
3. Final copy to userspace

This is the biggest zero-copy advantage.

Traditional:



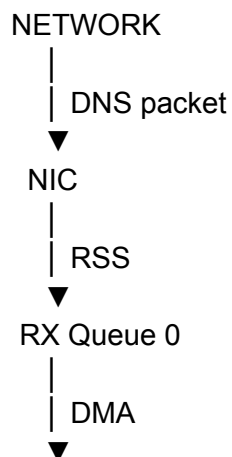
AF_XDP zero-copy:

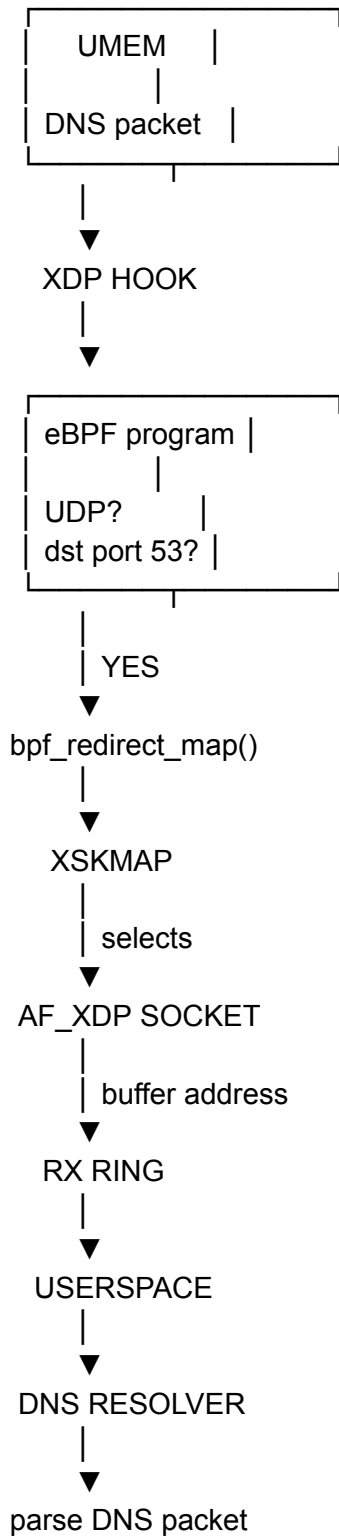


Userspace accesses the packet directly in the UMEM buffer.

21. The complete flow

Putting everything together:





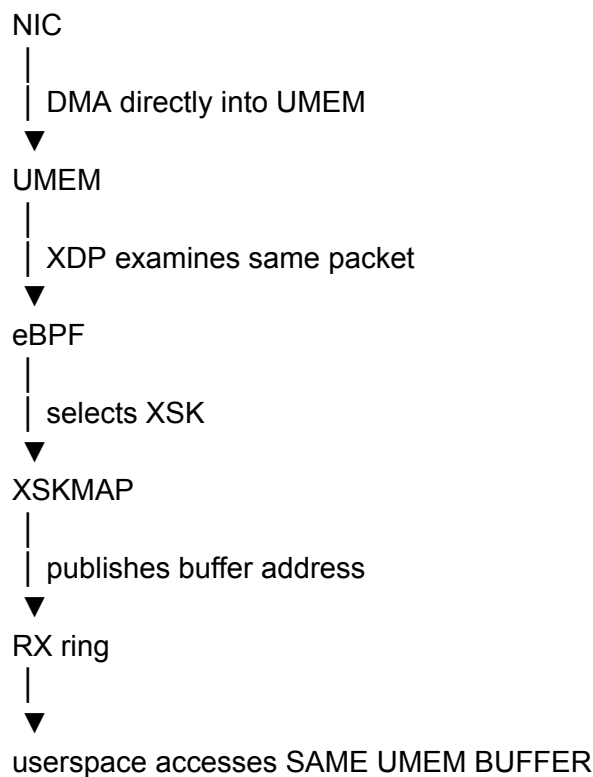
22. One subtle but very important correction

Don't think of the flow as:

"Packet enters kernel → XDP hook → eBPF copies it to userspace."

That's **not** what's happening.

Instead:

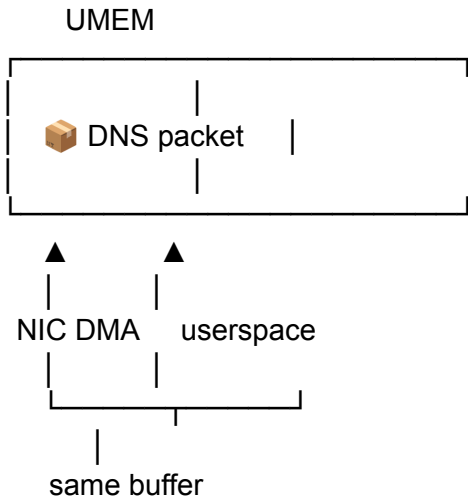


The packet bytes stay where they were placed by DMA.

Only **ownership/reference information—the buffer address—is passed through the ring.**

23. The simplest mental model

Think of UMEM as a shared warehouse.



The NIC says:

"I'll put the packet in box #42."

XDP says:

"Box #42 should go to this AF_XDP socket."

The RX ring says:

"Box #42 is ready."

Userspace says:

"Great, I'll open box #42 and process the packet."

Nobody copies the contents of box #42.

That's essentially what **AF_XDP zero-copy** is doing.

And this is why it is attractive for your DNS resolver: **DNS packets are small, numerous, and latency-sensitive, so avoiding `skb` processing, copies, and per-packet kernel/userspace overhead can significantly improve packet-processing efficiency.**

Monday 11:04 AM

When your C program calls `socket()`, this isn't just a regular function call — it's a request that: Traps the CPU into kernel mode. The kernel runs its own `socket()` implementation (allocates internal kernel data structures). Returns control back to your program in user mode, handing back a small integer (a file descriptor) that your program uses to refer to that thing later. This mode-switch has a cost. Switching from user mode → kernel mode → back to user mode takes real CPU cycles (saving registers, changing privilege state, etc.) — even before the kernel does

any actual work. This cost is called a syscall overhead, and it matters a lot later when we talk about why AF_XDP is faster. Explain

Absolutely. The key idea is: **socket()** looks like an ordinary C function, but the actual work is done by the Linux kernel.

Let's break the whole process down.

1. Your program is running in user mode

Suppose you write:

```
int sock = socket(AF_INET, SOCK_DGRAM, 0);
```

Your program normally runs in **user mode**.

User mode is intentionally restricted. Your program cannot directly:

- access kernel memory
- manipulate hardware directly
- create kernel networking structures
- change page tables
- control devices
- perform privileged CPU operations

This separation exists for **protection and stability**.

Think:

USER MODE

Your C program

```
int sock = socket(...);
```

```
printf(...)
```

```
malloc(...)
```

Your program is allowed to do normal computations, but it cannot simply say:

"Kernel, give me a UDP socket and modify your internal networking structures."

It needs to ask the kernel through a **system call**.

2. `socket()` is a system call

When you write:

```
socket(AF_INET, SOCK_DGRAM, 0);
```

it looks like an ordinary function call.

Conceptually, however:

Your program

```
|  
| socket()  
v
```

Operating System Kernel

The C library provides the interface that eventually invokes the Linux `socket` system call.

The important distinction is:

Normal function:

user code → user code

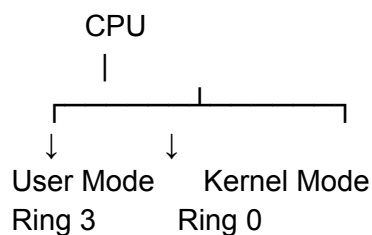
System call:

user code → KERNEL

3. How does the CPU enter kernel mode?

The CPU has privilege levels.

Simplifying to two:



Your application runs in **user mode**.

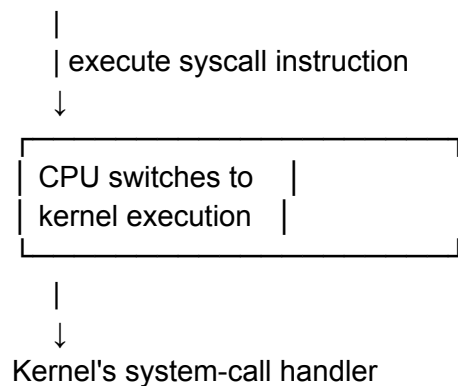
The kernel runs in **kernel mode**.

A system call uses a special CPU mechanism to transition between these privilege levels.

On modern x86-64 Linux, this is typically the `syscall` instruction.

Conceptually:

Your program



This is the **trap/entry into the kernel**.

4. What does the kernel do?

Now you're executing kernel code.

The kernel examines the system call number and arguments.

For example:

```
socket(AF_INET, SOCK_DGRAM, 0);
```

means roughly:

`AF_INET` → IPv4

`SOCK_DGRAM` → UDP

protocol 0 → choose appropriate default protocol

The kernel then creates the necessary internal networking objects.

Conceptually:

Kernel

struct socket

|

↓

struct sock

|

↓

networking state

|

↓

protocol-specific structures

There are many implementation details here, but the important point is:

The actual socket is a kernel-managed object.

Your C program doesn't receive a pointer to this kernel object.

Instead, it receives a **file descriptor**.

5. Why does it return a small integer?

Suppose the kernel creates the socket.

It might return:

3

So:

```
int sock = socket(...);
```

results in:

```
sock = 3
```

You can then do:

```
send(sock, ...);
```

```
recv(sock, ...);
```

```
close(sock);
```

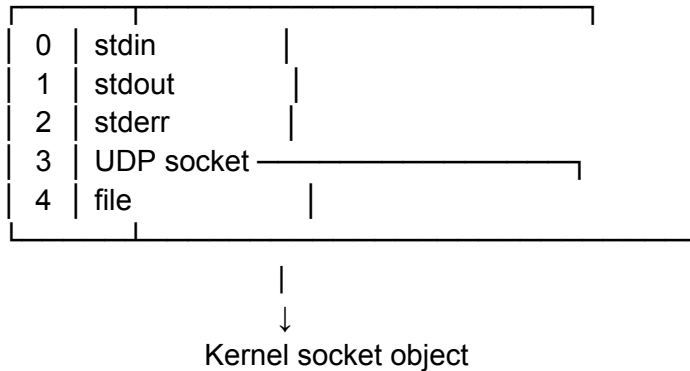
What is 3?

It is an index into your process's **file descriptor table**.

Conceptually:

Your process

File Descriptor Table



So your program doesn't directly manipulate the socket object.

It says:

"Kernel, perform this operation on object associated with descriptor 3."

6. Then the kernel returns to user mode

After creating the socket, the kernel needs to give the result back to your program.

Conceptually:

USER MODE

|
| socket()

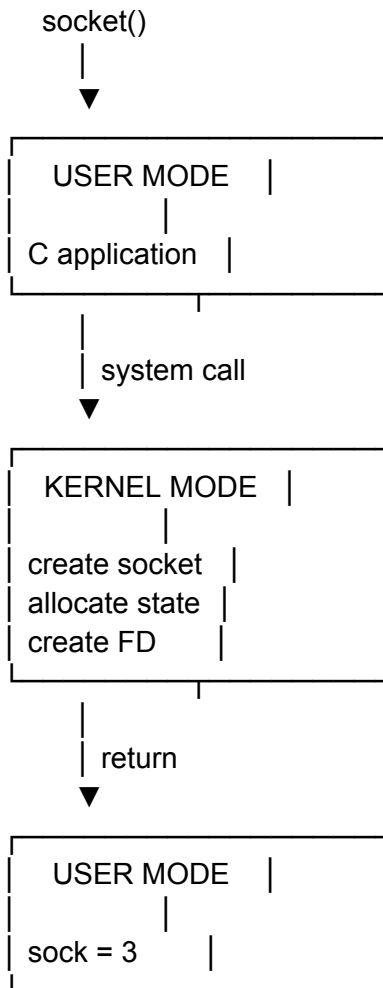


KERNEL MODE

|
| create socket
| allocate structures
| create FD
|

| return 3
↓
USER MODE
↓
sock == 3

So the entire operation looks like:



7. Why is this expensive?

There are actually **two different costs** here.

Cost A — entering/leaving the kernel

The CPU has to perform the privilege transition.

There is bookkeeping involved, such as:

- switching execution context appropriately
- saving/restoring relevant CPU state
- changing privilege level
- entering kernel-controlled code
- validating the system-call request
- returning safely to user mode

This is called **system-call overhead**.

Cost B — doing the actual work

After entering the kernel, the kernel still has to actually create the socket.

For example:

System-call overhead
+
Kernel socket creation work
=
Total socket() cost

So the mode switch isn't the entire cost.

8. Why does this matter for AF_XDP?

This becomes much more interesting when dealing with **packets**.

Consider a traditional UDP server.

Your application does:

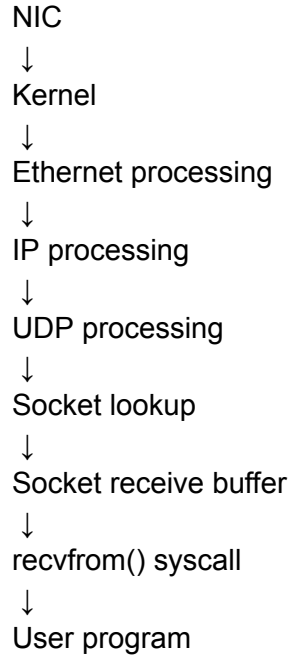
```
recvfrom(sock, buffer, ...);
```

The application wants:

"Give me the next UDP packet."

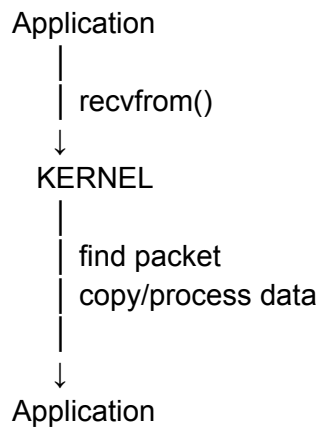
But the packet is ultimately managed through kernel networking.

A simplified path might look like:



Every time your application needs to interact with the kernel through a system call, there can be user↔kernel transition overhead.

For example:



For **one packet**, this overhead may be perfectly acceptable.

But imagine:

1 packet
10 packets
100 packets
1,000,000 packets

If you're processing enormous numbers of packets, small per-operation costs become significant.

9. This is where AF_XDP becomes interesting

AF_XDP tries to reduce the amount of work and copying involved in the traditional path.

Instead of:

NIC
↓
Kernel networking stack
↓
UDP socket
↓
syscall
↓
User program

you can have an XDP program redirect packets toward an **AF_XDP socket** and use shared memory/rings between kernel-side networking infrastructure and user space.

Very simplified:

Traditional UDP

NIC
↓
Kernel network stack
↓
UDP socket
↓
syscall
↓
User

versus:

AF_XDP

NIC
↓
XDP
↓
AF_XDP
↓
shared UMEM / rings
↓
User

The important idea isn't:

"AF_XDP has zero system calls."

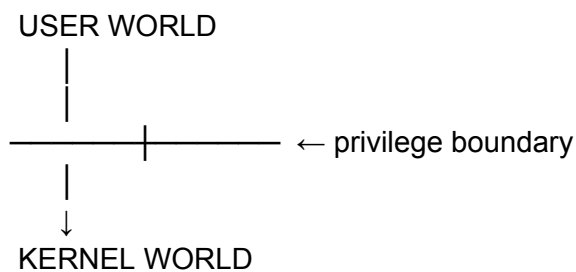
That's **not** correct.

Rather, AF_XDP is designed so that **the packet data path can avoid a system call for every packet**, using memory-mapped rings and user-space polling.

That distinction is very important.

10. The big picture

Think of a system call as crossing a border:



Every crossing has some overhead.

For something like:

socket()

you don't care much because you typically call it **once during setup**.

Program starts

↓
socket() ← once

bind() ← once
setup()
↓
Process millions of packets

But if your architecture requires frequent kernel interaction during the packet-processing loop, the overhead becomes much more important.

That's the fundamental performance motivation behind techniques such as **AF_XDP**:

Do expensive kernel setup when necessary, but make the high-frequency packet data path as efficient as possible.

And one subtle correction to your original statement: **the cost isn't simply "saving all registers and changing privilege state."** Modern CPUs have optimized system-call entry/exit mechanisms, and the exact overhead depends on CPU, kernel, workload, cache state, etc. The useful mental model is simply **"crossing the user/kernel boundary has non-zero overhead."**

When your program runs, the kernel gives it its own private slice of memory called an **address space**. Your program can only see and touch memory inside this slice — it cannot see another program's memory, and it cannot see the kernel's own internal memory,

what is kernels own internal memory mean"?

Think of **kernel's own internal memory** as memory that Linux uses to run and manage the entire computer.

Your program has its own memory, while the kernel has memory that it uses to keep track of things your program is not allowed to directly manipulate.

Example

Suppose you run:

```
int x = 10;  
char buffer[100];
```

These belong to **your program's address space**:

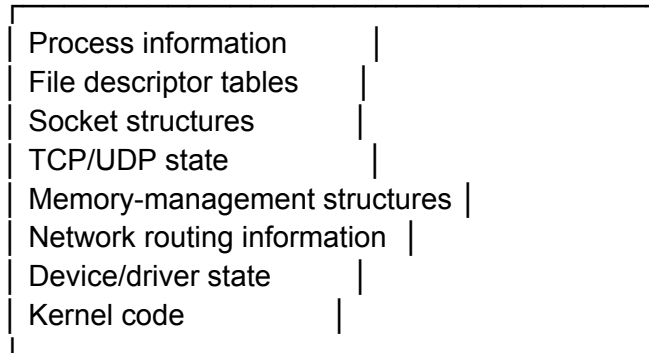
Your program's memory

Your code	
Your variables	
x = 10	
buffer[100]	
Your heap	



But Linux also needs memory to maintain its own data structures:

Kernel's memory



That is what **kernel internal memory** means.

Your `socket()` example makes this very clear

When you do:

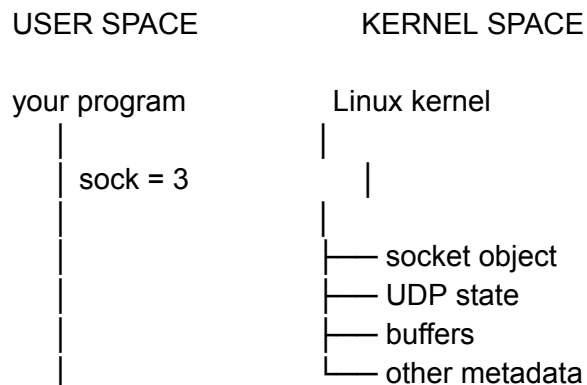
```
int sock = socket(AF_INET, SOCK_DGRAM, 0);
```

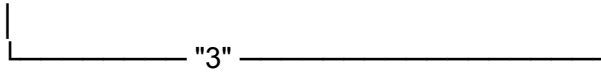
your program gets:

```
sock = 3
```

But somewhere in kernel memory, Linux has created/maintained objects representing that socket.

Conceptually:





Your program **doesn't get direct access to those kernel objects**.

The 3 is just a **file descriptor** that acts like a handle/reference.

When you later do:

```
send(sock, data, ...);
```

the kernel essentially says:

"This process gave me file descriptor 3. Let me look up what kernel object descriptor 3 refers to."

Then it performs the operation on the kernel-managed socket.

Why can't your program access kernel memory?

Because otherwise any ordinary program could do things like:

Program A



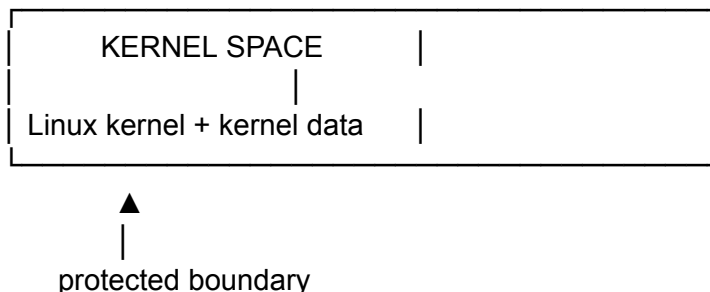
modify kernel memory

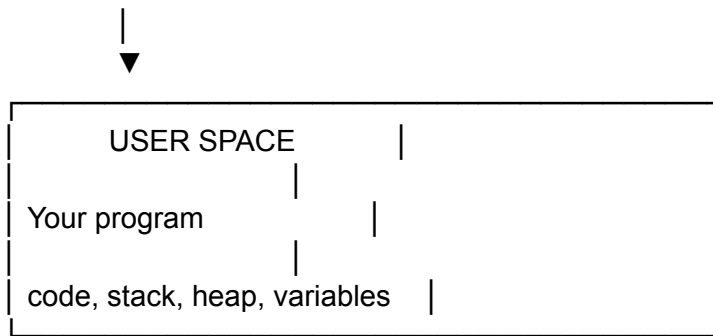


change Program B's memory
change network configuration
modify another process's files
crash the OS
etc.

That would be catastrophic.

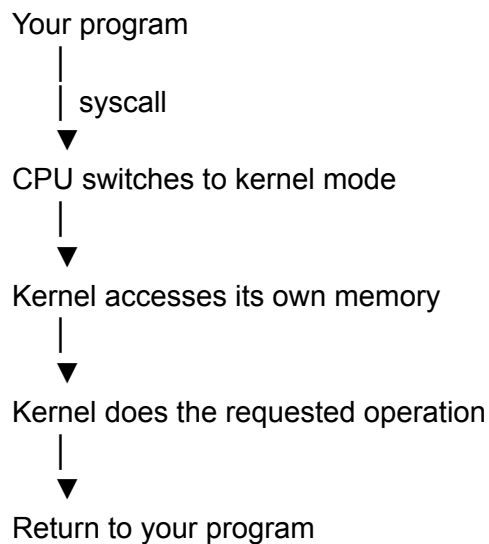
So hardware + OS memory protection enforce roughly:





When your program needs something from the kernel, it **doesn't directly enter kernel memory**.

Instead:



So when you hear **"kernel memory"**, think:

Memory reserved/protected for the operating system itself—its code, data structures, networking state, process information, device drivers, etc.

And **"user memory"** means:

Memory belonging to ordinary applications.

This separation is one of the fundamental ideas behind **process isolation and protection** in operating systems.

a

Sunday 10:45 PM

PATH 1: Normal UDP Socket Resolver (Baseline)

Step 0 — Setup (happens once, before any query)

The resolver program does:

c

```
int sock = socket(AF_INET, SOCK_DGRAM, 0);  
bind(sock, {0.0.0.0, port 53}, ...);
```

This tells the **kernel**: "any UDP packet arriving with destination port 53 belongs to *this* socket's receive queue." The kernel now maintains an internal table (a socket lookup table, keyed by protocol+port+address) so it knows where to deliver future port-53 packets.

Step 1 — Client sends the query (browser side, not our focus, but for completeness)

Browser → OS stub resolver → `sendto()` → kernel builds UDP header (src port = random ephemeral, e.g. 51041; dst port = 53) → kernel builds IPv4 header (src = client IP, dst = resolver IP) → kernel builds Ethernet header (dst MAC = default gateway) → NIC transmits bits onto the wire.

Step 2 — Packet arrives at the resolver's NIC (hardware)

The NIC receives the raw electrical/optical signal, reconstructs it into a frame, and — if using standard networking — DMAs (copies) the raw bytes into a kernel-managed **ring buffer in RAM** (this is the NIC driver's RX ring, separate from AF_XDP's ring). This triggers a hardware interrupt (or the driver is polling, via NAPI).

Step 3 — Kernel driver layer

The `sk_buff` (socket buffer) is allocated by the kernel to represent this packet. The raw bytes are attached to it. This is **copy #1**: bytes moved from NIC DMA buffer into kernel packet buffer.

Step 4 — Ethernet (Layer 2) processing

Kernel's `netif_receive_skb()` path parses the Ethernet header:

- Checks destination MAC matches this NIC (or is broadcast/multicast).
- Reads EtherType field (0x0800 = IPv4).
- Strips Ethernet header, hands the rest up to the IP layer.

Step 5 — IPv4 (Layer 3) processing

Kernel's IP receive function:

- Validates IP header checksum.
- Checks destination IP matches a local address (or needs forwarding — not our case).
- Reads Protocol field (17 = UDP).
- Reassembles fragments if needed (DNS queries usually aren't fragmented).
- Strips IP header, hands payload to UDP layer.

Step 6 — UDP (Layer 4) processing

Kernel's UDP receive function:

- Validates UDP checksum (optional but typically present).
- Reads destination port (53).
- **Looks up the socket table** built in Step 0 to find which socket owns port 53.
- Copies the UDP payload (the raw DNS message bytes) into that socket's **receive queue** (a buffer associated with the socket, not the app yet). This is **copy #2**.
- Wakes up any process blocked on `recvfrom()`.

Step 7 — Application reads it

Your resolver program was blocked here:

c

```
recvfrom(sock, buf, sizeof(buf), 0, &client_addr, &addr_len);
```

The kernel copies bytes from the socket's receive queue into your userspace buffer `buf`. This is **copy #3**. Now, finally, your application-layer code has:

- The raw DNS message bytes (already stripped of Ethernet/IP/UDP — the kernel did all that parsing for you).
- `client_addr` filled in automatically by the kernel (client's IP and source port), extracted from the headers it already parsed.

Step 8 — DNS logic (your code)

You parse the DNS header + question section (transaction ID, QNAME, QTYPE), look up or forward, build a DNS response message (just the DNS layer — question echoed back + answer records).

Step 9 — Sending the reply

c

```
sendto(sock, response_buf, response_len, 0, &client_addr, addr_len);
```

You only supply the **payload** and destination address. The kernel does everything else:

- **UDP layer:** builds UDP header (src port = 53, dst port = client's original source port, computes checksum).
- **IP layer:** builds IPv4 header (src = resolver IP, dst = client IP, TTL, checksum), consults the routing table to decide the next hop.
- **ARP/neighbor lookup:** kernel checks its ARP cache for the next hop's MAC address (may trigger an ARP request if not cached).
- **Ethernet layer:** builds Ethernet header (dst MAC = next hop's MAC, src MAC = resolver's NIC MAC).
- Hands the fully-formed frame to the NIC driver, which DMA's it out to the wire.

Full chain, non-XDP:

Client sendto()

→ kernel UDP hdr → kernel IP hdr → kernel Eth hdr → NIC → wire
 → Resolver NIC → DMA copy → sk_buff
 → kernel parses Eth → strips
 → kernel parses IP → strips
 → kernel parses UDP, socket lookup → copies to socket recv queue
 → app recvfrom() → copies to app buffer [app sees ONLY DNS bytes]
 → app builds DNS response
 → app sendto() with payload only
 → kernel builds UDP hdr → kernel builds IP hdr → ARP lookup → kernel builds Eth hdr
 → NIC → wire → client

Note: **three data copies** on receive path alone (DMA→skb, skb→socket queue, socket queue→userspace), and the application never sees or touches Ethernet/IP/UDP headers — the kernel abstracts all of it away via the socket API.

Explain this as I am not aware of the OS things, so explain this simply and explain that

Absolutely. The easiest way to understand this is to **forget DNS for a moment** and first understand what the **OS/kernel, NIC, socket, and application** are doing.

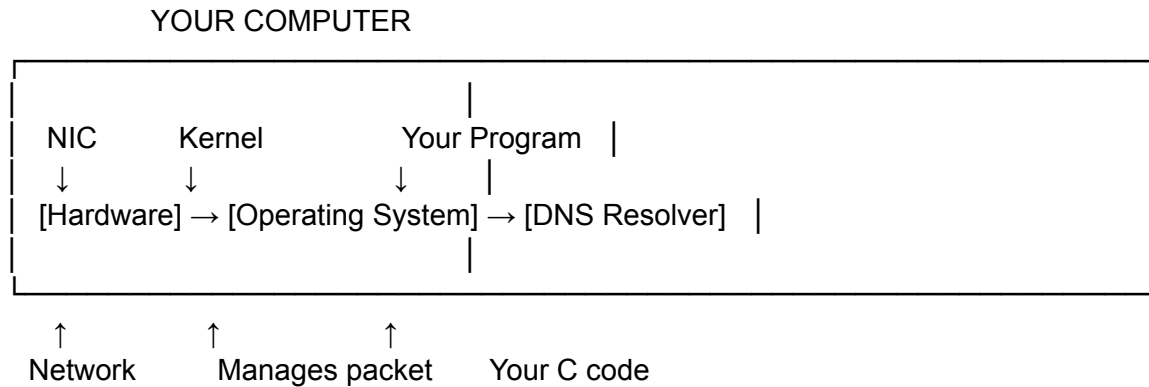
Your code is essentially saying:

“I want the OS to receive UDP packets on port 53 and give their data to my program.”

Let's walk through the packet from the network cable all the way into your C program.

1. First understand the 4 important components

Imagine a packet arriving at your computer:

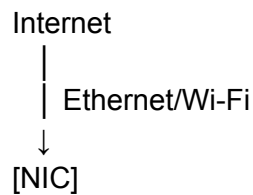


NIC

NIC = Network Interface Card

It is the hardware that connects your computer to the network.

For example:



The NIC receives the raw network frame.

Kernel

The **kernel is the core part of the operating system.**

It controls things such as:

- Network
- CPU
- Memory
- Files
- Processes
- Hardware

Your application normally **doesn't directly talk to the NIC.**

Instead:

Your program



Kernel



NIC



Network

The kernel acts as the middleman.

Your application

This is your DNS resolver program.

For example:

```
recvfrom(sock, buf, ...);
```

Your program says:

"Kernel, give me the next UDP packet that belongs to my socket."

Socket

A socket is basically an OS-managed communication endpoint.

Think of it like a mailbox.

Your program creates a socket:

```
int sock = socket(AF_INET, SOCK_DGRAM, 0);
```

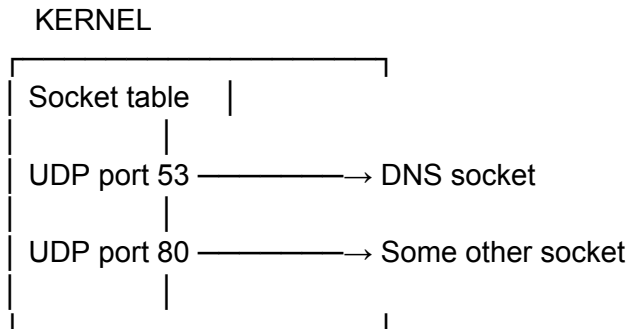
Then:

```
bind(sock, {0.0.0.0, 53}, ...);
```

means:

"Kernel, associate this socket with UDP port 53."

So conceptually:



Now when a UDP packet arrives with destination port **53**, the kernel knows:

"This packet belongs to the DNS resolver."

2. Before any DNS query: `socket()` and `bind()`

Your program does:

```
int sock = socket(AF_INET, SOCK_DGRAM, 0);
```

Let's break that down.

`AF_INET`

Means:

"I want IPv4 networking."

`SOCK_DGRAM`

Means:

"I want a UDP socket."

So:

```
socket(AF_INET, SOCK_DGRAM, 0);
```

basically means:

Create an IPv4 UDP communication endpoint for me.

Then:

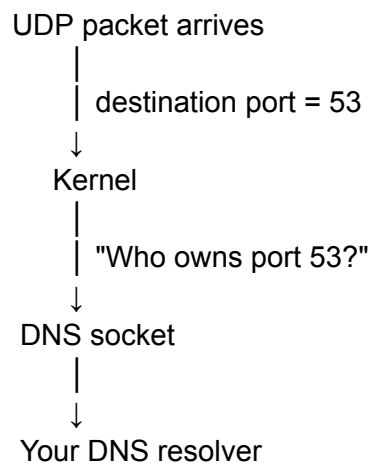
```
bind(sock, {0.0.0.0, 53}, ...);
```

means:

Associate this socket with UDP port 53.

Port 53 is the standard DNS port.

So after `bind()`:



3. Now a client sends a DNS query

Suppose your browser wants:

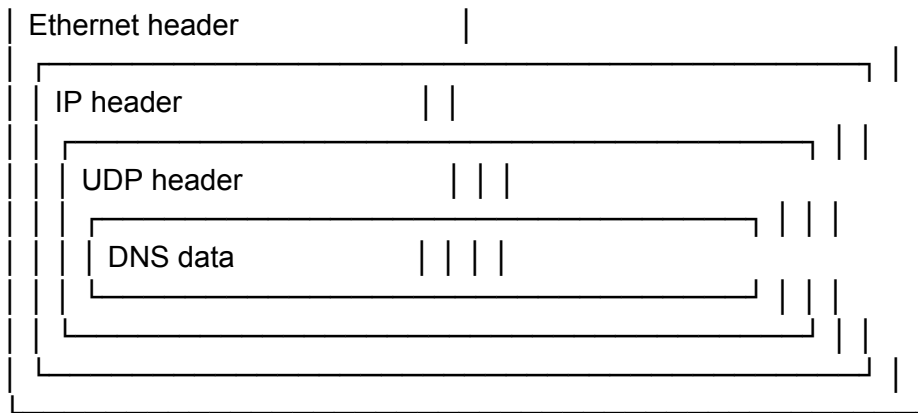
`www.google.com`

It sends a DNS query.

The important thing is that the DNS query doesn't travel alone.

The data gets wrapped in multiple headers.

Think of it like putting a letter inside several envelopes.



This is called **encapsulation**.

The DNS application creates DNS data.

UDP adds a UDP header.

IP adds an IP header.

Ethernet adds an Ethernet header.

4. What does the packet actually look like?

Let's make up an example.

The client might create:

DNS:

"What is the IP address of google.com?"

UDP adds:

Source port = 51041

Destination port = 53

IP adds:

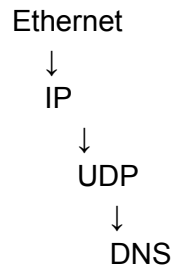
Source IP = 192.168.1.20

Destination IP = 8.8.8.8

Ethernet adds:

Source MAC = client's MAC
Destination MAC = router's MAC

So conceptually:



5. The packet reaches the resolver's NIC

Eventually the packet reaches the machine running your DNS resolver.

First thing that sees it?

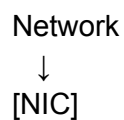
NIC

Not your C program.

Not even the kernel directly.

The NIC receives the electrical/optical/radio signals and reconstructs them into bytes.

Conceptually:



The NIC now has something like:

Ethernet header
IP header
UDP header
DNS data

6. What is DMA?

This sounds complicated but the idea is simple.

DMA = Direct Memory Access.

It allows hardware like the NIC to put received data directly into RAM without the CPU having to manually copy every byte.

Think:



The NIC places the received packet into a memory area prepared by the kernel/driver.

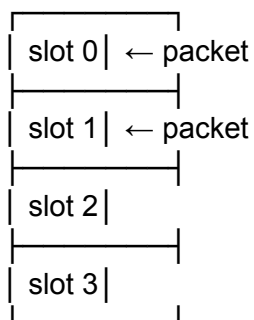
This is called an **RX ring**.

You can think of the RX ring as:

A queue of memory slots where the NIC can put incoming packets.

Something like:

RX Ring



7. Then the kernel gets involved

The NIC tells the system:

"Hey! I received a packet."

Historically this might involve an interrupt.

Modern Linux commonly uses **NAPI**, which can switch to polling under load.

You don't need to worry about the details yet.

The important concept is:

NIC received packet



Kernel network subsystem processes it

8. What is **sk_buff**?

This is an important Linux networking concept.

Linux needs some internal structure to represent a network packet.

Linux uses something called:

struct sk_buff

Usually called:

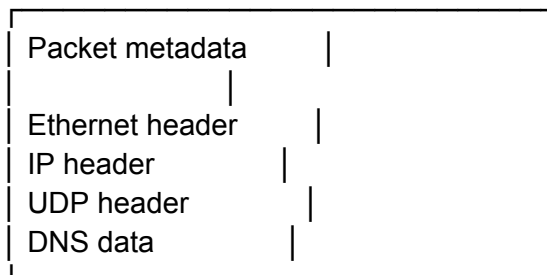
skb

Think of an **skb** as:

A kernel object that represents a network packet and contains information about that packet.

Conceptually:

skb



Your application does **not** see the `skb`.

It's an internal kernel data structure.

9. Now the kernel examines the Ethernet header

The kernel first looks at Layer 2.

Ethernet

It asks:

"Who is this frame intended for?"

For example:

Destination MAC = AA:BB:CC:DD:EE:FF

The NIC's MAC might be:

AA:BB:CC:DD:EE:FF

So:

"Yes, this frame is for me."

Then it sees:

EtherType = 0x0800

which means:

This contains IPv4.

So the kernel passes the packet to the IPv4 code.

10. Kernel examines the IP header

Now Linux looks at:

IP header

For example:

Source IP = 192.168.1.20

Destination IP = 10.0.0.5

Protocol = 17

The important field is:

Protocol = 17

Protocol 17 means:

UDP

So Linux says:

"Okay, this is an IPv4 packet carrying UDP."

It then passes it to the UDP layer.

11. Kernel examines UDP

Now Linux looks at:

UDP header

For example:

Source port = 51041

Destination port = 53

The important part is:

Destination port = 53

Remember what you did earlier?

```
bind(sock, {0.0.0.0, 53}, ...);
```

So the kernel has something conceptually like:

KERNEL SOCKET TABLE

Protocol	Port	Owner
UDP	53	DNS resolver
UDP	5000	Program A
TCP	80	Web server
TCP	22	SSH server

The kernel sees:

UDP
destination port = 53

and says:

"I know who owns this."

It finds your DNS socket.

12. The really important concept: socket receive queue

The kernel now has the DNS data.

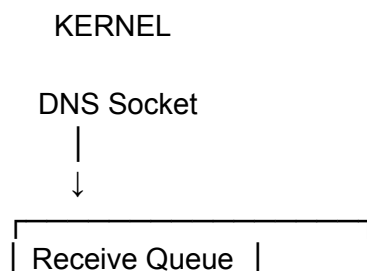
But your program isn't necessarily ready to execute right now.

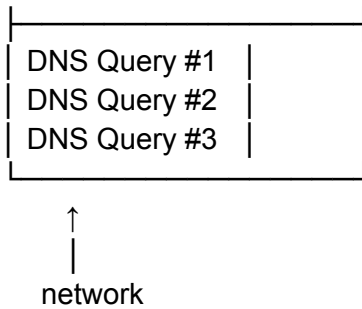
Your program might be sleeping inside:

recvfrom()

So the kernel puts the received data into the socket's **receive queue**.

Think of this like a mailbox:





The receive queue belongs to the socket.

13. Why does the kernel need a receive queue?

Because the network and application operate independently.

Imagine:

Packet arrives at 10:00:00

but your program is busy doing something else.

The kernel can't simply throw away the packet.

So:

Network



Kernel



Socket receive queue



Application later reads it

This is one of the major jobs of the OS.

14. Your program calls `recvfrom()`

Your program has:

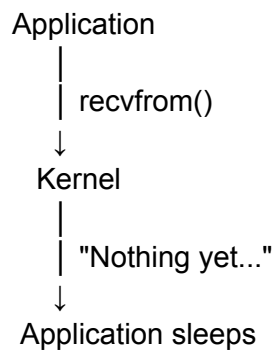
```
recvfrom(  
    sock,  
    buf,  
    sizeof(buf),  
    0,  
    &client_addr,  
    &addr_len  
);
```

This means:

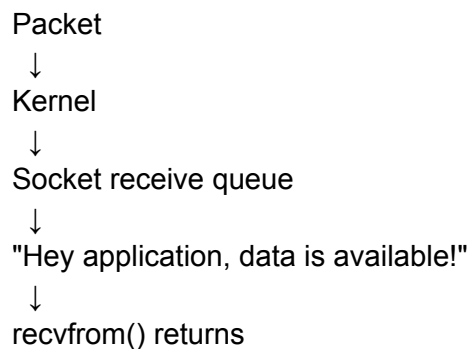
"Kernel, give me the next data available on this socket."

If there is no packet available, your program can sleep/block.

Conceptually:



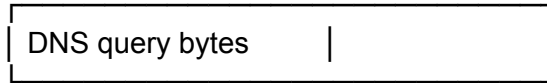
When a packet arrives:



15. Where does **buf** come in?

Suppose the kernel has:

Socket receive queue



Your application has:

```
char buf[512];
```

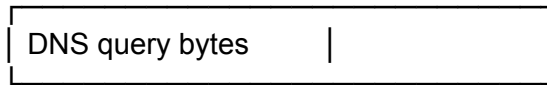
When you call:

```
recvfrom(sock, buf, sizeof(buf), ...);
```

the kernel copies the DNS data into:

Application memory

buf



Now your C program can access it.

16. This is why the application sees **ONLY DNS**

This is one of the most important points in your project.

Your application receives:

DNS data

It does **NOT** receive:

Ethernet header

IP header

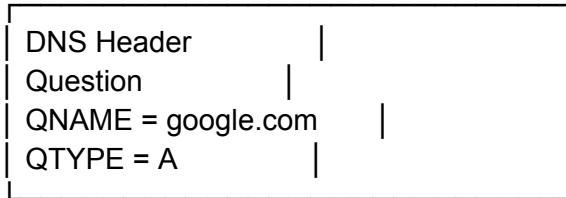
UDP header

Why?

Because the kernel already processed those.

Your application sees:

buf



It doesn't need to do:

```
parse_ethernet();
parse_ip();
parse_udp();
```

The OS did that already.

17. So what are the "3 copies"?

This part of your description is important for understanding why AF_XDP exists.

Conceptually, on the normal socket path, the packet's data may pass through several kernel/application buffers.

Copy 1

From the NIC's DMA buffer into a kernel packet representation/buffer:

NIC DMA memory



kernel packet buffer / skb

Copy 2

The UDP payload gets placed into the socket's receive-buffer/queue:

kernel packet data
↓
socket receive buffer

Copy 3

`recvfrom()` copies data from kernel memory into your application's memory:

socket receive buffer
↓
userspace buf[]

So conceptually:

NIC
↓
Kernel packet buffer
↓
Socket receive buffer
↓
Application buffer

The exact implementation details can vary with kernel/NIC/driver optimizations, so don't memorize this as "Linux always physically performs exactly three copies." The useful idea is:

The normal socket path involves kernel-managed packet processing and a userspace copy, whereas AF_XDP can give userspace much more direct access to packet buffers.

That distinction is what matters for your project.

18. Now your DNS resolver finally does its job

Your application now has:

DNS query

For example:

Transaction ID = 1234
QNAME = www.google.com
QTYPE = A

Your code parses it.

Then it decides:

"What IP address does www.google.com have?"

It may:

- answer from cache, or
- forward the query to another DNS server.

Then it constructs:

DNS response

For example:

Transaction ID = 1234

Question:
www.google.com

Answer:
142.250.x.x

19. Now your program sends the response

Your code does:

```
sendto(  
    sock,  
    response_buf,  
    response_len,  
    0,  
    &client_addr,  
    addr_len  
);
```

Notice something very interesting.

You give the kernel:

DNS response

and:

destination IP

destination port

You don't manually construct Ethernet/IP/UDP headers.

20. Kernel constructs the UDP header

Your application gives:

DNS response

The kernel adds:

UDP header

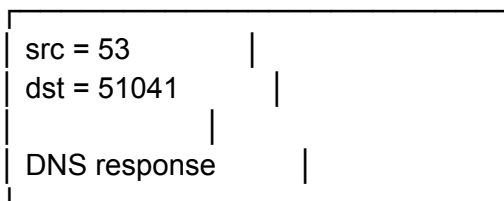
Source port = 53

Destination port = 51041

because the client originally used port 51041.

Conceptually:

UDP



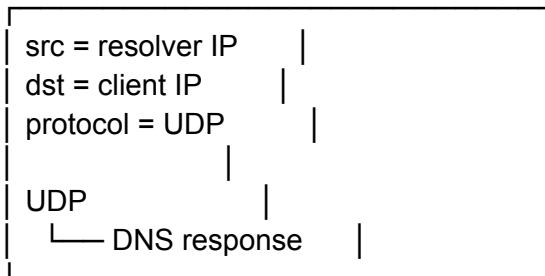
21. Kernel constructs IP header

Then the IP layer adds:

Source IP = resolver IP
Destination IP = client IP
Protocol = UDP
TTL = ...

Now:

IP



22. Kernel determines where to send it

Suppose the client is not on the same local network.

The kernel checks its **routing table**.

Something conceptually like:

Destination	Next hop

192.168.1.0/24	directly connected
0.0.0.0/0	192.168.1.1

So the kernel decides:

"To reach this destination, send the packet to my gateway 192.168.1.1."

23. How does Ethernet know where to send it?

Ethernet doesn't use IP addresses.

Ethernet uses:

MAC addresses

So the kernel needs to know:

"What MAC address corresponds to my next hop?"

For IPv4, it can use **ARP**.

For example:

IP:

192.168.1.1

MAC:

AA:BB:CC:DD:EE:FF

The kernel may already have this in its ARP/neighbor cache.

If not, it may need to perform neighbor discovery/ARP.

You don't need to worry about this deeply yet.

24. Kernel creates Ethernet frame

Now the kernel has:

Ethernet

↓

IP

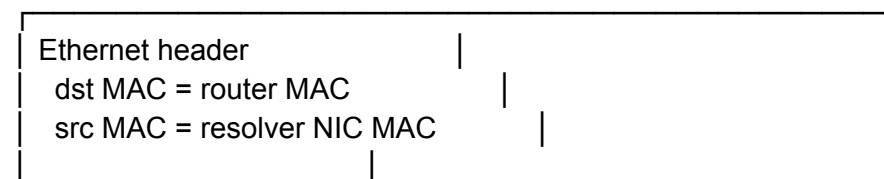
↓

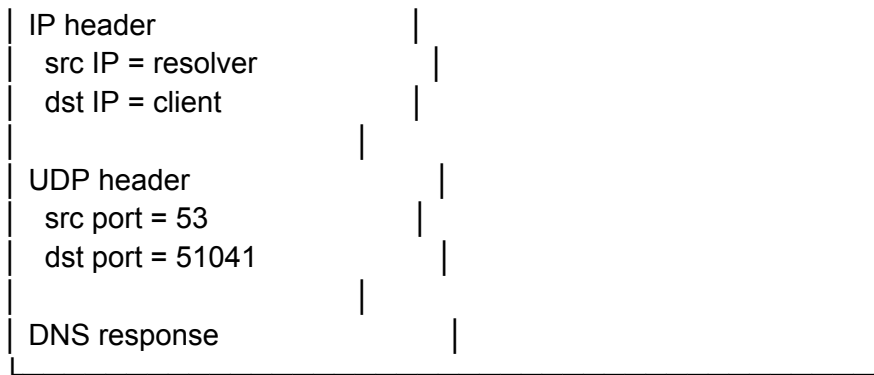
UDP

↓

DNS response

Something like:

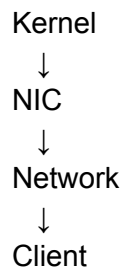




Then the kernel gives the frame to the NIC.

25. NIC sends it onto the network

Finally:

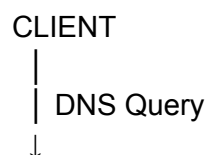


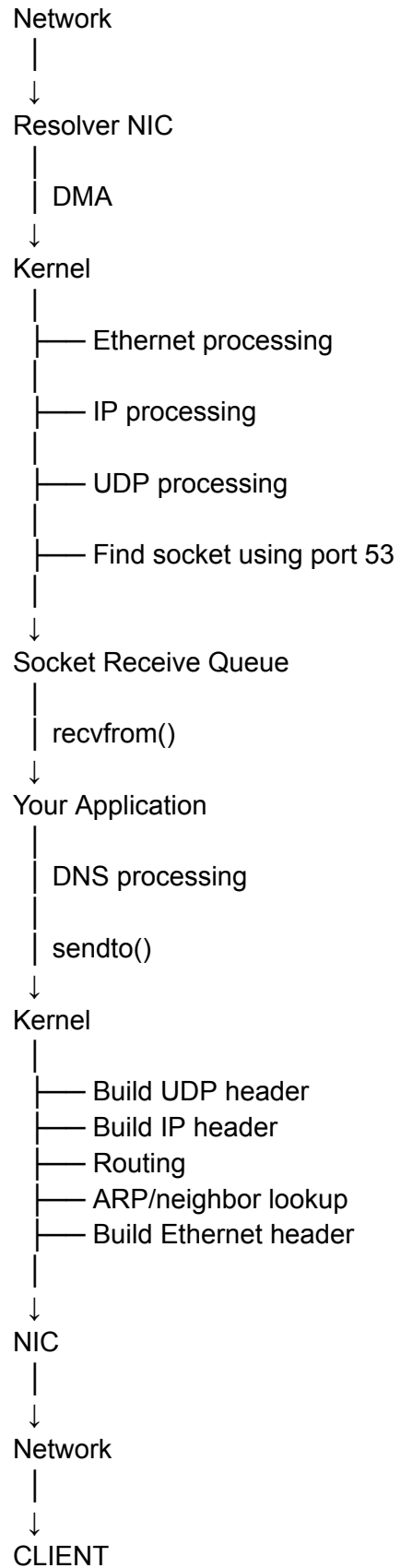
The client receives the response.

26. So the entire journey is this

This is the most important diagram to remember:

NORMAL UDP SOCKET





27. The most important distinction: Kernel vs Application

You should have this mental model before studying AF_XDP.

Normal UDP socket

Your application says:

recvfrom()

and the kernel says:

"I'll take care of the network."

Your application gets:

DNS payload

The kernel handles:

Ethernet

IP

UDP

routing

checksums

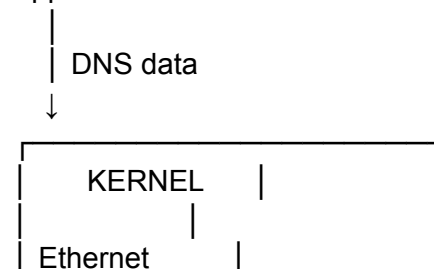
socket lookup

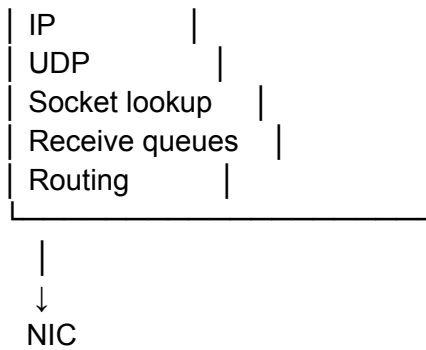
etc.

So:

NORMAL SOCKET

Application

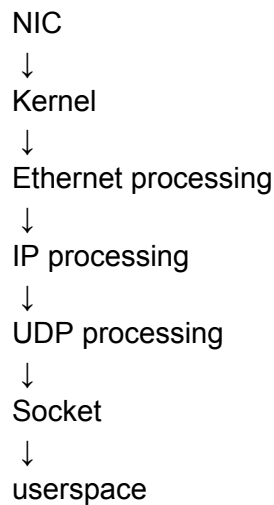




28. And THIS is why AF_XDP is interesting

Now you can understand the motivation behind your project.

Normal socket:

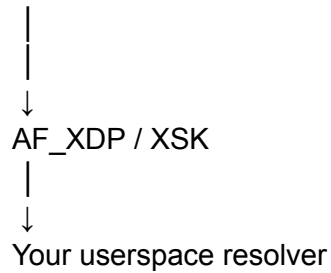


A lot of work happens in the kernel before your application gets the data.

With **AF_XDP**, we try to move packet handling much closer to userspace.

Conceptually:





Your application can work with much more of the actual packet.

For your DNS project, the application can construct:

Ethernet

+

IPv4

+

UDP

+

DNS

itself.

That's why your AF_XDP resolver is fundamentally different from:

recvfrom()
sendto()

29. Normal UDP vs AF_XDP — the big picture

Normal UDP	AF_XDP
Kernel handles packet processing	XDP can process packet very early
Application receives DNS payload	Application can access packet buffers directly
Kernel constructs UDP/IP/Ethernet headers	Application can construct raw headers
Uses normal socket API	Uses AF_XDP/XSK
<code>recvfrom()</code>	XSK RX ring

`sendto()`

XSK TX ring

Kernel abstracts network headers

Application can work with Ethernet/IP/UDP/DNS

More kernel networking overhead

Designed for high-performance packet processing

30. One analogy that makes all of this easy

Imagine a **postal system**.

Normal UDP

You are a customer sending/receiving mail.

You say:

"I want to send this letter to this address."

You don't worry about:

- envelopes
- sorting centers
- postal routes
- trucks
- destination bins

The postal system handles everything.

That's like:

`sendto()`
`recvfrom()`

The **kernel is the postal system**.

AF_XDP

Now imagine you're running a specialized mail center **inside the postal facility**.

You have much more direct access to incoming packages.

Instead of saying:

"Give me the contents of the letter."

you can say:

"Give me the actual package, and I'll inspect/build the headers myself."

That's much closer to what AF_XDP enables.

31. Finally, memorize these 8 things

For your project/interview, make sure you understand these:

1. NIC

Hardware that sends/receives network frames.

Network ↔ NIC

2. Kernel

Operating-system component that manages networking and hardware.

Application ↔ Kernel ↔ NIC

3. Socket

Kernel-managed communication endpoint.

UDP port 53 → DNS socket

4. `bind()`

Associates a socket with an address/port.

```
bind(sock, ..., 53, ...);
```

means:

"Packets destined for UDP port 53 should be delivered to this socket."

5. `recvfrom()`

Moves data from the kernel's socket receive area into your userspace buffer.

Kernel → Application

6. `sendto()`

Tells the kernel:

"Send this data to this destination."

The kernel handles the lower-level networking.

7. `sk_buff`

Linux's kernel-side representation of a network packet.

skb ≈ kernel object representing packet

8. `AF_XDP`

A high-performance Linux networking mechanism that allows userspace applications to interact with packets much earlier/more directly than the traditional socket path.

The one sentence you should remember

In a normal UDP socket, the NIC gives the packet to the kernel, the kernel processes Ethernet → IP → UDP, finds the socket using the destination port, queues the payload, and `recvfrom()` copies that payload into the application; when sending, the application gives the payload to the kernel and the kernel constructs the lower-layer headers and sends the frame through the NIC.

Once this is clear, **`AF_XDP` becomes much easier**, because the next question is simply:

"What exactly does `AF_XDP` change in this pipeline, and what are XDP, XSK, UMEM, RX ring, TX ring, Fill ring and Completion ring?"

Those are the pieces we should learn next.

Mon, Aug 31 at 2:14 PM

"Project Description: This project implements a DNS resolver that sends and receives DNS packets using an AF_XDP (XSK) socket instead of the usual UDP/TCP socket. An XDP program attached to the NIC redirects DNS traffic (UDP dst port 53) to an AF_XDP socket; a user-space resolver crafts raw Ethernet/IPv4/UDP/DNS frames, transmits them via the XSK TX ring, and receives responses on the XSK RX ring. The project demonstrates low-latency, high-throughput packet I/O, zero-copy packet handling tradeoffs, and the extra plumbing required (UMEM, rings, descriptor management) compared to conventional sockets. Tools / Technologies Linux kernel AF_XDP / XDP support (recent kernel) clang/llvm (for compiling the XDP BPF program) libbpf / libxsk (or equivalent XSK helper code) for user-space XSK setup C (user-space resolver) for packet crafting and XSK ring handling bpftool / xdp-loader (to load/attach the XDP program) tcpdump / Wireshark / pkt-gen (for validation and packet inspection) Expected Outcome: Comparison vs standard UDP sockets: measurements of latency, throughput, and CPU overhead showing AF_XDP tradeoffs (zero-copy performance benefits vs implementation complexity)." We have been assigned this project we have just started the project explain this project in detail for a person who has just started learning cn.....and give me explanation if each and every requirement of this project

Absolutely. This project is **quite advanced for someone just starting Computer Networks**, because it combines normal networking concepts with Linux kernel networking, eBPF/XDP, and high-performance packet I/O.

The good news is that you **do not need to understand everything at once**. The project becomes much easier if you understand it as a sequence:

**DNS → UDP → IP → Ethernet → XDP → AF_XDP → UMEM/Rings →
User-space packet crafting → Measurement**

I'll build it from the ground up and then map **every single requirement in your project description** to what you actually need to implement.

1. First: What are you actually building?

Your project is essentially:

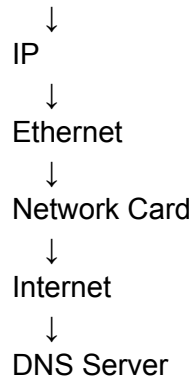
A DNS resolver that normally would use UDP sockets, but instead directly sends and receives DNS packets through an AF_XDP socket.

Normally, a DNS client does something like:

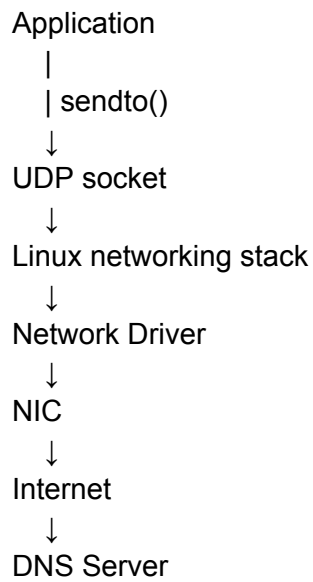
Your program

↓

UDP socket

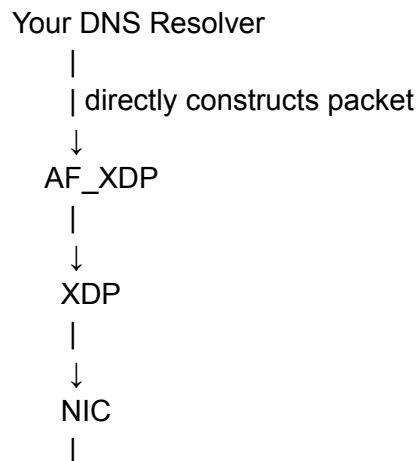


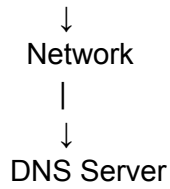
For example:



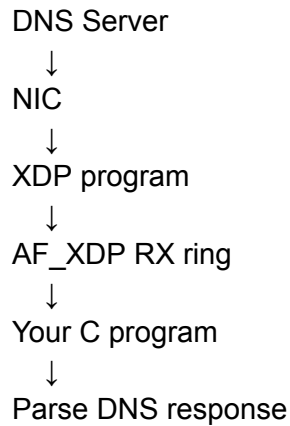
Your project changes the middle part.

Instead:





And on receiving:



So the central idea is:

Use AF_XDP as a high-performance packet I/O mechanism instead of the normal UDP socket API.

2. Before AF_XDP: What is DNS?

Since you're starting CN, start here.

DNS means:

Domain Name System

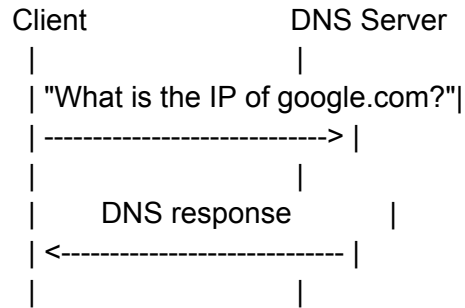
Its job is to translate:

google.com

into something like:

142.250.x.x

A simple DNS request looks like:



Usually your computer communicates with a DNS server such as:

8.8.8.8

or

1.1.1.1

3. DNS normally uses UDP

DNS commonly uses:

UDP

Port 53

So when your application asks:

google.com

a DNS query may look conceptually like:

Source IP = your IP

Destination IP = 8.8.8.8

Source port = random temporary port

Destination port = 53

DNS payload:

Query ID

Flags

Question

...

The important thing for your project is:

UDP destination port = 53

Your XDP program will use this to recognize DNS traffic.

4. What is a network packet?

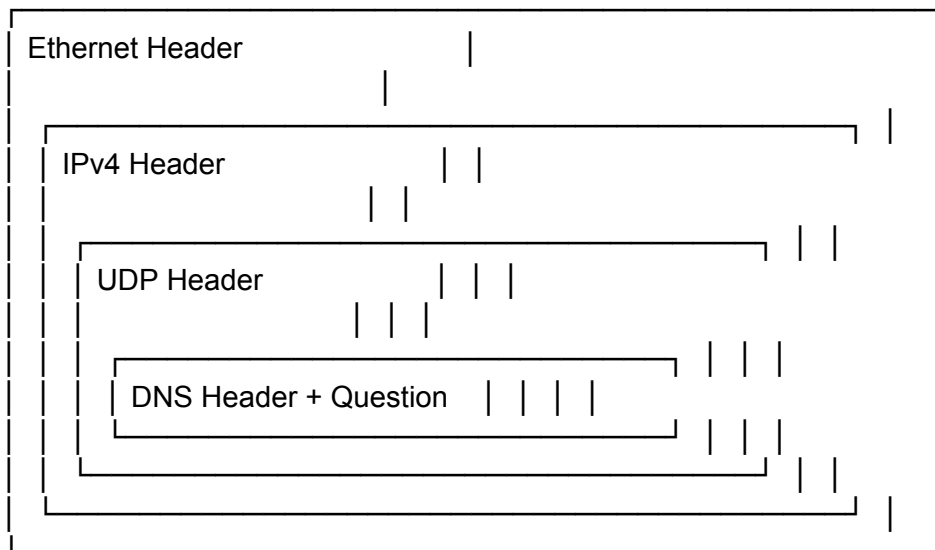
This is one of the most important concepts you need before starting implementation.

When you send a DNS query, you're not simply sending:

"Give me the IP of google.com"

The information is wrapped in multiple protocol headers.

Think of it like envelopes:



Therefore:

Ethernet



IPv4



UDP



DNS

This is extremely important because your program will **construct this entire packet manually**.

5. What normally happens when sending UDP?

Suppose you write:

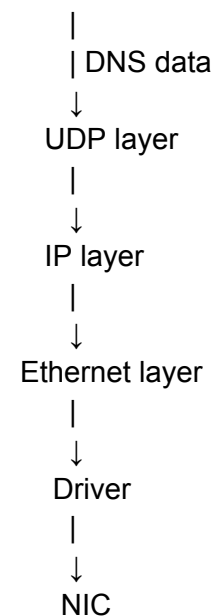
```
sendto(sockfd, buffer, length, ...);
```

You don't construct Ethernet/IP/UDP headers yourself.

Linux does a lot of work for you.

Conceptually:

Your application



Linux handles:

- UDP header
- IP header
- Ethernet framing
- routing
- checksums
- socket buffers

- much of the packet processing

This is convenient.

6. What changes with AF_XDP?

AF_XDP gives user space a much more direct way to interact with packets.

Instead of saying:

sendto()

you essentially give the networking subsystem a memory buffer containing a complete packet.

For example:

[Ethernet][IPv4][UDP][DNS]

Your program creates it.

Then AF_XDP/XDP machinery handles moving that packet between your program and the NIC.

So:

Traditional:

Application
↓
UDP socket
↓
Kernel network stack
↓
NIC

versus:

Your project:

User-space resolver
↓
AF_XDP
↓
XDP

↓
NIC

The second approach can reduce overhead, especially for very high packet rates.

7. What exactly is XDP?

XDP stands for:

eXpress Data Path

It is a Linux mechanism that allows a small BPF program to run **very early in the packet receive path**.

Very roughly:

NIC
↓
Driver
↓
XDP program
↓
Normal networking stack

The XDP program can inspect a packet and decide what to do.

For example:

XDP program

packet arrives

```
|  
+---- DNS? -----> redirect to AF_XDP  
|  
+---- something else ---> normal networking stack
```

Your project specifically says:

"An XDP program attached to the NIC redirects DNS traffic (UDP dst port 53) to an AF_XDP socket."

That is one of your core requirements.

8. Why does the XDP program care about port 53?

Suppose a packet arrives:

Ethernet



IPv4



UDP



destination port = 53

Your XDP program checks:

Is this UDP?



YES



Is destination port 53?



YES



Redirect to AF_XDP

For other packets:

HTTP

SSH

ICMP

TCP

etc.

the XDP program does not redirect them to your DNS resolver.

9. What is AF_XDP?

This is the heart of the project.

AF_XDP is a Linux socket address family designed for:

high-performance packet processing from user space.

Traditional socket:

```
socket(AF_INET, SOCK_DGRAM, 0);
```

AF_XDP:

```
socket(AF_XDP, SOCK_RAW, 0);
```

The exact setup is significantly more complicated.

AF_XDP provides a mechanism where packet buffers are shared between user space and the kernel/network driver.

That is where:

UMEM

comes in.

10. What is UMEM?

UMEM is basically a region of memory that AF_XDP uses for packet buffers.

Imagine you allocate:

UMEM

frame 0	
frame 1	
frame 2	
frame 3	
...	

Each frame can contain a packet.

For example:

Frame 0:

[Ethernet][IP][UDP][DNS]

Frame 1:

[Ethernet][IP][UDP][DNS]

Frame 2:

empty

Frame 3:

response packet

The important concept:

UMEM is the shared packet-buffer memory used by AF_XDP.

11. Why do we need UMEM?

Because copying packets around is expensive.

Imagine:

NIC

↓

Kernel buffer

↓

copy

↓

user buffer

That's additional work.

AF_XDP is designed to allow more efficient packet movement.

In some configurations it can use:

zero-copy

where packet buffers can be used with minimal/no copying between the NIC and user-space-visible memory.

But this requires hardware/driver/kernel support.

That's why your project description specifically says:

"zero-copy performance benefits vs implementation complexity"

You need to investigate and measure this tradeoff.

12. What does "zero-copy" mean?

Suppose the NIC receives a packet:

DNS response

Traditional path may involve copying data between buffers.

With a zero-copy-capable AF_XDP setup, the packet can be placed into a buffer from the AF_XDP UMEM and consumed by user space without the traditional extra packet-copy step.

Conceptually:

Traditional

NIC

↓

Kernel buffer

↓ copy

Application buffer

AF_XDP zero-copy

NIC

↓

UMEM buffer

↓

User space

This can reduce CPU overhead and latency.

But:

AF_XDP does not automatically mean zero-copy.

There are different operating modes, and the actual behavior depends on driver/hardware/kernel support.

13. What are rings?

AF_XDP uses rings to communicate packet ownership/descriptors between kernel and user space.

You need to understand four major rings:

FILL
COMP
RX
TX

These are critical to your project.

14. RX ring

RX means:

Receive

When a packet arrives and XDP redirects it to your AF_XDP socket:

NIC
↓
XDP
↓
AF_XDP RX ring
↓
Your program

The RX ring tells your program:

"There is a packet available at this location in UMEM."

The descriptor essentially tells you where the packet is.

Conceptually:

RX descriptor

address = 4096

length = 74

Meaning:

UMEM + 4096

contains

74 bytes

Your C program then reads:

Ethernet

IPv4

UDP

DNS

from that buffer.

15. TX ring

TX means:

Transmit

When your resolver wants to send a DNS query:

Your program



create packet in UMEM



TX ring



AF_XDP



NIC

You put a descriptor into the TX ring saying essentially:

UMEM offset = X
packet length = Y

The networking subsystem/NIC then transmits that packet.

16. FILL ring

The FILL ring provides buffers that can be used for receiving packets.

Think:

FILL ring

"Here are some free UMEM buffers.
You can use these for incoming packets."

For example:

FILL

↓

frame 0
frame 1
frame 2
frame 3

The kernel/driver can use those buffers for incoming packets.

17. COMPLETION ring

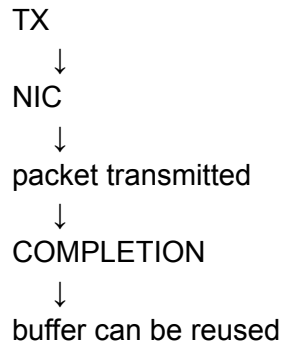
After a transmitted packet is completed, the completion ring tells user space:

"This TX buffer is available again."

Conceptually:

Your program

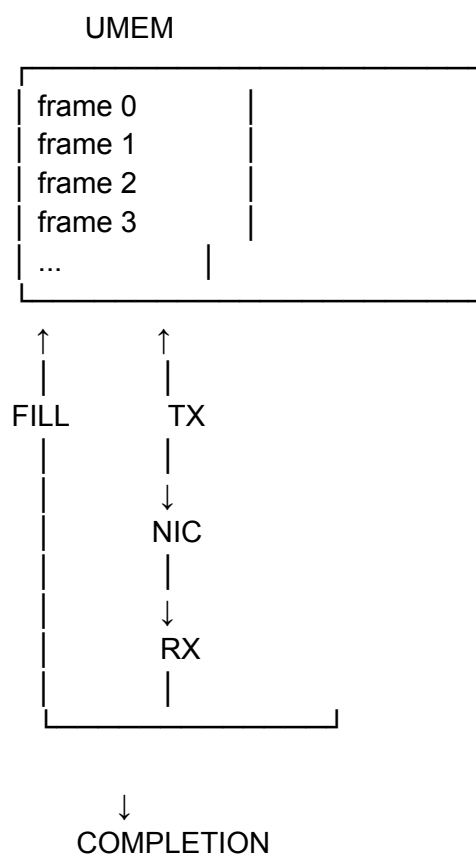
↓



Without properly recycling buffers, eventually you'll run out of UMEM frames.

18. Put the four rings together

This is one diagram you should eventually be able to draw in your viva:



More accurately, ownership moves between these components, but this mental model is sufficient initially.

19. What does "XSK" mean?

XSK is commonly used to refer to an:

AF_XDP socket

So you'll see terminology like:

XDP
AF_XDP
XSK
XSK socket

Don't get confused.

Your project essentially uses:

XDP program
+
AF_XDP/XSK socket
+
UMEM
+
RX/TX rings

20. Your XDP program

Your XDP program is likely a small C program compiled into eBPF bytecode using:

clang/LLVM

Its basic job:

1. Receive packet
2. Check Ethernet
3. Check IPv4
4. Check UDP
5. Check destination port
6. If port = 53:
 - redirect packet to AF_XDP

7. Otherwise:
 - choose an appropriate non-redirect action

Conceptually:

```
if (packet_is_ipv4 &&
    packet_is_udp &&
    udp_dest_port == 53)
{
    redirect_to_xsk();
}
```

That's the basic logic.

21. Why do we need clang/LLVM?

Your XDP program is not just an ordinary executable.

You write something like:

```
SEC("xdp")
int dns_filter(...)
{
    ...
}
```

Then compile it into BPF object code.

A typical compilation pipeline is:

XDP C source

↓

clang

↓

BPF object (.o)

↓

load/attach

↓

NIC

So:

clang/LLVM = compiler toolchain for the XDP BPF program.

22. What is BPF/eBPF?

You will encounter this constantly.

BPF is a technology that allows small programs to run inside the Linux kernel in controlled execution environments.

eBPF is the modern evolution of BPF.

XDP programs are one important application of eBPF.

So the hierarchy is roughly:

```
eBPF
↓
XDP
↓
Packet processing
```

Don't think of eBPF and XDP as exactly the same thing.

23. What is libbpf?

libbpf is a library for working with BPF programs.

It can help with:

- loading BPF objects
- attaching programs
- maps
- BPF program lifecycle
- interaction with kernel BPF facilities

You may use it to load/manage your XDP program.

24. What is libxsk?

`libxsk` is associated with AF_XDP functionality.

It provides helper APIs for things such as:

- UMEM setup
- XSK creation
- ring management
- packet descriptors
- interaction with AF_XDP

Instead of implementing every low-level detail yourself, you can use these helpers.

Your project says:

```
"libbpf / libxsk (or equivalent XSK helper code)"
```

So your team needs to decide whether to use the libraries directly or an equivalent helper implementation.

For a first implementation, **using the established libraries is much more practical.**

25. What is bpftool?

`bpftool` is a Linux utility for inspecting/managing BPF.

It can help you investigate:

loaded BPF programs
BPF maps
links
etc.

For your project, it's useful for debugging questions like:

Did my XDP program actually load?

Is it attached?

Is the kernel seeing it?

So:

```
clang
↓
compile XDP
↓
bpftool/libbpf
↓
load/attach
↓
NIC
```

26. What is xdp-loader?

`xdp-loader` is another tool commonly used to load/attach XDP programs.

Your project specifically mentions:

bpftool / xdp-loader

This means these are possible ways of getting your XDP program attached to the network interface.

You don't necessarily need to use both.

27. What is the NIC?

NIC means:

Network Interface Card/Controller

Examples:

Ethernet adapter
Wi-Fi adapter

For AF_XDP/XDP development, the interface and its driver matter a lot.

For example:

eth0
ens33
enp3s0

Your project needs an interface on which the XDP program can be attached.

You can discover interfaces using commands such as:

ip link

28. A major practical issue: Wi-Fi vs Ethernet

This is important.

AF_XDP/XDP behavior and zero-copy support depend heavily on the network driver.

You shouldn't assume:

"I have Linux, therefore zero-copy will work."

It may not.

You need to identify:

Network interface



Driver



XDP support



AF_XDP support



zero-copy support

Therefore, one of the first project tasks should be:

Determine which NIC/interface and driver you are actually going to use.

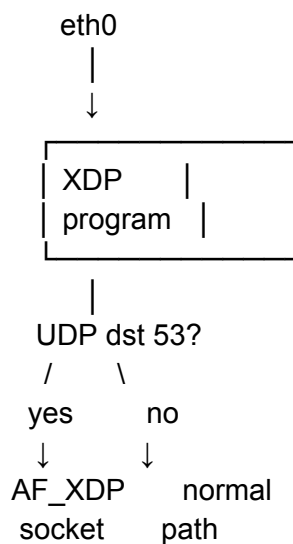
29. What exactly does "attached to the NIC" mean?

Suppose your interface is:

eth0

Your XDP program gets attached to that interface.

Conceptually:



30. How does the receive path work?

This is probably the most important flow in your project.

Suppose the DNS server sends:

DNS response

The NIC receives it.

Step 1

NIC receives Ethernet frame.

Ethernet

↓
IPv4
↓
UDP
↓
DNS

Step 2

XDP runs.

It checks:

IPv4?
UDP?
destination port 53?

If yes:

XDP_REDIRECT

Step 3

Packet goes to AF_XDP.

XDP
↓
XSK
↓
RX ring

Step 4

Your C program checks the RX ring.

It gets a descriptor:

address
length

Step 5

Your program finds that packet in UMEM.

Step 6

Your program parses:

Ethernet header

IPv4 header

UDP header

DNS response

Step 7

You extract the answer.

For example:

google.com

↓

142.250.xxx.xxx

31. What does "user-space resolver crafts raw Ethernet/IPv4/UDP/DNS frames" mean?

This sentence is extremely important.

It means your C program isn't simply doing:

```
sendto(sockfd, dns_data, ...);
```

Instead, you construct something like:

Ethernet Header

+

IPv4 Header

+

UDP Header

+

DNS Header

+

DNS Question

in a memory buffer.

32. Ethernet header

The Ethernet header contains things such as:

Destination MAC

Source MAC

EtherType

Conceptually:

Destination MAC	
Source MAC	
EtherType = IPv4	

Usually IPv4 EtherType is:

0x0800

Your resolver needs to construct this correctly.

33. IPv4 header

Then:

Ethernet

↓

IPv4

IPv4 header includes fields such as:

Source IP

Destination IP

TTL

Protocol

Total Length

Identification
Flags
Fragment offset
Header checksum
...

For UDP:

Protocol = 17

So your packet says:

IPv4 protocol = UDP

34. UDP header

UDP header contains:

Source port
Destination port
Length
Checksum

For DNS:

Destination port = 53

For example:

Source port = 40000
Destination port = 53

35. DNS header

Then you create the DNS message.

A DNS header contains fields such as:

Transaction ID
Flags
Question count

Answer count
Authority count
Additional count

Then comes the question section.

For:

google.com

you need DNS wire-format encoding.

It's not simply ASCII:

google.com

It is encoded using DNS labels, conceptually:

06 google
03 com
00

Then:

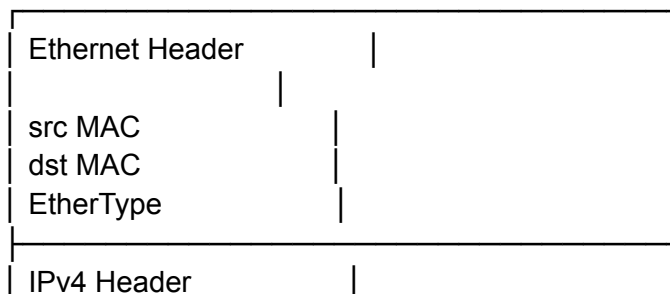
QTYPE
QCLASS

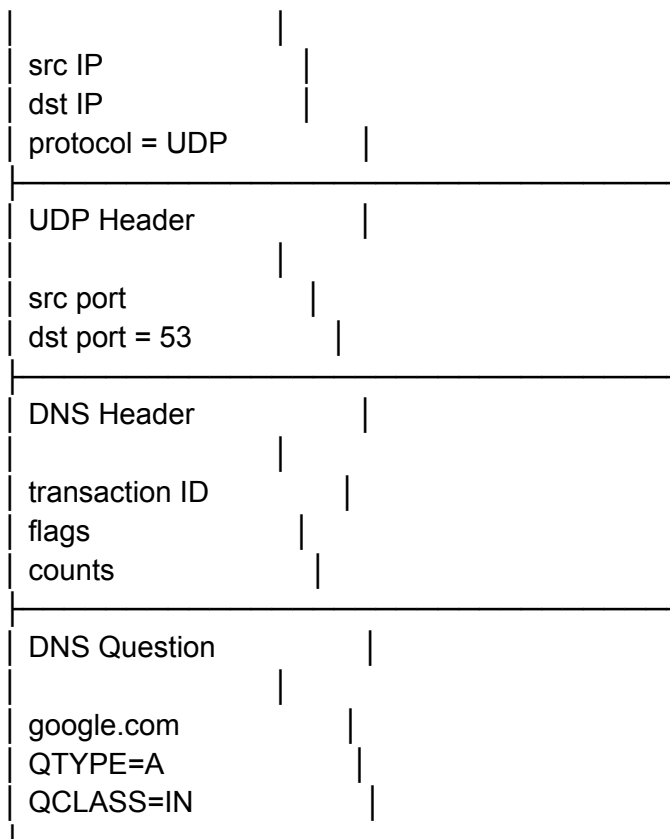
For an IPv4 address lookup:

QTYPE = A
QCLASS = IN

36. So your outgoing packet looks like

Conceptually:





That's what "crafting raw frames" means.

37. How does transmission work?

Suppose you have created the packet in UMEM:

UMEM frame:

[Ethernet][IPv4][UDP][DNS]

You then create/populate an entry in the TX ring.

Conceptually:

UMEM



packet at offset 4096

TX descriptor



Then you need to deal with TX completion and recycle the buffer.

38. Why not simply use raw sockets?

A very reasonable question.

You could construct raw Ethernet frames using other mechanisms.

But the project specifically wants:

AF_XDP

because it demonstrates high-performance packet I/O.

The educational goal isn't merely:

"Can I create a DNS packet?"

It's:

"Can I build a high-performance user-space packet-processing pipeline using XDP + AF_XDP?"

39. What does "low-latency, high-throughput packet I/O" mean?

Two important performance metrics:

Latency

How long does one operation take?

For example:

DNS query sent

↓

DNS response received

If:

t_1 = query transmission

t_2 = response received

then approximately:

latency = $t_2 - t_1$

You should be careful about exactly what portion you measure.

40. Throughput

Throughput asks:

How many packets/data can we process per second?

For example:

100,000 packets/sec

or:

1 Gbps

For your project you can compare:

UDP socket implementation

vs

AF_XDP implementation

41. CPU overhead

This asks:

How much CPU work is required to achieve the same networking workload?

Suppose:

UDP:

80% CPU

AF_XDP:

50% CPU

at the same workload.

That would suggest AF_XDP is more CPU-efficient for that workload.

But you should actually measure rather than assume.

42. Why might AF_XDP be faster?

Potential advantages include:

1. Reduced networking-stack overhead

You bypass substantial portions of the conventional socket/networking path.

2. Efficient buffer management

UMEM allows packet buffers to be managed explicitly.

3. Batch processing

AF_XDP can process packets in batches.

4. Potential zero-copy

On supported drivers/configurations.

5. User-space packet processing

Your application has much more direct control over packets.

43. But why doesn't everyone use AF_XDP?

Because it's much harder.

Traditional:

```
socket()  
bind()  
sendto()  
recvfrom()
```

AF_XDP requires things like:

```
UMEM  
RX ring  
TX ring  
FILL ring  
COMPLETION ring  
XDP program  
XSK  
interface binding  
frame management  
descriptor management  
buffer recycling  
packet parsing  
checksums  
driver compatibility
```

That's exactly what your project means by:

"zero-copy performance benefits vs implementation complexity."

44. What is descriptor management?

A descriptor is essentially metadata describing where a packet is located and how large it is.

Imagine UMEM:

Frame 0 → offset 0

Frame 1 → offset 2048

Frame 2 → offset 4096

Frame 3 → offset 6144

A descriptor could say:

address = 4096

length = 74

Meaning:

UMEM[4096 ... 4169]

contains the packet.

Your program has to correctly manage:

Which frame is free?

Which frame is being transmitted?

Which frame contains a received packet?

Which frame can be reused?

That's descriptor/buffer management.

45. A useful analogy for the rings

Imagine a restaurant.

UMEM = kitchen tables.

Table 1

Table 2

Table 3

Table 4

FILL ring:

"Here are empty tables available for incoming orders."

RX ring:

"These tables now contain completed incoming orders."

TX ring:

"These tables contain food ready to be delivered."

COMPLETION ring:

"These delivered tables are free again."

It's basically a controlled buffer-passing system.

46. What does "libxsk or equivalent XSK helper code" mean?

You have two broad choices.

Option A — Use libxsk

You use the existing AF_XDP library APIs.

Advantages:

- Less code
- Less low-level complexity
- Fewer mistakes

Option B — Implement more yourself

You directly interact with the AF_XDP mechanisms.

Advantages:

- More understanding
- More control

Disadvantages:

- Much more complicated
- More bugs

More debugging

For your project, I'd recommend:

Start with libbpf/libxsk.

Once the basic system works, understand what the libraries are doing internally.

47. What exactly should the DNS resolver do?

Your resolver can initially be very simple.

For example:

```
./dns_xsk google.com
```

Your program:

1. Initialize AF_XDP
2. Initialize UMEM
3. Initialize rings
4. Load/attach XDP
5. Create DNS query
6. Build Ethernet header
7. Build IPv4 header
8. Build UDP header
9. Build DNS payload
10. Put packet into UMEM
11. Put descriptor into TX ring
12. Send
13. Wait for RX packet
14. Read RX descriptor
15. Parse packet
16. Parse DNS response
17. Print result

For example:

Query: google.com

Answer:
142.250.xxx.xxx

Latency:
X ms

48. Do you need to implement a full recursive DNS server?

Probably **no**, based on the project description.

There's an important distinction:

DNS resolver/client

You
↓
DNS server

asks another DNS server for an answer.

Recursive DNS server

Client
↓
Your server
↓
Root DNS
↓
TLD DNS
↓
Authoritative DNS

Your description says:

"DNS resolver"

and describes crafting DNS requests and receiving responses.

So your likely target is a **DNS client/resolver**, not a full recursive DNS server.

Your instructor should confirm the exact expected resolver behavior.

49. What DNS server should you use?

For development, you could use a known resolver such as:

8.8.8.8

or:

1.1.1.1

But your exact environment may affect routing, firewalling, and connectivity.

The important thing is that your outgoing packet needs:

Destination IP = DNS server IP

Destination UDP port = 53

50. One subtle problem: Ethernet destination MAC

This is something beginners often miss.

Suppose:

Your IP:

192.168.1.10

DNS server:

8.8.8.8

The destination isn't on your local Ethernet network.

Your computer sends the Ethernet frame to the:

MAC address of the next-hop router/default gateway

not directly to the MAC address of 8.8.8.8.

So you need to understand:

IP routing

+

ARP

+

Ethernet

Your raw packet must use an appropriate destination MAC.

This is one reason raw packet crafting is significantly harder than UDP sockets.

51. ARP

ARP means:

Address Resolution Protocol

Suppose your gateway is:

192.168.1.1

Your machine needs to know:

192.168.1.1 → AA:BB:CC:DD:EE:FF

That's ARP's job.

You can inspect the neighbor table using Linux networking commands.

For the first version of your project, you may leverage the existing system routing/neighbor information rather than implementing ARP yourself.

52. IP routing

You also need to know:

Where should this packet go?

Your machine has a routing table.

For example:

Destination	Gateway
0.0.0.0/0	192.168.1.1

Meaning:

For destinations I don't know directly, send through the gateway.

Your raw packet still needs the correct:

source IP

destination IP

Ethernet destination MAC

53. Checksums

This is another important requirement hidden inside:

"crafts raw Ethernet/IPv4/UDP/DNS frames"

Some headers contain checksums.

IPv4 has:

IPv4 header checksum

UDP has:

UDP checksum

If you create packets manually, you need to ensure these are correct.

Otherwise the packet may be dropped or rejected.

This is an excellent part of the project to understand because it connects your CN theory directly to implementation.

54. What does tcpdump do?

`tcpdump` is a packet-capture tool.

It allows you to see traffic such as:

IP 192.168.1.10.45000 > 8.8.8.8.53:
UDP

You can use it to answer:

Did my program actually transmit a DNS packet?

This is extremely useful during debugging.

55. Wireshark

Wireshark is a graphical packet analyzer.

It can decode:

Ethernet
IPv4
UDP
DNS

So instead of staring at bytes like:

45 00 00 48 ...

you can see:

Internet Protocol Version 4

Source: ...
Destination: ...

User Datagram Protocol

Source Port: ...
Destination Port: 53

Domain Name System

Query:
google.com

For your project, Wireshark will be very useful for validating packet construction.

56. pkt-gen

`pkt-gen` is useful for generating large numbers of packets.

Why?

Because testing one DNS packet doesn't demonstrate:

high throughput.

You may eventually want to test:

1,000 packets/sec

10,000 packets/sec

100,000 packets/sec

...

and compare your implementations.

The exact generator/toolchain you use should depend on what your instructor expects and what your NIC supports.

57. The standard UDP baseline

This is very important.

Your project isn't just:

"Make AF_XDP DNS resolver."

It says:

"Comparison vs standard UDP sockets."

Therefore you need **two implementations**.

Implementation A

Normal UDP:

DNS program
↓
UDP socket
↓
Linux networking stack
↓
NIC

Implementation B

AF_XDP:

DNS program
↓
AF_XDP
↓
XDP
↓
NIC

Then compare them.

58. Why do you need a baseline?

Suppose your AF_XDP program takes:

100 μ s

Is that good?

You can't tell.

But if:

UDP socket = 150 μ s

AF_XDP = 100 μ s

now you can say:

AF_XDP reduced the measured latency by approximately 33% in this experimental setup.

That's a meaningful result.

59. What should you measure?

Your project explicitly mentions:

1. Latency

Measure:

request → response

Potential metric:

average latency

median latency

p95

p99

I'd strongly recommend reporting at least:

Average

Median

p95

p99

because averages can hide spikes.

60. Throughput

Measure:

packets/sec

and possibly:

bits/sec

For example:

UDP:

80,000 packets/sec

AF_XDP:
150,000 packets/sec

The exact numbers will depend heavily on your hardware and experiment.

Do not assume AF_XDP will automatically win for every workload.

61. CPU overhead

Measure CPU usage while generating the same workload.

For example:

	CPU
UDP	65%
AF_XDP	42%

You can use Linux performance tools such as:

top
htop
pidstat
perf

depending on what your project allows.

62. Memory usage

Although not explicitly required, it can be useful to report:

UMEM size
number of frames
frame size
application memory

because AF_XDP requires explicit memory management.

63. What exactly does "zero-copy performance benefits vs implementation complexity" mean?

Your final report should not merely say:

AF_XDP is faster.

Instead, you should discuss:

Benefit

Potentially lower latency
Potentially higher packet rate
Potentially lower CPU overhead

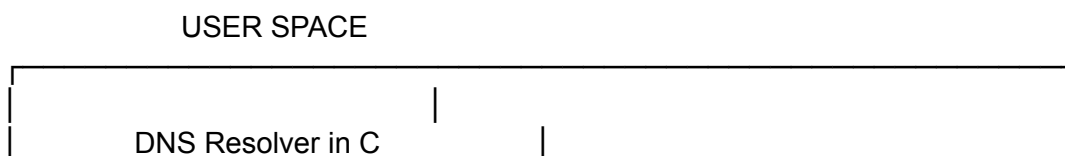
Cost

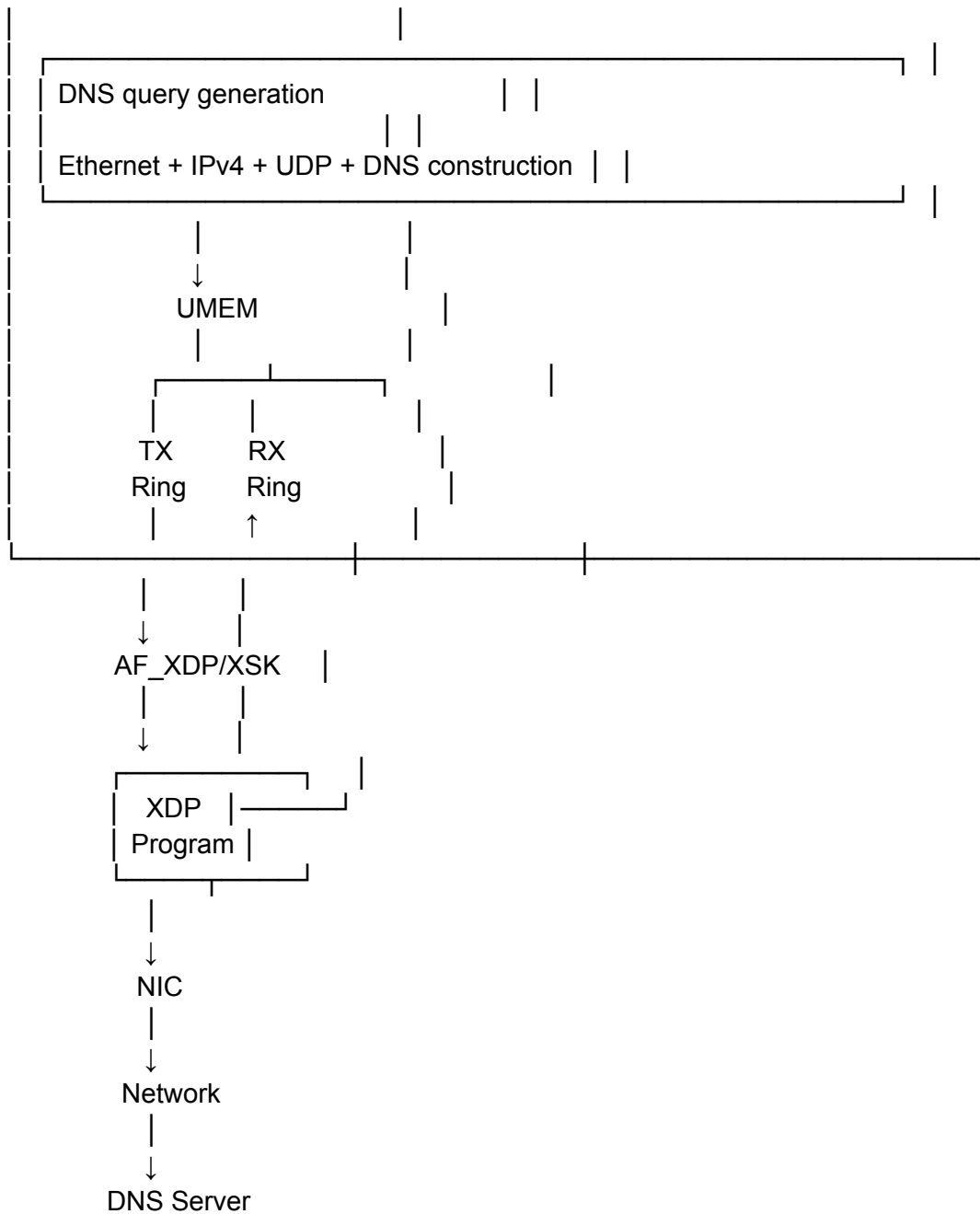
More complicated initialization
UMEM management
Ring management
XDP program
Driver dependencies
Manual packet construction
Manual buffer recycling
Harder debugging

That's one of the main educational outcomes.

64. Your complete architecture

Here's the architecture you should understand:





65. Now let's map EVERY requirement from your project description

Your description says:

Linux kernel AF_XDP / XDP support (recent kernel)

Meaning

Your system needs Linux with support for:

XDP
AF_XDP
BPF

You'll need to verify:

`uname -r`

and inspect your NIC/interface and driver.

66. clang/llvm

Requirement:

"clang/llvm (for compiling the XDP BPF program)"

You need:

XDP C source

↓

clang

↓

BPF object

You'll write an XDP program in C.

67. libbpf / libxsk

Requirement:

"libbpf / libxsk (or equivalent XSK helper code)"

Purpose:

libbpf

↓
manage/load BPF/XDP

libxsk

↓
AF_XDP setup
UMEM
rings
XSK

These greatly reduce boilerplate.

68. C user-space resolver

Requirement:

"C (user-space resolver) for packet crafting and XSK ring handling"

This is your main program.

It needs to:

initialize XSK
initialize UMEM
manage rings
construct DNS query
construct network headers
transmit packet
receive response
parse DNS
display answer

69. bpftool / xdp-loader

Requirement:

"to load/attach the XDP program"

You need a mechanism to go:

xdp_program.o



NIC interface

These tools can help you do that.

70. tcpdump / Wireshark

Requirement:

"for validation and packet inspection"

Use these to verify:

Outgoing

Source IP correct?

Destination IP correct?

Source port correct?

Destination port = 53?

DNS question correct?

Checksums correct?

Incoming

Response reaches machine?

XDP catches it?

DNS answer correct?

71. pkt-gen

Requirement:

"for validation"

Use traffic generation to test:

packet rate

throughput

CPU usage

especially when moving beyond a single request.

72. Expected Outcome

Your project says:

"Comparison vs standard UDP sockets"

Therefore final project should have:

UDP Socket	AF_XDP
-----	-----

Latency

Throughput

CPU usage

Complexity

You should perform controlled experiments.

73. A good experimental design

Don't simply run:

UDP once

AF_XDP once

and compare.

Instead:

Test 1

1000 requests

Test 2

5000 requests

Test 3

10000 requests

Test 4
higher packet rate

Record:

Latency
Throughput
CPU
Packet loss

74. Important: DNS itself can contaminate your measurements

This is a subtle experimental problem.

Suppose you measure:

DNS query → DNS response

Your result includes more than AF_XDP processing.

It includes:

Your program

+

network propagation

+

router

+

DNS server processing

+

network propagation back

+

your program

So if your goal is to measure **packet I/O overhead**, Internet DNS is noisy.

For serious benchmarking, consider designing a controlled setup, such as a local DNS responder/test endpoint or a controlled packet generator, while separately demonstrating real DNS resolution.

This distinction can make your final project much stronger.

75. Two different experiments

I would recommend separating:

Experiment A — Functional DNS

Goal:

Does the AF_XDP resolver actually resolve domains?

google.com



DNS server



answer

Experiment B — Performance

Goal:

How efficiently can our packet-processing pipeline handle traffic?

Use controlled traffic generation.

This lets you make a much cleaner performance comparison.

76. Project phases

Do **not** start by writing the entire AF_XDP DNS resolver.

That will become extremely difficult.

Instead, build it incrementally.

Phase 0 — Learn the networking basics

Before coding, understand:

Ethernet
MAC address
IPv4
IP address
routing
ARP
UDP
ports
DNS
packet
checksum

You don't need mastery.

You need to know what each header means.

Phase 1 — Make a normal DNS resolver

First write a simple C program using UDP sockets.

Conceptually:

```
socket(AF_INET, SOCK_DGRAM, 0);
```

Then:

create DNS query

↓

```
sendto()
```

↓

```
recvfrom()
```

↓

parse response

This gives you the baseline.

Phase 2 — Capture it

Run:

tcpdump

or Wireshark.

Observe:

Ethernet

IPv4

UDP

DNS

This is extremely valuable.

You should be able to point to each field and say:

"This is the Ethernet header."

"This is the IP header."

"This is UDP."

"This is DNS."

Phase 3 — Learn XDP separately

Don't immediately combine DNS and AF_XDP.

Write a tiny XDP program that does something simple.

For example:

Receive packet

↓

check protocol

↓

print/count/drop/pass

Then attach it to an interface.

Understand:

XDP program

XDP hook

XDP actions

Phase 4 — Learn AF_XDP separately

Build the smallest possible AF_XDP example.

Goal:

Create UMEM



Create XSK



Set up rings



Receive packet



print packet length

Don't worry about DNS yet.

Phase 5 — Redirect packets

Now combine:

XDP

+

AF_XDP

Your XDP program:

UDP dst port 53



redirect



AF_XDP RX

Your user-space program:

RX packet



inspect it



print headers

At this point you've achieved an important milestone.

Phase 6 — Receive a real DNS packet

Now:

DNS server



NIC



XDP



AF_XDP



C program

Your program should successfully parse:

Ethernet

IPv4

UDP

DNS

and print:

DNS response received

Phase 7 — Transmit manually

Now implement:

Ethernet

IPv4
UDP
DNS

construction.

Put it into:

UMEM

Then:

TX ring

↓

NIC

↓

DNS server

This is probably one of the hardest stages.

Phase 8 — Complete DNS resolver

Now connect:

Input:
google.com

↓

DNS packet construction

↓

AF_XDP TX

↓

DNS server

↓

AF_XDP RX



DNS parsing



Output:
IP address

Phase 9 — Add UDP implementation

If you haven't already done so:

UDP DNS resolver

should produce the same functional result.

Now you have:

Resolver 1:
UDP socket

Resolver 2:
AF_XDP

Phase 10 — Benchmark

Finally:

UDP vs AF_XDP

Measure:

Latency
Throughput
CPU
Packet loss

and ideally:

p50
p95
p99

77. Suggested team/project division

If you're working in a group, you can divide the project into components.

Person 1 — DNS

DNS packet format
query generation
response parsing

Person 2 — XDP

XDP program
packet filtering
redirect

Person 3 — AF_XDP

UMEM
XSK
rings
descriptor management

Person 4 — Benchmarking

UDP baseline
AF_XDP benchmark
tcpdump/Wireshark
performance analysis

But everyone should understand the overall architecture.

78. What files might your project eventually have?

Something like:

```
dns-af-xdp/
├── Makefile
├── xdp/
│   └── dns_redirect.c
├── user/
│   ├── dns_xsk.c
│   ├── dns_packet.c
│   ├── dns_packet.h
│   ├── xsk_setup.c
│   └── xsk_setup.h
├── udp/
│   └── dns_udp.c
├── scripts/
│   ├── attach_xdp.sh
│   └── detach_xdp.sh
├── tests/
├── benchmarks/
└── README.md
```

You don't have to use exactly this structure, but separating XDP, AF_XDP, DNS, and benchmarking will make debugging much easier.

79. The most important concepts you need to learn

I'd organize your learning in this order:

Level 1 — Basic CN

OSI/TCP-IP

Ethernet

MAC

IP
UDP
ports
DNS

Level 2 — Linux networking

NIC
network interface
routing table
ARP
packet capture

Level 3 — Linux packet processing

kernel networking stack
XDP
eBPF
BPF program
XDP actions

Level 4 — AF_XDP

XSK
UMEM
RX
TX
FILL
COMPLETION
descriptors
frame recycling

Level 5 — Packet crafting

Ethernet header
IPv4 header
UDP header
DNS header
checksums

Level 6 — Performance

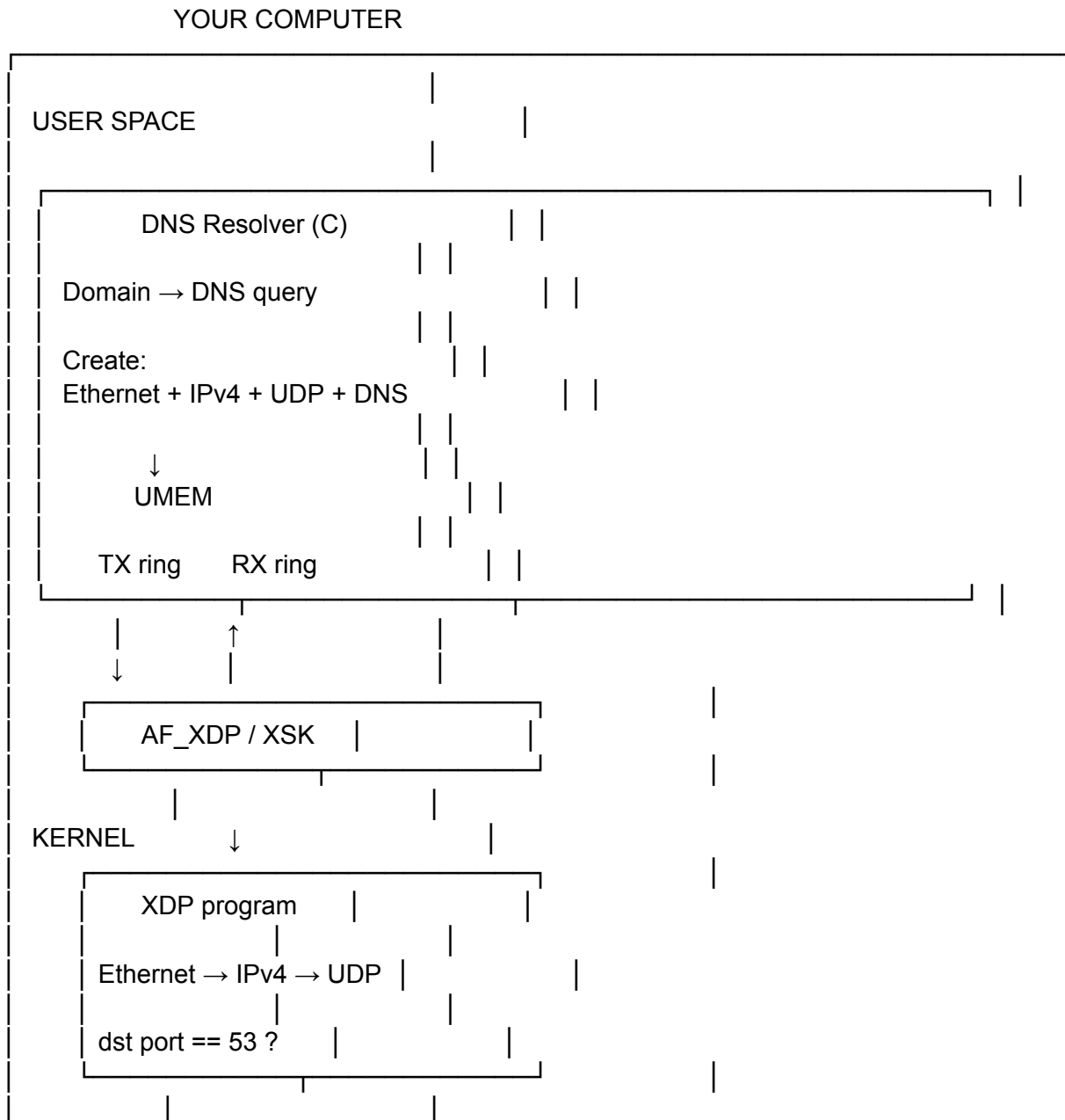
latency
throughput
CPU utilization
zero-copy
batching

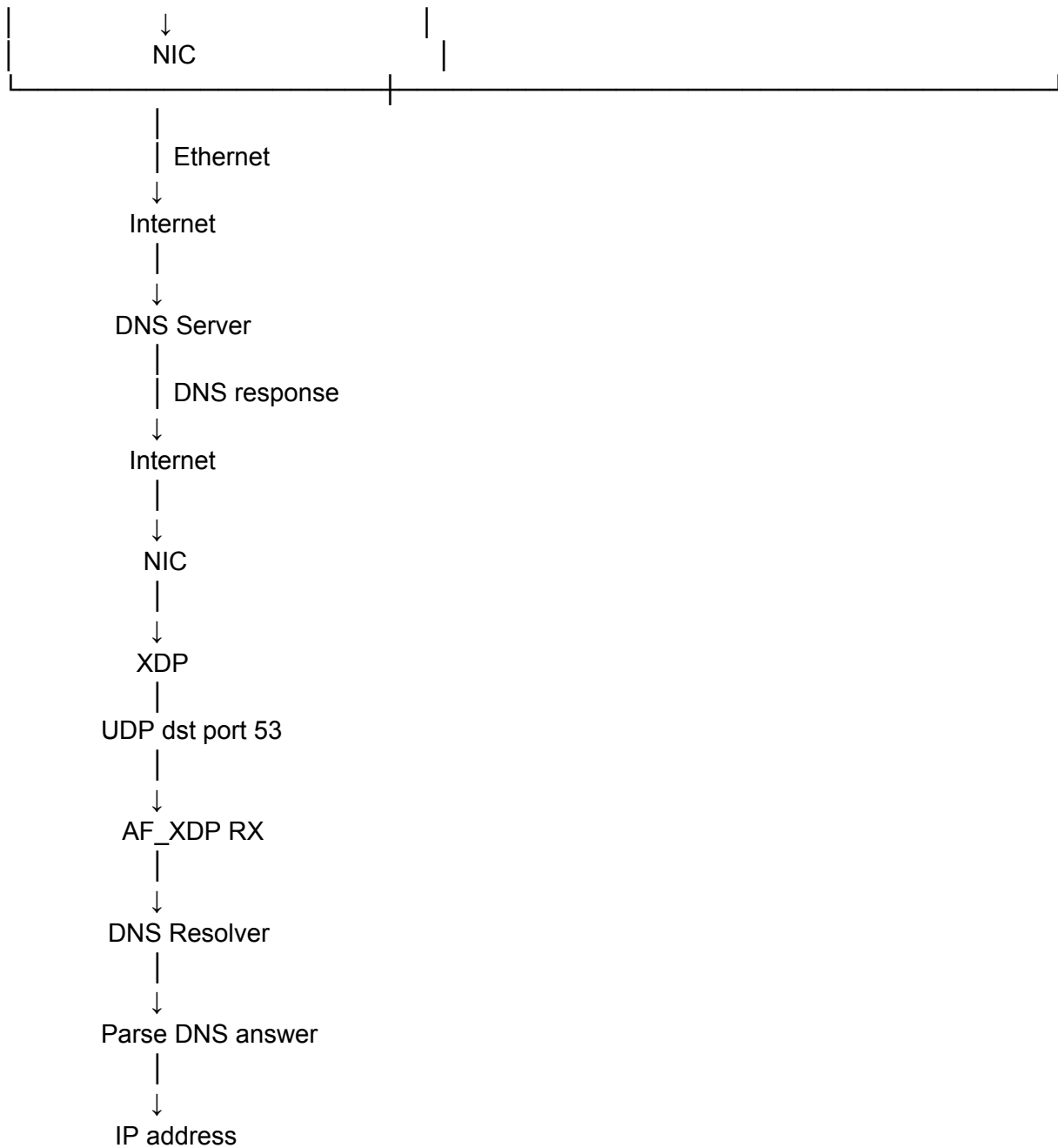
packet loss

If you follow this order, the project becomes much less intimidating.

80. The entire project in one picture

This is the picture I'd recommend keeping beside you while working:





81. What your final demonstration should look like

Ideally, at the end you should be able to demonstrate:

Terminal 1

Attach XDP:

XDP program attached to interface ...

Terminal 2

Run your resolver:

`./dns_xsk google.com`

Output:

Query: google.com

DNS response received

Answer:

142.250.xxx.xxx

Latency:

X ms

Terminal 3

Run:

`tcpdump`

and show the DNS packet.

Then

Run your standard UDP resolver:

`./dns_udp google.com`

Then compare.

82. Your final report should answer these questions

Your project report should eventually be able to answer:

Functionality

1. Can we send a DNS query using AF_XDP?
2. Can we receive the DNS response?
3. Can we parse the response?

XDP

4. How does XDP identify DNS traffic?
5. How does XDP redirect packets to the XSK?

AF_XDP

6. What is UMEM?
7. What are RX/TX/FILL/COMPLETION rings?
8. How are descriptors managed?
9. How are buffers recycled?

Packet construction

10. How is the Ethernet header constructed?
11. How is the IPv4 header constructed?
12. How is the UDP header constructed?
13. How is the DNS query constructed?
14. How are checksums handled?

Performance

15. Is AF_XDP faster?
16. Does it reduce CPU usage?
17. What happens at high packet rates?
18. Does zero-copy work on your hardware?
19. What is the latency distribution?

Tradeoffs

20. What did you gain?

21. What complexity did you introduce?

83. What you should NOT do right now

Since you've **just started**, don't immediately try to write:

`dns_xsk.c`

with 1000 lines of `AF_XDP` code.

You'll probably get errors like:

XSK creation failed

bind failed

UMEM setup failed

ring error

XDP attach failed

invalid descriptor

no packets received

and have no idea which layer is broken.

Instead:

DNS

↓

UDP

↓

packet capture

↓

XDP

↓

`AF_XDP` RX

↓

`AF_XDP` TX

↓

DNS packet crafting

↓

complete resolver

↓

benchmark

One layer at a time.

84. Your first milestone

For **right now**, I would make your first milestone much smaller:

"Understand and verify the Linux environment and successfully run a minimal XDP program."

Before touching DNS or AF_XDP, you should establish:

Linux kernel
↓
NIC/interface
↓
driver
↓
XDP support
↓
clang
↓
libbpf/libxsk
↓
minimal XDP program

Then move to AF_XDP.

The mental model to remember

If you remember only this initially, remember:

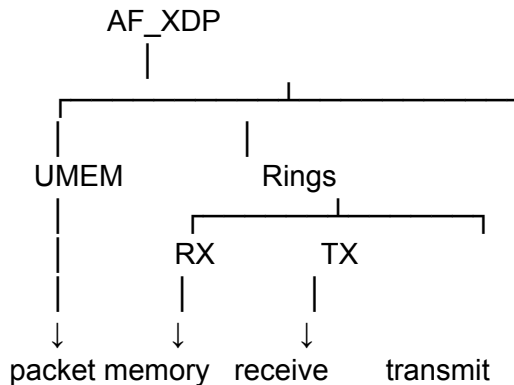
DNS
↓
UDP port 53
↓
IPv4
↓
Ethernet
↓

NIC

Your project takes that normal path and says:

"Instead of letting the normal Linux socket stack handle everything, let's intercept DNS packets very early using XDP and give them to a user-space program through AF_XDP."

And AF_XDP gives you:



Then your C program manually creates:

Ethernet

+

IPv4

+

UDP

+

DNS

and sends it through the TX path.

Finally, you compare this against:

normal UDP socket

to determine whether the extra complexity actually gives you:

lower latency

higher throughput

lower CPU overhead

under your chosen workload.

That is the entire project in one sentence:

Build a DNS resolver that bypasses the conventional UDP socket path by using XDP to redirect DNS packets into an AF_XDP user-space packet-processing pipeline, manually craft and parse the network frames, and experimentally compare its performance and complexity against a conventional UDP implementation.